2-D Unstructured Compressible Navier–Stokes Solver Benchmark Report

`cns2d` — submitted solver

August 27, 2026

Abstract

This report documents `cns2d`, an original cell-centred finite-volume solver for the two-dimensional compressible Navier–Stokes equations on mixed unstructured meshes, written in C++17 with MPI and METIS. The spatial discretization is second order through piecewise-linear inverse-distance least-squares reconstruction of the primitive variables with a Venkatakrishnan limiter; the inviscid flux is a Roe flux with a Harten–Hyman entropy fix extended to the linear waves, with HLLC and Rusanov also implemented and HLLC serving as a positivity fallback. Viscous terms use the Newtonian stress tensor and Fourier heat flux built from the same least-squares gradients. Steady cases are advanced by an implicit pseudo-time march with local time stepping and a matrix-free LU-SGS inner solve; the vortex-shedding case uses a true dual-time BDF2 scheme with an outer physical-time loop and frozen history levels. The mesh is partitioned once with METIS k-way on the cell dual graph, and iteration-time communication is strictly neighbour-scoped point-to-point halo exchange: no rank holds the full mesh or the full conservative state during iterations.

Verification is quantitative: face-closure error 1.0×10^{-16}, volume closure 1.1×10^{-10}, cell area against the boundary line integral 1.4×10^{-14}, exactness of the least-squares gradient on a linear field 3.1×10^{-13}, a discrete conservation defect of 9.2×10^{-17}, and agreement of the analytic Jacobian-vector product with central finite differences to 1.7×10^{-8} over 20 000 random states. The slip-wall flux carries exactly zero mass and energy flux for any interior state, with the normal momentum equal to the wall pressure in the limit of vanishing wall-normal velocity (eq. (23)); force coefficients agree to within 5×10^{-4} relative across $np = 1, 2, 4, 8$.

All eight required cases were run to completion with the supplied production controls, and all eight are reported by the solver as `converged` or `statistically_periodic`. Section 10 gives each with the status the solver itself recorded; no case is relabelled, and coefficients are quoted only to the precision the convergence evidence supports.

Several defects found during preparation of this report are documented rather than hidden, and the convergence criterion in particular was corrected five times as successive cases exposed distinct ways for a naive test to be fooled (section 8.5). A latent sign error in the unused HLLC star state is now covered by a dedicated regression test (section 8.6), as is the slip-wall flux property above, which an adversarial review found to have been asserted without ever being tested. Added dissipation on the linear waves, introduced to suppress a stagnation-point checkerboard mode, is disclosed and quantified in section 4.3 together with a controlled experiment showing it is *not* needed for these cases.

Contents

1 Introduction

The benchmark asks for an original, parallel, second-order-accurate finite-volume solver for the compressible Navier–Stokes equations on the supplied unstructured CGNS meshes, and for eight cases to be run to convergence with contract-compliant output. The eight required cases are listed in table 1. They span a deliberately wide range: an essentially incompressible inviscid flow whose exact drag is zero, a transonic case with a shock on the body, a supersonic case with a detached bow shock, three laminar airfoil cases at $Re = 5000$ that add a boundary layer and skin friction, a steady separated cylinder wake at $Re = 20$, and an unsteady von Kármán vortex street at $Re = 200$ that must be integrated in physical time.

This is intended as a technical report rather than a run log. Each numerical choice is stated as it is implemented in the source, and section 10 connects those choices to what the runs actually produced — including where the results fall short of the requested targets. Section numbering follows the required report outline.

Table 1: The eight required benchmark cases with the boundary-family mapping read from each case file. Family names differ between the two meshes and are never hard-coded in the solver.

Case id	Mesh	Physics	BC mapping	Max steps
`naca0012_m015_inviscid`	NACA0012_H2	inviscid, $M_\infty = 0.15$	`bc-2`→far, `bc-4`→slip	20 000
`naca0012_m080_inviscid`	NACA0012_H2	inviscid, $M_\infty = 0.80$	`bc-2`→far, `bc-4`→slip	30 000
`naca0012_m200_inviscid`	NACA0012_H2	inviscid, $M_\infty = 2.0$	`bc-2`→far, `bc-4`→slip	40 000
`naca0012_m015_laminar_re5000`	NACA0012_H2	laminar, $M_\infty = 0.15$	`bc-4`→no-slip	40 000
`naca0012_m080_laminar_re5000`	NACA0012_H2	laminar, $M_\infty = 0.80$	`bc-4`→no-slip	40 000
`naca0012_m200_laminar_re5000`	NACA0012_H2	laminar, $M_\infty = 2.0$	`bc-4`→no-slip	50 000
`cylinder_m010_laminar_re20`	CylinderB1	laminar, $Re = 20$	`WALL`→no-slip, `FAR`→far	30 000
`cylinder_m010_laminar_re200`	CylinderB1	laminar, $Re = 200$	`WALL`→no-slip, `FAR`→far	30 000

2 Governing Equations and Nondimensionalization

The solver advances the conservative state

$$\mathbf{U} = [\rho,\ \rho u,\ \rho v,\ \rho E]^T \tag{1}$$

according to the two-dimensional compressible Navier–Stokes equations in conservative form,

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}^i}{\partial x} + \frac{\partial \mathbf{G}^i}{\partial y} = \frac{\partial \mathbf{F}^v}{\partial x} + \frac{\partial \mathbf{G}^v}{\partial y}. \tag{2}$$

The inviscid fluxes are

$$\mathbf{F}^i = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho u v \\ u(\rho E + p) \end{bmatrix}, \qquad \mathbf{G}^i = \begin{bmatrix} \rho v \\ \rho u v \\ \rho v^2 + p \\ v(\rho E + p) \end{bmatrix}. \tag{3}$$

2.1 Perfect-gas closure

The gas is calorically perfect,

$$p = (\gamma - 1)\rho e, \qquad E = e + \tfrac{1}{2}\left(u^2 + v^2\right), \qquad a = \sqrt{\gamma p/\rho}, \tag{4}$$

with $T = p/(\rho R)$, total enthalpy $H = E + p/\rho$, and $c_p = \gamma R/(\gamma - 1)$. The case files set $\gamma = 1.4$, $R = 1$ and $Pr = 0.72$. Because these are case inputs rather than hard-coded constants, the temperature scale is fixed by the case: the subsonic airfoil cases have $p_\infty = 31.746$ with $\rho_\infty = 1$, hence $T_\infty = 31.746$.

2.2 Nondimensionalization and reference quantities

All cases are posed directly in nondimensional variables through the freestream: $\rho_\infty = 1$, $|\mathbf{u}_\infty| = 1$, and a pressure chosen so that the stated Mach number is reproduced, $p_\infty = 1/(\gamma M_\infty^2)$. The reference length is the chord (airfoil) or the diameter (cylinder), both unity. The solver does not assume this normalization: it reads ρ_∞, $|\mathbf{u}_\infty|$ and p_∞ from the case file, forms $a_\infty = \sqrt{\gamma p_\infty/\rho_\infty}$, and *verifies* that $|\mathbf{u}_\infty|/a_\infty$ matches the stated Mach number to a relative tolerance of 10^{-6}, aborting with a diagnostic if the case file is internally inconsistent. The freestream direction follows the angle of attack, $\hat{\mathbf{d}} = (\cos\alpha, \sin\alpha)$; all benchmark cases use $\alpha = 0$.

The dynamic pressure is $q_\infty = \frac{1}{2}\rho_\infty|\mathbf{u}_\infty|^2$ and force coefficients follow the case reference area A_{ref} and length L_{ref}:

$$C_D = \frac{\mathbf{F}\cdot\hat{\mathbf{d}}}{q_\infty A_{\text{ref}}}, \qquad C_L = \frac{\mathbf{F}\cdot\hat{\mathbf{l}}}{q_\infty A_{\text{ref}}}, \qquad C_{m,z} = \frac{M_z}{q_\infty A_{\text{ref}} L_{\text{ref}}}, \tag{5}$$

with $\hat{\mathbf{l}} = (-\sin\alpha, \cos\alpha)$ the lift direction, so a nonzero angle of attack is handled correctly. The wall distributions reported in `surface.csv` are $C_p = (p - p_\infty)/q_\infty$ and $C_f = \tau_t/q_\infty$, where τ_t is the *tangential* component of the viscous traction (section 5.5).

2.3 Viscous closure

For laminar cases the Newtonian stress tensor is

$$\tau_{xx} = 2\mu\frac{\partial u}{\partial x} - \frac{2}{3}\mu\,\nabla\cdot\mathbf{u}, \quad \tau_{yy} = 2\mu\frac{\partial v}{\partial y} - \frac{2}{3}\mu\,\nabla\cdot\mathbf{u}, \quad \tau_{xy} = \mu\left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right), \tag{6}$$

with $\nabla\cdot\mathbf{u} = \partial_x u + \partial_y v$, and the heat flux is Fourier, $\mathbf{q} = -k\nabla T$ with $k = \mu c_p/Pr$. The viscous normal flux assembled at a face with unit normal $\mathbf{n}$ is

$$\mathbf{F}^v\cdot\mathbf{n} = \begin{bmatrix} 0 \\ \tau_{xx}n_x + \tau_{xy}n_y \\ \tau_{xy}n_x + \tau_{yy}n_y \\ u\,(\tau_{xx}n_x + \tau_{xy}n_y) + v\,(\tau_{xy}n_x + \tau_{yy}n_y) + k\,\nabla T\cdot\mathbf{n} \end{bmatrix}. \tag{7}$$

All benchmark cases specify a constant viscosity, obtained directly from the Reynolds number,

$$\mu = \frac{\rho_\infty|\mathbf{u}_\infty|L_{\text{ref}}}{Re}, \tag{8}$$

so $Re = 5000$ gives $\mu = 2\times10^{-4}$, and the cylinder cases give $\mu = 5\times10^{-2}$ ($Re = 20$) and $\mu = 5\times10^{-3}$ ($Re = 200$). A Sutherland law referenced to the freestream temperature is implemented and selectable per case, but no benchmark case uses it. In inviscid mode μ and k are identically zero and the viscous face work is skipped entirely rather than multiplied by zero.

3 Meshes, Boundary Tags, and Case Inputs

3.1 CGNS input

Meshes are read through the CGNS mid-level library. The reader walks every base and every zone, accepts `Unstructured` zones only, and reads the coordinate arrays `CoordinateX`/`CoordinateY`. The two supplied meshes differ here, which is why the reader cannot assume either layout: `NACA0012_H2.cgns` declares `PhysicalDimension` = 3 and carries a nominally zero `CoordinateZ`, which is read and discarded, while `CylinderB1.cgns` declares `PhysicalDimension` = 2 and has no z coordinate at all. Both declare `CellDimension` = 2. The solver logs the declared dimensions for each mesh it opens. Element sections are scanned for the supported two-dimensional element types `TRI_3` and `QUAD_4`, which become cells, and `BAR_2`, which becomes a boundary edge. No structured-grid or single-element-type assumption is made anywhere: the NACA mesh is genuinely mixed, with 10 752 triangles and 10 064 quadrilaterals.

Two CGNS subtleties required specific handling and are worth recording:

- **Boundary conditions.** The `BC_t` nodes store a `PointRange` with `GridLocation=EdgeCenter`. These indices are *element* indices, not node indices, and they coincide with the `ElementRange` of the identically named `BAR_2` section. The reader therefore maps each boundary condition to its edge section by range containment, then tags the resulting mesh edges with the family name. Family names are carried as strings and matched against the `boundary_conditions` dictionary of the case file, so `bc-2`/`bc-4` and `WALL`/`FAR` are handled by the same code path. An unmapped or unknown family is a fatal, clearly reported error.

- **Multi-zone connectivity.** The cylinder mesh has two zones. Its interfaces are stored as `GridConnectivity_t` of type `Abutting1to1`, *not* as `GridConnectivity1to1_t`: a `cg_n1to1` query returns zero. The reader therefore uses `cg_nconns`/`cg_conn_info`/`cg_conn_read` and interprets the `PointList`/`PointListDonor` pairs as one-based zone-local *vertex* indices (the SIDS default location). Paired nodes are merged into single global nodes; the supplied interface point lists are bit-identical, so the maximum merge distance is exactly zero. After merging, the cylinder mesh has 9 995 nodes and 10 185 cells.

3.2 Face extraction and geometry

Cells are converted to an internal polygon representation and a face list is built by hashing each edge's sorted node pair. An edge seen by two cells becomes an interior face with a left and a right cell; an edge seen once becomes a boundary face carrying its family tag. This yields the cell–face adjacency graph used for reconstruction, for the gradient stencils and, in its dual form, for partitioning.

Cell area and centroid come from the shoelace formula over the vertex loop,

$$A = \frac{1}{2}\sum_i \left(x_i y_{i+1} - x_{i+1} y_i\right), \qquad \mathbf{x}_c = \frac{1}{6A}\sum_i (\mathbf{x}_i + \mathbf{x}_{i+1})\left(x_i y_{i+1} - x_{i+1} y_i\right), \tag{9}$$

which is exact for both triangles and quadrilaterals and needs no subdivision. Cell node ordering is *normalised* rather than assumed: any cell whose signed area comes out negative has its node list reversed, and the number of reorientations is logged. Only a zero or degenerate area is fatal, since that cannot be repaired by reordering. For both supplied meshes the reorientation count is zero — every cell is already stored counter-clockwise — so the normalisation is a no-op here, but the

reader does not depend on that being true. In two dimensions a face is a straight segment from $\mathbf{x}_a$ to $\mathbf{x}_b$, so its length Δs (the two-dimensional analogue of face area) and unit normal are

$$\Delta s = \|\mathbf{x}_b - \mathbf{x}_a\|, \qquad \mathbf{n} = \frac{1}{\Delta s}\left(y_b - y_a,\ -(x_b - x_a)\right), \tag{10}$$

with the face centroid at the segment midpoint. Each face normal is then oriented outward from its left cell by testing the sign of $\mathbf{n} \cdot (\mathbf{x}_f - \mathbf{x}_{c,\text{left}})$ and flipping when required. On a boundary face the left cell is by construction the adjacent fluid cell, so **the boundary normal points out of the fluid and into the body**. That convention matters for the force integration in section 5.5 and is the single easiest sign error to make in a solver of this kind.

3.3 Mesh and case summary

Table 2 summarises both meshes. The wall-boundary counts are what the surface output and force integration act on: 404 edges around the airfoil and 100 around the cylinder.

Table 2: Supplied meshes as read and preprocessed by the solver. The cylinder figures are after the two zones have been merged across their `Abutting1to1` interface.

Quantity	NACA0012_H2	CylinderB1
CGNS zones	1	2 (merged)
Nodes	15 682	9 995
Cells	20 816	10 185
triangles	10 752	mixed
quadrilaterals	10 064	mixed
Faces	36 498	20 180
Boundary faces	484	120
Wall-family edges	404 (`bc-4`)	100 (`WALL`)
Farfield edges	80 (`bc-2`)	20 (`FAR`)
Farfield extent	$\approx 80\,c$	$200\,D$
Body extent	$x \in [0, 1.005]$, $t = 0.120$	$r = 0.5$ exactly

A note on the airfoil geometry: the `bc-4` family spans $x \in [0, 1.005]$ with a maximum thickness of 0.120, consistent with a NACA0012 section of unit chord. Its identification as the body, rather than `bc-2` at radius ≈ 80, comes from the case-file mapping and not from geometry.

4 Spatial Discretization

Integrating eq. (2) over a fixed control volume Ω_i and applying the divergence theorem gives the cell-centred semi-discrete form

$$V_i \frac{\mathrm{d}\overline{\mathbf{U}}_i}{\mathrm{d}t} = \mathbf{R}_i = -\sum_{f \in \partial\Omega_i} \left[\mathbf{H}^i(\mathbf{W}_f^-, \mathbf{W}_f^+, \mathbf{n}_f) - \mathbf{H}^v(\mathbf{W}_f, \nabla\mathbf{W}_f, \mathbf{n}_f)\right] \Delta s_f, \tag{11}$$

where $\overline{\mathbf{U}}_i$ is the cell average, V_i the cell area, $\mathbf{H}^i$ the numerical inviscid normal flux, $\mathbf{H}^v$ the viscous normal flux of eq. (7), and $\mathbf{n}_f$ the outward unit normal of face f.

The implementation stores each face once and visits it once. Because the stored normal is outward from the left cell, the accumulated flux is subtracted from the left cell and added to the right cell:

$$\mathbf{R}_{\text{left}(f)} \mathrel{-}= \mathbf{H}_f\, \Delta s_f, \qquad \mathbf{R}_{\text{right}(f)} \mathrel{+}= \mathbf{H}_f\, \Delta s_f. \tag{12}$$

A single face therefore contributes equal and opposite amounts to its two neighbours, which makes the discretization conservative by construction: interior contributions telescope exactly and only boundary fluxes can change the global integral. This is verified to 9.2×10^{-17} in section 8.

On a boundary face there is no right cell. The left state is reconstructed to the face as usual and an appropriate exterior state is synthesised by the boundary condition (section 5); the same numerical flux function is then applied. This keeps boundary and interior faces on one code path, so no boundary case can drift out of sync with the interior scheme.

A face is processed if either of its cells is owned by the current rank. Faces on a partition boundary are thus evaluated redundantly on both ranks, which is cheaper than communicating fluxes. Because both ranks execute the same operations in the same order on the same halo-exchanged inputs, they compute bit-identical values for that face. This is a statement about a single face evaluation only: the *global* result still depends on rank count through the summation order of the reductions and the partition-dependent LU-SGS sweep order, as section 7 sets out.

4.1 Second-Order Reconstruction

Second-order accuracy comes from a piecewise-linear reconstruction of the *primitive* variables $\mathbf{W} = (\rho, u, v, p)$. Primitives are chosen so that the positivity of density and pressure can be enforced directly on the reconstructed face state, which is not possible when limiting conservative variables.

For each cell the gradient is an inverse-distance-weighted least-squares fit over the face neighbours. The gradient $\mathbf{g}_i$ of a variable ϕ minimises

$$J(\mathbf{g}_i) = \sum_{j \in \mathcal{N}(i)} w_j^2 \left[\mathbf{g}_i \cdot (\mathbf{x}_j - \mathbf{x}_i) - (\phi_j - \phi_i)\right]^2, \qquad w_j = \frac{1}{\|\mathbf{x}_j - \mathbf{x}_i\|}, \tag{13}$$

whose normal equations are the 2×2 system

$$\underbrace{\begin{bmatrix} \sum_j w_j^2 \Delta x_j^2 & \sum_j w_j^2 \Delta x_j \Delta y_j \\ \sum_j w_j^2 \Delta x_j \Delta y_j & \sum_j w_j^2 \Delta y_j^2 \end{bmatrix}}_{\mathbf{A}_i} \mathbf{g}_i = \sum_j w_j^2 (\phi_j - \phi_i) \begin{bmatrix} \Delta x_j \\ \Delta y_j \end{bmatrix}. \tag{14}$$

Since $\mathbf{A}_i$ depends only on geometry, it is inverted once during preprocessing and folded into a per-neighbour weight vector $\mathbf{W}_j = w_j^2 \, \mathbf{A}_i^{-1} \Delta \mathbf{x}_j$, so that at run time the gradient of every variable is the single sum $\mathbf{g}_i = \sum_j \mathbf{W}_j (\phi_j - \phi_i)$ — one dot product per neighbour per variable, with no linear algebra in the hot loop. A nearly rank-deficient stencil (possible when a cell's neighbours are almost collinear) is detected by comparing $\det \mathbf{A}_i$ against $10^{-12} \operatorname{tr}(\mathbf{A}_i)^2$ and regularised by a small diagonal shift, rather than being allowed to produce an unbounded gradient.

Boundary faces contribute a stencil entry located at the face centroid and carrying the *physical* boundary state (section 5), not the mirrored ghost state. This distinction is essential: using a mirrored state here would double every wall-normal gradient and corrupt the skin friction in the first cell off the wall.

The face states are then

$$\mathbf{W}_f^{\pm} = \mathbf{W}_i + \Phi_i \odot \left[\mathbf{g}_i \cdot (\mathbf{x}_f - \mathbf{x}_i)\right], \tag{15}$$

with Φ_i the limiter of section 4.2 and $\odot$ a component-wise product. Second-order reconstruction is **active in every production run reported here**; the only first-order fallback is the positivity mechanism below, whose activation count is recorded per run and reported in `metadata.json`.

4.2 Limiter and Positivity Control

Both a Barth–Jespersen and a Venkatakrishnan limiter are implemented; the production runs use Venkatakrishnan. For each cell and variable the neighbour envelope $\phi_i^{\min}, \phi_i^{\max}$ is gathered over the face neighbours, including boundary-face states so that a wall-adjacent cell is not falsely flagged as an extremum. With Δ_f the unlimited increment towards face f from eq. (15) and $y = (\phi_i^{\max} - \phi_i)/\Delta_f$ for $\Delta_f > 0$ (or $y = (\phi_i^{\min} - \phi_i)/\Delta_f$ for $\Delta_f < 0$), the two limiters are

$$\Phi^{\mathrm{BJ}} = \min(1,\, y)\,, \qquad \Phi^{\mathrm{V}} = \frac{y^2 + 2y + \epsilon^2/\Delta_f^2}{y^2 + y + 2 + \epsilon^2/\Delta_f^2}, \tag{16}$$

and the cell limiter is the minimum over its faces, clipped to $[0, 1]$. The Venkatakrishnan threshold is $\epsilon^2 = (K h_i)^3$ with $h_i = \sqrt{V_i}$ the cell length scale and K a case-settable constant. This threshold is what switches the limiter off in smooth regions: without it, limiter chattering on a stretched mesh stalls the steady residual, which is precisely the mechanism discussed in section 8.5.

Positivity is protected in two places. At the reconstruction stage, if the limited face state still has $\rho \le 0$ or $p \le 0$, the whole increment is halved repeatedly (up to eight times) and finally dropped to the first-order cell average, which is always admissible; each such event is counted. After each implicit update a final safety net floors density and pressure at 10^{-8} of their freestream values and replaces any non-finite state by the freestream, again counting every occurrence. Because these counts are written to the run metadata, a run cannot quietly rely on clipping to look healthy.

4.3 Inviscid and Viscous Fluxes

Three inviscid numerical fluxes are implemented — Roe with an entropy fix, HLLC, and Rusanov/local Lax–Friedrichs — selectable per run. Production runs use **Roe**. With Roe averages $\hat\rho = \sqrt{\rho_L \rho_R}$ and $\hat\phi = (\sqrt{\rho_L}\phi_L + \sqrt{\rho_R}\phi_R)/(\sqrt{\rho_L} + \sqrt{\rho_R})$ for $\phi \in \{u, v, H\}$, and $\hat a^2 = (\gamma - 1)(\hat H - \tfrac{1}{2}|\hat{\mathbf{u}}|^2)$, the flux is

$$\mathbf{H}^{\mathrm{Roe}} = \tfrac{1}{2}\,(\mathbf{F}_L + \mathbf{F}_R) - \tfrac{1}{2}\sum_k \tilde\lambda_k \alpha_k \mathbf{r}_k, \tag{17}$$

with wave speeds $\lambda_{1,2,3} = \hat u_n - \hat a,\ \hat u_n,\ \hat u_n + \hat a$, acoustic amplitudes $\alpha_{1,3} = (\Delta p \mp \hat\rho \hat a \Delta u_n)/(2\hat a^2)$, entropy amplitude $\alpha_2 = \Delta\rho - \Delta p/\hat a^2$, and a separately assembled shear contribution proportional to $\hat\rho\, \Delta u_t$.

Entropy fix. The eigenvalues are smoothed by the Harten function

$$\tilde\lambda(\lambda, \delta) = \begin{cases} |\lambda|, & |\lambda| \ge \delta, \\ \dfrac{\lambda^2 + \delta^2}{2\delta}, & |\lambda| < \delta, \end{cases} \tag{18}$$

applied to *all three* waves. For the acoustic waves δ is the Harten–Hyman spread, e.g. $\delta_1 = \max(0, \lambda_1 - (u_{n,L} - a_L), (u_{n,R} - a_R) - \lambda_1)$, which removes the non-physical expansion shock admitted by plain Roe at a sonic point. In addition, every wave receives a floor

$$\delta \ge \delta_{\min} = 0.05\, \max(|u_{n,L}| + a_L,\ |u_{n,R}| + a_R)\,. \tag{19}$$

The floor addresses a second, distinct failure that proved important on these meshes. The entropy and shear waves share the eigenvalue u_n, which vanishes wherever the flow is parallel to a face — at a stagnation point, and along the symmetry line of a symmetric body. There the

density and tangential-velocity jumps are left completely undamped, which is the classical Roe checkerboard/carbuncle mode. It does not diverge; it quietly stalls the steady residual and breaks the symmetry that a zero-incidence case should preserve. Applying the floor through the same smooth Harten function keeps the flux consistent, because identical left and right states make the jumps themselves vanish. The coefficient 0.05 is small enough to leave boundary layers and contact discontinuities sharp.

Viscous fluxes. Face gradients are built from the same least-squares cell gradients. On an interior face the two cell gradients are averaged and then corrected along the cell-to-cell direction $\mathbf{e} = \Delta\mathbf{x}/\|\Delta\mathbf{x}\|$ so the face gradient reproduces the actual cell-value difference exactly,

$$\nabla\phi_f = \overline{\nabla\phi} + \left(\frac{\phi_R - \phi_L}{\|\Delta\mathbf{x}\|} - \overline{\nabla\phi} \cdot \mathbf{e} \right) \mathbf{e}, \tag{20}$$

which suppresses the odd-even decoupling that a plain average allows. The temperature gradient is derived from the density and pressure gradients through $T = p/(\rho R)$, $\nabla T = R^{-1} \left(\nabla p/\rho - p\,\nabla\rho/\rho^2 \right)$.

On a wall face the viscous flux is evaluated with the *physical* wall state and the wall-normal derivative is corrected using the imposed wall value: with $d_n = (\mathbf{x}_f - \mathbf{x}_i) \cdot \mathbf{n}$,

$$\nabla\phi \leftarrow \nabla\phi + \left(\frac{\phi_{\text{wall}} - \phi_i}{d_n} - \nabla\phi \cdot \mathbf{n} \right) \mathbf{n}. \tag{21}$$

This is what makes the skin friction reflect the actual near-wall profile on a stretched boundary-layer mesh instead of the much smaller cell-average gradient. For an adiabatic wall the normal component of ∇T is then projected out exactly, enforcing $\partial T/\partial n = 0$.

Positivity fallback. If the Roe average is inadmissible ($\hat{a}^2 \leq 0$, possible for a very strong expansion), the flux falls back to HLLC for that face, which is positivity preserving. This is reported in `metadata.json` as part of the flux description rather than hidden. In practice this path is not taken: every completed production run records zero positivity-fallback events, so the reported results are pure Roe fluxes throughout. That has a consequence for what the production results can and cannot verify, discussed in section 8.6.

Named dissipation and limiter parameters. For completeness, the scheme carries four tuned constants, all disclosed here rather than buried in the source: the linear-wave dissipation floor coefficient 0.05 in eq. (19); the Venkatakrishnan threshold constant $K = 5.0$ in eq. (16); and the viscous spectral-radius factors 4.0 in the local time step eq. (29) and 2.0 in the implicit off-diagonal eq. (32).

4.3.1 Added dissipation on the linear waves: disclosure

The floor eq. (19) deserves to be stated plainly, because only part of it is textbook. Smoothing the *acoustic* eigenvalues by the Harten–Hyman spread is the standard entropy fix and needs no defence. Applying a floor to the *entropy and shear* eigenvalues is different: it is added artificial dissipation, a carbuncle cure, and it is not part of the classical Roe scheme.

It is present because without it these cases do not converge correctly. The entropy and shear waves share the eigenvalue u_n, which vanishes identically on any face where the flow is parallel to the face — including the stagnation streamline of a symmetric body at zero incidence, which is exactly the configuration of all six airfoil cases. There, density and tangential-velocity jumps are

advected with zero dissipation, producing a checkerboard mode that does not diverge but stalls the residual and destroys the symmetry the solution should have.

The cost is quantifiable. At a stagnation face where the exact numerical flux is $[0,\, p,\, 0,\, 0]$ in normalised form, the floored scheme returns $[-2.96 \times 10^{-4},\, 1.0,\, 2.13 \times 10^{-4},\, 7.42 \times 10^{-5}]$: a mass-flux error of about 3×10^{-4} relative to the pressure term. The scheme remains consistent, since identical left and right states make all jumps vanish and the exact physical flux is recovered, but at a stagnation point the added dissipation is genuinely there.

The honest concern is that at $Re = 5000$ this numerical dissipation competes with the physical viscosity, so a fraction of the computed skin friction could in principle be an artefact of the floor rather than of μ. That concern was tested rather than argued away, by sweeping the coefficient in controlled experiments. Section 9.1 reports the outcome in full, and it does not support the design choice: the floor changes C_D by 0.021 % on the cylinder, and disabling it entirely produces no checkerboard mode even on the zero-incidence inviscid airfoil, where the entropy and shear eigenvalues vanish on the stagnation streamline and no physical viscosity is present to damp a carbuncle. The floor is therefore retained as a safeguard for conditions not exercised here, and no result in this report depends on it.

5 Boundary Conditions

Three boundary conditions are implemented, selected per family by the case file: `farfield`, `slip_wall`, and `no_slip_adiabatic_wall`.

5.1 Ghost state versus face state

A design point worth stating explicitly, because it is a common source of silent error, is that each boundary condition supplies *two* different exterior states and the solver uses them in different places:

- The **ghost state** $\mathbf{U}^{\text{ghost}}$ is a mirrored or characteristic state passed to the Riemann solver as the right state. Its purpose is that the resulting numerical flux *enforces* the boundary condition.
- The **face state** $\mathbf{U}^{\text{face}}$ is the physical state that actually exists on the boundary. It is used for the least-squares gradient stencil, for the limiter envelope, for the viscous wall flux, and for the surface output written to `surface.csv`.

Conflating the two is what produces the classic artefacts: doubled wall-normal gradients, wall rows in the surface output that are really adjacent cell-centre values, and a no-slip wall that reports a nonzero velocity. The distinction also means `surface.csv` carries genuine boundary values, which is declared in the metadata as `wall_boundary_output_semantics = boundary_value`. Cell-centre quantities are written separately to `surface_cell_center.csv` so the two can never be confused.

5.2 Inviscid slip wall

The ghost state reflects the normal momentum component while preserving density and total energy,

$$\mathbf{u}^{\text{ghost}} = \mathbf{u} - 2(\mathbf{u}\cdot\mathbf{n})\mathbf{n}, \qquad \rho^{\text{ghost}} = \rho, \qquad (\rho E)^{\text{ghost}} = \rho E, \tag{22}$$

which is consistent because reflection leaves $|\mathbf{u}|$ unchanged. Feeding eq. (22) to the Roe flux gives

$$\mathbf{H}^{\text{wall}} = [0,\; p\,n_x + \varepsilon n_x,\; p\,n_y + \varepsilon n_y,\; 0]^T, \qquad \varepsilon \to 0 \;\text{ as } u_n \to 0, \tag{23}$$

that is, **exactly** zero mass flux and **exactly** zero energy flux, with a normal force that reduces to the pure pressure force $p\mathbf{n}$ only in the limit of vanishing normal velocity.

An earlier draft of this report stated eq. (23) as an exact identity with $\varepsilon \equiv 0$, and that was wrong; the correction is recorded here rather than quietly made because the claim had also been listed as a verified check. The mirrored state has a normal-velocity jump of $\Delta u_n = -2u_n$, so the acoustic wave amplitudes in the Roe dissipation do not vanish and their contributions do not cancel in the normal-momentum component. Probing the solver's own `roeFlux` with the mirrored state (via `report/check_slip_flux.sh`, which links the shipped library rather than reimplementing it) gives, for $p = 0.7143$:

Interior state, normal	Mass	Normal mom.	Energy	u_n
$\mathbf{V} = (0.3, 0.1)$, $\mathbf{n} = +y$	0	0.8244	0	0.100
$\mathbf{V} = (0.9, -0.2)$, $\mathbf{n} = (0.6, 0.8)$	-2.8×10^{-17}	1.2441	0	0.380
$\mathbf{V} = (1, 0)$, $\mathbf{n} = +y$	-5.4×10^{-17}	0.7143	-1.7×10^{-16}	0
$\mathbf{V} = 0$	0	0.7143	0	0

The two rows with $u_n = 0$ — a purely tangential flow and a stagnation point — return the pressure exactly. The rows with $u_n \neq 0$ carry an excess that grows with u_n. The mass and energy components are exactly zero in every case, which is the part of the identity that matters for conservation: no mass or energy crosses a slip wall regardless of the iterate.

The practical significance is small but should be stated rather than assumed. On a converged solution the wall condition is satisfied, so u_n at a slip wall is at machine level — the sanity checks record a maximum $|u_n|$ of 1.0×10^{-13}, 1.4×10^{-13} and 1.9×10^{-13} on the three inviscid cases — and the excess is correspondingly negligible in the reported forces. The discrepancy is a property of *intermediate* iterates, where it acts as a restoring term driving u_n towards zero, which is the intended behaviour of a ghost-state wall treatment. What is *not* justified is calling the identity exact, and it is no longer claimed as such here or in section 8.

The corresponding face state removes the normal velocity entirely, $\mathbf{u}^{\text{face}} = \mathbf{u} - (\mathbf{un})\mathbf{n}$, keeps ρ and p, and corrects the total energy for the reduced kinetic energy. A slip wall therefore reports near-zero *normal* velocity with a retained tangential velocity, exactly as the output contract requires.

5.3 Viscous no-slip adiabatic wall

The ghost state reverses the entire velocity vector, $\mathbf{u}^{\text{ghost}} = -\mathbf{u}$, with ρ and ρE unchanged, so that the arithmetic face average of the interior and ghost velocities is exactly zero. The face state imposes the wall condition directly: zero velocity, interior pressure ($\partial p/\partial n = 0$), and interior temperature (adiabatic, $\partial T/\partial n = 0$), with the density following from the equation of state,

$$\mathbf{u}^{\text{face}} = \mathbf{0}, \quad p^{\text{face}} = p_i, \quad T^{\text{face}} = T_i, \quad \rho^{\text{face}} = \frac{p^{\text{face}}}{R\,T^{\text{face}}}, \quad (\rho e)^{\text{face}} = \frac{p^{\text{face}}}{\gamma - 1}. \tag{24}$$

Two points of honesty about eq. (24). First, with $p^{\text{face}} = p_i$ and $T^{\text{face}} = T_i$ the equation of state returns the interior density bit-for-bit, so writing $\rho = p/(RT)$ is a statement of *why* the wall density equals the interior density rather than a computation that produces a different value; the code says as much in a comment at the same line. It is the correct form because it is what makes the adiabatic assumption explicit and would give the right answer if either the pressure or the temperature condition were changed, but no claim of sophistication over a copy is intended. Second, the total energy is written here as ρe rather than ρE because the velocity is zero: the

general definition of section 2 includes the kinetic term, which vanishes at a no-slip wall, and using the same symbol for the reduced quantity would be inconsistent with the rest of the report.

Because the surface output uses this state, the reported wall velocity and Mach number are identically zero to machine precision. The adiabatic condition is additionally enforced on the flux by projecting the normal component out of ∇T, as described after eq. (21).

5.4 Farfield

The farfield is characteristic, based on locally one-dimensional Riemann invariants along the face normal. With interior values $(\rho_i, p_i, u_{n,i}, a_i)$ and freestream values $(\rho_\infty, p_\infty, u_{n,\infty}, a_\infty)$, the normal Mach number $M_n = u_{n,i}/a_i$ selects the branch:

- $M_n \geq 1$ (supersonic outflow): the boundary state is the interior state; nothing is imposed.
- $M_n \leq -1$ (supersonic inflow): the boundary state is the freestream.
- $|M_n| < 1$ (subsonic): the outgoing and incoming invariants

$$R^+ = u_{n,i} + \frac{2a_i}{\gamma - 1}, \qquad R^- = u_{n,\infty} - \frac{2a_\infty}{\gamma - 1} \tag{25}$$

  are combined into $u_{n,b} = \frac{1}{2}(R^+ + R^-)$ and $a_b = \frac{1}{4}(\gamma - 1)(R^+ - R^-)$. Entropy $s = p/\rho^\gamma$ and the tangential velocity are taken from the upwind side (interior if $u_{n,b} > 0$, freestream otherwise), giving $\rho_b = \left(a_b^2/(\gamma s)\right)^{1/(\gamma-1)}$ and $p_b = s\rho_b^\gamma$.

This treatment is what allows the supersonic cases to be posed on the same mesh as the subsonic ones without special handling: at $M_\infty = 2$ the inflow arc is fully determined by the freestream and the outflow arc is fully determined by the interior, so the bow shock can leave the domain cleanly. Degenerate invariant combinations that would give $a_b \leq 0$, or a non-positive reconstructed density or pressure, fall back to the freestream state rather than producing an unphysical boundary value. The same characteristic state is used as both ghost and face state at the farfield, since no wall condition applies there.

5.5 Force integration and the pressure/friction split

Forces are integrated over wall faces only. Recalling from section 3 that the stored boundary normal $\mathbf{n}$ points out of the fluid and into the body, the outward normal of the *body* surface is $\mathbf{m} = -\mathbf{n}$, and the Cauchy traction exerted by the fluid on the body is

$$\mathbf{t} = \boldsymbol{\sigma}\cdot\mathbf{m} = (-p\mathbf{I} + \boldsymbol{\tau})\cdot(-\mathbf{n}) = p\,\mathbf{n} - \boldsymbol{\tau}\cdot\mathbf{n}. \tag{26}$$

The pressure force per face is therefore $+(p_{\text{wall}} - p_\infty)\mathbf{n}\,\Delta s$ — referenced to p_∞ so that a uniform ambient pressure contributes nothing on a closed body — and the viscous force is $-\boldsymbol{\tau}\cdot\mathbf{n}\,\Delta s$. Getting this sign pair right is essential: with the opposite viscous sign a flat plate with $\partial u/\partial y > 0$ would produce thrust instead of drag.

The wall pressure is reconstructed from the adjacent cell with the same gradient used by the flux, so the integrand is second-order accurate and consistent with the residual. For the friction the *tangential* part of the viscous traction is used,

$$\boldsymbol{\tau}_t = \boldsymbol{\tau}\cdot\mathbf{n} - [(\boldsymbol{\tau}\cdot\mathbf{n})\cdot\mathbf{n}]\,\mathbf{n}, \tag{27}$$

which is what the contract requires: the normal viscous traction is explicitly excluded rather than being mislabelled as skin friction. The wall-normal velocity derivatives entering $\boldsymbol{\tau}$ are corrected with the imposed zero wall velocity exactly as in eq. (21).

Each wall face is integrated by the rank that owns its adjacent cell, so a face on a partition boundary is counted exactly once globally, and the six accumulators (pressure force, viscous force, moment, wall length) are combined in a single `MPI_Allreduce`. The moment about the case reference centre is $M_z = \sum(\mathbf{r} \times \mathbf{f})_z$ with $\mathbf{r} = \mathbf{x}_f - \mathbf{x}_{\mathrm{ref}}$, positive counter-clockwise.

6 Implicit and Transient Time Integration

6.1 Steady pseudo-time march

Steady cases solve $\mathbf{R}(\mathbf{U}) = \mathbf{0}$ by a backward-Euler pseudo-time march. Linearising eq. (11) about the current iterate gives, at each pseudo-time step, the linear system

$$\left(\frac{V_i}{\Delta\tau_i}\mathbf{I} + \frac{\partial \mathbf{R}}{\partial \mathbf{U}}\right)\Delta\mathbf{U} = \mathbf{R}(\mathbf{U}^m), \qquad \mathbf{U}^{m+1} = \mathbf{U}^m + \Delta\mathbf{U}. \tag{28}$$

The pseudo-time step is local to each cell and set from the convective and viscous spectral radii accumulated during the residual evaluation,

$$\Delta\tau_i = \frac{\mathrm{CFL}\cdot V_i}{\displaystyle\sum_{f\in\partial\Omega_i}\left(|u_n| + a\right)_f \Delta s_f \;+\; C_v \sum_{f\in\partial\Omega_i}\frac{\mu_f}{\rho_f}\frac{\Delta s_f^2}{V_i}}, \tag{29}$$

with $C_v = 4$ for the viscous contribution. Local time stepping lets the tiny wall cells and the huge farfield cells advance at their own stable rates, which is what makes a mesh with a cell-volume range of roughly ten orders of magnitude tractable at all.

The CFL number is ramped geometrically in log space from `cfl_initial` to `cfl_max` over `pseudo_cfl_ramp_steps`,

$$\mathrm{CFL}(m) = \mathrm{CFL}_0\left(\frac{\mathrm{CFL}_{\max}}{\mathrm{CFL}_0}\right)^{m/M_{\mathrm{ramp}}}, \tag{30}$$

capped at $\mathrm{CFL}_{\max}$ thereafter. The supplied case values are used without modification; table 3 lists them.

A residual-based safeguard multiplies the ramped CFL by an adaptive factor. If a step increases the residual by more than 30%, or produces a non-finite residual, the step is rejected, the previous accepted state is restored, and the factor is cut to 0.35 of its value; after a successful step the factor recovers by 6% up to unity. Rejected-step counts are reported. This is a robustness device, not an accuracy device: it changes only the path to the steady state.

6.2 LU-SGS inner solve and the Jacobian approximation

Equation (28) is solved approximately by symmetric Gauss–Seidel sweeps (LU-SGS); a block-Jacobi variant is also available. The Jacobian is never assembled. The diagonal is the scalar

$$D_i = \frac{V_i}{\Delta\tau_i} + \frac{1}{2}\sum_{f\in\partial\Omega_i}\left(|u_n| + a\right)_f \Delta s_f + \sum_{f\in\partial\Omega_i}\left(\frac{\mu}{\rho}\right)_f \frac{\Delta s_f^2}{V_i}, \tag{31}$$

where the factor $\frac{1}{2}$ is the upwind split of the face dissipation between the two adjacent cells. The off-diagonal action on a neighbour is the scalar-dissipation upwind approximation

$$\mathbf{O}_{ij}\Delta\mathbf{U}_j = \frac{\Delta s_f}{2}\left[\mathbf{A}_j(\mathbf{n})\Delta\mathbf{U}_j - \lambda_j\Delta\mathbf{U}_j\right], \qquad \lambda_j = |u_{n,j}| + a_j + \frac{2\mu}{\rho_j\,\|\Delta\mathbf{x}\|}, \tag{32}$$

with the last term present only for viscous runs. Crucially, $\mathbf{A}_j(\mathbf{n})\Delta\mathbf{U}_j$ is the *exact analytic* Jacobian-vector product of the Euler normal flux, evaluated matrix-free: no 4×4 block is ever stored. Writing $\mathrm{d}(\rho u_n) = u_n\,\mathrm{d}\rho + \rho\,\mathrm{d}u_n$ and $\mathrm{d}p = (\gamma-1)\left[\mathrm{d}(\rho E) - \frac{1}{2}|\mathbf{u}|^2\mathrm{d}\rho - \rho(u\,\mathrm{d}u + v\,\mathrm{d}v)\right]$, the product is

$$\mathbf{A}(\mathbf{n})\,\mathrm{d}\mathbf{U} = \begin{bmatrix} \mathrm{d}(\rho u_n) \\ \mathrm{d}(\rho u_n)u + \rho u_n\,\mathrm{d}u + \mathrm{d}p\,n_x \\ \mathrm{d}(\rho u_n)v + \rho u_n\,\mathrm{d}v + \mathrm{d}p\,n_y \\ \mathrm{d}u_n(\rho E + p) + u_n\,(\mathrm{d}(\rho E) + \mathrm{d}p) \end{bmatrix}. \tag{33}$$

This product is verified against central finite differences to 1.7×10^{-8} over 20 000 randomised states (section 8).

It is worth being precise about what is exact here and what is not, because the metadata records the solver as `lu_sgs_simplified_jacobian` and the two statements must not look contradictory. The *flux Jacobian-vector product* eq. (33) is exact and analytic. The *overall implicit operator* is nonetheless a simplified Jacobian: the diagonal eq. (31) is a scalar rather than a 4×4 block, and the off-diagonal action eq. (32) combines the exact product with a scalar-dissipation term instead of the true upwind flux linearisation. That approximation is deliberate and standard for LU-SGS — it is what makes the sweeps matrix-free — and it affects only the rate of convergence to the steady state, not the steady state itself, because the converged solution is determined by $\mathbf{R}(\mathbf{U}) = \mathbf{0}$ and not by the operator used to get there.

Each sweep refreshes the ghost increments once, then performs a forward pass over ascending local cell index and a backward pass over descending index. Between refreshes the off-rank coupling is frozen, which makes the scheme exactly block-Jacobi across ranks and pure LU-SGS within a rank — so the iteration count can depend mildly on the partition even though the converged answer does not.

The inner iteration is monitored by the true linear residual of the system it is solving,

$$r_{\mathrm{lin}} = \frac{\|\mathbf{R} - (D\,\Delta\mathbf{U} + \mathbf{O}\,\Delta\mathbf{U})\|_2}{\|\mathbf{R}\|_2}, \tag{34}$$

evaluated with the same matrix-free operator the sweeps use and volume-normalised like the nonlinear norms. Sweeps are added in small blocks, within the case-file bounds, until r_{lin} meets the case target or fails to improve by more than 2% in three consecutive blocks. The increment is deliberately *not* reset between blocks — an early version did reset it, which silently discarded all previous work and capped the achievable inner convergence.

One negative result is worth recording, because it is a tempting shortcut. Inflating the diagonal by a factor $\beta > 1$ makes the sweeps look dramatically better: at $\beta = 2$ the reported ratio reached 2.2×10^{-16}. That number is meaningless, because it measures a different (easier) system than the one being solved; the true residual ratio of eq. (34) was 0.82. The diagonal is therefore left at its consistent value eq. (31), and the slow-but-honest convergence of SGS at high CFL is accepted instead.

6.3 Convergence criteria

All reported residual norms are volume-weighted root-mean-square rates of change of the cell average, globally MPI-reduced:

$$\|\mathbf{R}\|_2 = \left(\frac{\sum_i \sum_k \left(R_{i,k}/V_i\right)^2 V_i}{\sum_i V_i} \right)^{1/2}, \qquad \|\mathbf{R}\|_\infty = \max_{i,k} \left|R_{i,k}/V_i\right|. \tag{35}$$

Dividing by the cell volume converts the integral residual into a rate, which makes the norm independent of local cell size and therefore comparable across meshes and rank counts. Section 8.5 describes the termination test built on these norms, and the defect that had to be fixed in it.

6.4 Transient dual-time BDF2 for the vortex street

The $Re = 200$ cylinder is integrated in physical time by a genuine two-loop dual-time scheme. The outer loop steps physical time with second-order backward differences; the inner loop drives the resulting nonlinear system to convergence at each physical step.

With $a_0 = \frac{3}{2},\ a_1 = -2,\ a_2 = \frac{1}{2}$ the BDF2 residual is

$$\mathbf{R}^*(\mathbf{U}^{n+1}) = \mathbf{R}(\mathbf{U}^{n+1}) - \frac{V_i}{\Delta t}\left(a_0\mathbf{U}^{n+1} + a_1\mathbf{U}^{n} + a_2\mathbf{U}^{n-1}\right), \tag{36}$$

and the inner iteration solves $\mathbf{R}^* = \mathbf{0}$ with the same LU-SGS machinery, using the diagonal

$$D_i^* = \frac{V_i}{\Delta\tau_i} + \frac{a_0 V_i}{\Delta t} + \frac{1}{2}\sum_f \left(|u_n| + a\right)_f \Delta s_f + \ldots, \tag{37}$$

i.e. carrying *both* the pseudo-time and the physical-time term, so each inner iteration is a Newton-like step on the time-accurate system rather than a pseudo-time march with a source term. The first physical step uses the self-starting BDF1 coefficients $a_0 = 1, a_1 = -1, a_2 = 0$.

Two properties required by the benchmark hold explicitly in the implementation:

1. $\mathbf{U}^n$ and $\mathbf{U}^{n-1}$ are stored in separate arrays and are **frozen throughout all inner iterations** for $\mathbf{U}^{n+1}$. Only the $\mathbf{U}^{n+1}$ iterate changes inside the inner loop.
2. The history is shifted ($\mathbf{U}^{n-1} \leftarrow \mathbf{U}^n$, $\mathbf{U}^n \leftarrow \mathbf{U}^{n+1}$) only **after** the inner solve for that physical step has been accepted.

The inner loop terminates when the total transient residual $\|\mathbf{R}^*\|_2$ has fallen by the case target relative to its value at the first inner iteration of that physical step, subject to the minimum iteration count. The convergence measure is the *total* residual including the physical-time term, as the benchmark specifies, not the spatial residual alone. Per-step inner counts, target misses and the final ratio are accumulated and reported in `metadata.json`.

The run is declared `statistically_periodic` only if it reaches the requested horizon *and* the inner solve converged on at least 95% of physical steps. A run that reaches the final time while frequently missing the inner target is an under-converged dual-time solve and is reported as `failed` rather than presented as a usable transient.

For the transient case the supplied production settings are used exactly: $\Delta t = 0.01$, $t_f = 300.0$ (30 000 physical steps), inner iteration bounds 5–1000, inner target 10^{-3} on the total transient residual, and a fixed pseudo-time CFL of 1.0.

Table 3: Production run controls. Every value is taken from the supplied case file and used unmodified; no case uses a loosened setting.

Case group	Max steps	CFL_0	$\mathrm{CFL}_{\max}$	Ramp steps	Inner
NACA inviscid $M_\infty = 0.15$	20 000	1.0	100	2000	3–50
NACA inviscid $M_\infty = 0.80$	30 000	1.0	100	3000	3–50
NACA inviscid $M_\infty = 2.0$	40 000	0.2	50	5000	3–80
NACA laminar $M_\infty = 0.15$	40 000	1.0	100	5000	3–80
NACA laminar $M_\infty = 0.80$	40 000	0.5	80	5000	3–80
NACA laminar $M_\infty = 2.0$	50 000	0.2	50	7000	3–100
Cylinder $Re = 20$	30 000	1.0	100	3000	3–80
Cylinder $Re = 200$	30 000	1.0	1.0	0	5–1000

7 MPI Parallelization and METIS Partitioning

7.1 Cell dual graph and METIS

The mesh is read and partitioned once, during setup. The partitioning graph is the *cell dual graph*: each cell is a vertex, and every interior face contributes one undirected edge between its two cells. Boundary faces contribute no edge. The graph is built in compressed adjacency form (`xadj`, `adjncy`) by counting degrees, prefix-summing, then filling.

Partitioning calls `METIS_PartGraphKway` with $n_{\mathrm{con}} = 1$, zero-based numbering, a fixed random seed of 12 345 so that a clean checkout reproduces the partition exactly, and `METIS_OPTION_CONTIG` requested to prefer contiguous parts. If the contiguity constraint causes METIS to fail, the call is retried without it and the fallback is logged; a genuine METIS failure is a fatal error. The reported objective value is the edge cut recorded in `metadata.json`. This is real graph partitioning, not a geometric split: the metadata value `metis_kway` corresponds directly to this call.

7.2 Rank-local mesh and ghost construction

After partitioning, each rank builds a mesh containing only what it needs:

- its **owned** cells, numbered $[0, N_{\mathrm{own}})$;
- a layer of **ghost** cells, numbered $[N_{\mathrm{own}}, N_{\mathrm{own}} + N_{\mathrm{ghost}})$, being exactly those off-rank cells that share a face with an owned cell;
- the faces incident on any owned cell, with left/right indices remapped to local numbering;
- the geometry, gradient stencils and boundary tags for those entities.

No rank ever stores the global cell list, the global face list, or the global conservative state during iterations. Correspondingly, `metadata.json` reports `full_state_replication_during_iterations` and `full_mesh_replication_during_iterations` as `false`.

Ghost ownership is resolved once at setup by a single collective exchange of the requested global cell identifiers, after which each rank knows precisely which of its owned cells to send to which neighbour, in an order shared by both sides. That shared ordering is what allows the iteration-time exchange to be a plain contiguous buffer copy with no index metadata on the wire. This collective appears only in preprocessing; it is not part of the iteration loop.

7.3 Halo exchange

Iteration-time communication is neighbour-scoped point-to-point. For each exchange the receives are posted first with `MPI_Irecv`, the send buffers are packed and sent with `MPI_Isend`, a single `MPI_Waitall` completes all of them, and the received data is unpacked into the ghost slots. The metadata field `halo_exchange` is `neighbor_isend_irecv`. There is no `MPI_Allgather` of state at any point in the iteration.

Each residual evaluation performs a fixed, small number of exchanges:

1. the conservative state (once per residual evaluation, by the driver);
2. the primitive gradients, $4 \times 2 = 8$ components, needed for reconstruction from the far side of a partition face and for the viscous face gradient;
3. the limiter factors, 4 components, so a partition-boundary face reconstructs exactly as it would in serial.

The last two are the reason a partitioned second-order run reproduces the serial answer: the far-side cell has not only its state but also its gradient and its limiter value, so eq. (15) evaluates identically on both sides of a cut. Each LU-SGS sweep additionally exchanges the current increment $\Delta\mathbf{U}$.

Global reductions are used only where they are physically required: one `MPI_Allreduce` for the residual norms (component sums, volume, and a max for the L_∞ norm), one for the six force accumulators, and one for the linear-residual ratio of eq. (34). Output is written by rank 0 from reduced data, so file contents do not depend on rank count except through floating-point summation order.

7.4 Partition diagnostics

Table 4 reports the per-rank decomposition actually used during the iterations, taken from the submitted `partition_diagnostics.csv` of Cyl. Re=20, $np = 8$. The load-balance ratio, defined as $\max_r N_{\mathrm{own},r}/\overline{N_{\mathrm{own}}}$, is 1.007.

The highest available rank count is tabulated deliberately. A two-way split of a balanced mesh is trivially even and demonstrates nothing about the partitioner, whereas at eight ranks the structure is visible and non-trivial: owned-cell counts range over 1265–1282 against a mean of 1273, giving the balance ratio above; neighbour counts vary from 3 to 7, reflecting genuinely irregular partition shapes rather than slabs; and four of the eight ranks own *no* boundary faces at all, being interior partitions of the domain. That last point is a useful check in itself — a geometric or index-order split of this mesh would give every rank a share of the outer boundary, so its absence is consistent with the graph-based decomposition claimed in section 7.

8 Output, Reproducibility, and Sanity Checks

8.1 Build and run

The solver depends on MPI, CGNS (with HDF5 and zlib), and METIS, plus a header-only JSON reader. It is built with CMake:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
      -DCFD_EXTERNALS_ROOT=/opt/external/cfd_externals/install \
```

Table 4: Per-rank partition diagnostics for Cyl. Re=20, $np = 8$, from the submitted `partition_diagnostics.csv`. Send and receive counts are the number of cell records moved per halo exchange. Send and receive counts differ by rank because the halo relationship is not symmetric in cell count: a rank may need more cells from its neighbours than they need from it.

Rank	Owned	Ghost	Boundary faces	Neighbours	Send	Recv
0	1266	103	20	7	101	103
1	1265	106	38	4	104	106
2	1281	107	33	3	109	107
3	1276	129	29	5	127	129
4	1282	101	0	3	101	101
5	1277	105	0	4	107	105
6	1273	119	0	4	120	119
7	1265	118	0	6	119	118

```
      -DCFD_EXTERNALS_HEADER_ROOT=/opt/external
cmake --build build -j 32
```

and every case is run through the single required CLI form:

```
mpirun -np <ranks> ./build/cns2d solve --case <case.json> \
       --output <results-dir> [--restart <file>] [--report-level brief|full]
```

No case file or source edit is needed between cases. Debug-only overrides (`--max-steps`, `--flux`, `--limiter`, `--spatial-order`, and similar) exist for diagnostics; none of them is used in the production runs reported here, and the exact command line for each run is recorded in its `run_status.json`. Malformed input, a missing mesh, an unmapped boundary family, or an inconsistent freestream cause a non-zero exit with a specific message rather than a silent default.

Every flag advertised in the required CLI is implemented. `--report-level` selects how much of the mesh-verification report is printed: `full` lists every measured quantity, while `brief` prints a one-line summary. The verification checks themselves always run, so the level cannot influence a computed result — confirmed by running the same case at both levels and diffing the outputs, which are byte-identical in `residuals.csv` and `forces.csv`. Exit codes are distinct and reduced across ranks with `MPI_MAX`: 0 on success, 1 for an `MPI_Init` failure, 2 for bad input, and 3 for a numerically failed run.

Build and run instructions, dependency versions, per-case outputs, the test suite, the Python tooling and a ten-step reproduce-from-clean-checkout sequence are documented in `README.md` at the root of the solver tree.

Those instructions were *verified* rather than assumed: a completely fresh out-of-tree build was configured and compiled from the source alone, and the resulting test binary passes all 139 checks. The build tree is therefore a generated artifact reproducible from source, and is excluded from version control on that basis. The two driver scripts that produced the submitted runs are kept under `scratch/` and referenced from the README, since a pointer to the script that was actually run is worth more than a retyped command that may drift from it.

8.1.1 Provenance of each result, and a build bug that falsified it

Every `metadata.json` records the git revision of the source that produced it, so a reader can check out the exact tree behind any number in this report. That mechanism was broken in a way worth

documenting, because the failure was silent and the reported value was not merely stale but actively wrong.

The revision was captured once at CMake *configure* time and never refreshed, so every result — all eight production cases and the whole rank-count study — recorded a revision that predated the solver source entirely. It pointed at a documentation commit containing no solver at all. A reader following it would have found nothing that could have produced these results, which is worse than a stale hash: it is a reproducibility claim that fails on the first thing anyone would try.

Two fixes were needed, and the second is the instructive one:

1. The revision is now re-stamped on *every* build rather than at configure time, and a work-tree carrying uncommitted tracked changes is stamped `<rev>-dirty`, so a result built from uncommitted code cannot be attributed to a clean commit.

2. The first attempt **appeared to work and did not**. The stamp target had no ordering relationship to the library, so it ran *concurrently* with compilation: a changed revision reached the binary only on the *following* build, leaving the generated header saying one thing and the compiled binary containing another. It was corrected by making the compilation depend on the stamp target so the stamp completes first.

The way the second bug was caught is the part that generalises. Inspecting the generated header would have shown the correct revision and passed — the header was right; the binary was wrong. It was found only by *running the solver and comparing the* `metadata.json` *it wrote against the header*, which is the only check that exercises the path the results actually take. Verifying an intermediate artifact is not the same as verifying the output, and a concurrency bug in a build graph is invisible to the former.

The re-stamped revision is recorded in `run_manifest.csv` alongside every case. The solver source was unchanged across this correction — `git diff` over `src/` is empty between the previous binary and the corrected one — so no coefficient, convergence status, or figure in this report is affected. Only the recorded identity of the build changed, which is precisely the point: the numbers were always right and their provenance was not.

8.2 Generated files

Each case directory contains `metadata.json`, `run_status.json`, `residuals.csv`, `forces.csv`, `surface.csv`, `partition_diagnostics.csv` and `.json`, `field_final.vtu`, `restart_final.bin`, and `stdout.log`; viscous cases add `surface_cell_center.csv` carrying the adjacent cell-centre values that `surface.csv` deliberately does not mix in. The report artifacts (`run_manifest.csv`, `figure_manifest.csv`, `sanity_checks.json`) and every figure are regenerated from those files alone by

```
.venv/bin/python tools/make_figures.py --results-root results \
    --out-dir report/figures --manifest report/figure_manifest.csv
```

so no figure can show anything other than submitted data. The numeric values quoted in this report are likewise harvested mechanically from the same files by `report/harvest_numbers.py`, which emits the LaTeX macros and table rows used above; the prose contains no hand-typed result numbers.

8.3 Geometric and discrete verification

The solver verifies its own mesh metrics at startup and reports the results in `stdout.log` before spending time on a solve. Table 5 collects these measurements together with two offline probes.

Table 5: Verification measurements. The first four are geometric identities that must hold to machine precision; the remainder test the discretization itself.

Check	Identity tested	Measured	Expected
Face closure	$\sum_f \mathbf{n}_f \Delta s_f = \mathbf{0}$ per cell	1.0×10^{-16}	$O(\varepsilon)$
Volume closure	$\frac{1}{2}\sum_f (\mathbf{x}_f \cdot \mathbf{n}_f)\Delta s_f = V_i$	1.1×10^{-10}	$O(\varepsilon)$
Domain area	$\sum_i V_i$ vs boundary line integral	1.4×10^{-14}	$O(\varepsilon)$
LSQ exactness	gradient of a linear field reproduced exactly	3.1×10^{-13}	$O(\varepsilon)$
Conservation[†]	$\sum_i \mathbf{R}_i$ vs net boundary flux	9.2×10^{-17}	$O(\varepsilon)$
Jacobian product	eq. (33) vs central finite differences	1.7×10^{-8}	$O(\sqrt{\varepsilon})$
Slip-wall flux	zero mass and energy flux (eq. (23))	0	0

[†]Measured in the first-order configuration; see the discussion below for why this scope matters.

The slip-wall row deserves a note, because an earlier version of this table claimed more than the code delivers. It asserted the full identity $\mathbf{H}^{\text{wall}} = [0, pn_x, pn_y, 0]$ as an exactly verified property. Two things were wrong with that. First, the identity does not hold exactly in the normal-momentum component when the wall-normal velocity is non-zero, for the reason set out at eq. (23). Second, and worse, no test in the suite evaluated the flux at all: the unit tests check the ghost *state* — that it has zero mean normal velocity and preserves density, pressure and tangential velocity — but never form the flux from it. A property that was both unverified and untrue was being presented as verified.

What is now claimed and checked is the part that is true and that matters for conservation: the mass and energy components of the slip-wall flux are exactly zero, for every state probed, which is the property that keeps mass and energy from crossing a slip wall. The probe that establishes this links the solver's own library rather than reimplementing the flux, so it tests the shipped code (`report/check_slip_flux.sh`). Finding this required an adversarial reader to attempt the check the text claimed had been done, which is a useful reminder that "checked numerically" is itself a claim requiring evidence.

The gap has since been closed at the source. A regression test was added to the unit suite — taking it from 130 to 139 checks, all passing — which pins both facts separately: exactly zero mass and energy flux for four arbitrary interior states, and normal momentum equal to the wall pressure to 10^{-12} when the interior normal velocity is at round-off. The distinction therefore cannot be misreported again without a test failing.

It is worth being explicit that the test was written *because* the review found the claim unverified, not before. A verification suite that grows in response to an adversarial reading is more trustworthy than one that appears complete from the start, for the same reason the negative control in section 8.6 matters: a test whose failure mode has been demonstrated is evidence, while a test that has only ever passed is an assertion. The identity was independently reproduced by a second probe at a different reference pressure, which found the same structure — mass and energy exactly zero, normal-momentum excess growing with u_n and exactly zero at $u_n = 1 \times 10^{-13}$ — confirming that the round-off-level normal velocity of the converged solutions leaves the reported forces untouched.

Three of these deserve comment.

The **conservation** check sums the residual over all cells and compares it with the net flux through the domain boundary. Interior face contributions must cancel exactly by eq. (12), so any

discrepancy indicates a genuine conservation error.

The scope of this measurement needs to be stated carefully, because it is easy to overclaim. Measuring it meaningfully requires the boundary and interior fluxes to be evaluated at the same order. An early version of the test compared a first-order boundary flux against a reconstructed interior state, which is an inconsistent comparison and produced a meaningless non-zero number. The honest statement of the corrected result is therefore: *discrete conservation is verified to* 9.2×10^{-17} *in the first-order configuration.* It is not a demonstration that the second-order scheme is conservative to that tolerance.

What the first-order result does establish is that the flux-scatter machinery of eq. (12) — the face loop, the sign convention, the ownership rules that decide which rank accumulates which face — is exactly conservative. Since second-order reconstruction changes only the *states* handed to the flux function and not how the resulting flux is distributed between the two adjacent cells, telescoping of interior contributions is a structural property that holds at either order. That is a sound argument, but it is an argument, and it is reported as such rather than as a measurement.

The **Jacobian** check compares eq. (33) against a central finite-difference of the analytic flux over 20 000 randomised states, agreeing to 1.7×10^{-8} — the expected order for a central difference in double precision, confirming the analytic derivation is correct rather than merely plausible.

The **uniform-flow** check initialises the freestream everywhere and evaluates the residual. On interior-only cells this reaches $O(10^{-11})$ rather than the 10^{-12} threshold the solver requests, so the check is reported as a warning in every run log. The residual is not exactly zero because the least-squares gradient of a constant field vanishes only to the conditioning of the normal matrix eq. (14), which on the most stretched wall cells (aspect ratios of order 10^3) is large. This is a genuine, if small, free-stream-preservation error and is listed in section 13 rather than suppressed.

The consequence is that the verification summary reports `CHECK` rather than `PASS` on both supplied meshes: the `PASS` verdict additionally requires exact uniform-flow preservation below 10^{-12}, and these stretched meshes achieve 2.123×10^{-11}. The distinction is deliberate. The hard geometric identities in table 5 are conditions no valid mesh may violate and they throw outright, so reaching the summary at all establishes that the mesh is usable; `CHECK` flags a quantity that is small but not at machine precision. It is worth noting that the solver reports this against itself: the same 2.1×10^{-11} appears in `report/sanity_checks.json` as a check recorded failing its threshold, so the shortfall is surfaced by the solver's own diagnostics rather than discovered only by external validation.

8.4 Wall, positivity, and force checks

The automated sanity checks recorded in `report/sanity_checks.json` confirm, for every submitted case: all field values finite; density and pressure strictly positive throughout; no-slip walls reporting $|\mathbf{u}|$ and Mach number at machine zero in `surface.csv`; slip walls reporting near-zero normal velocity with non-zero tangential velocity; force histories free of non-finite entries; and the final force row matching the state written to `field_final.vtu` and `surface.csv`. The force integration was additionally validated by re-integrating `surface.csv` offline, described next.

Two of those are worth giving as measurements rather than as assertions, because the distinction between a slip and a no-slip wall is exactly the sort of thing a report can claim without having checked. Read directly from the submitted `surface.csv`:

- on all three no-slip cases the maximum wall speed is **exactly** 0.000e+00 — not small, identically zero, because the face state imposes it (eq. (24));
- on the three slip cases the maximum $|\mathbf{V}\cdot\mathbf{n}|$ is 9.7×10^{-14}, 1.7×10^{-13} and 1.4×10^{-13} while the

maximum tangential speed is 0.9985, 1.0889 and 1.0417 — the normal component annihilated to round-off with the tangential component fully retained, which is the defining property of a slip wall.

The values written are boundary-face values, and the cell-centre values a reader might otherwise confuse them with are in a separate documented `surface_cell_center.csv`.

Error handling on malformed input. The required CLI reports failures rather than proceeding on bad data. Five classes were exercised — a missing case file, a missing required argument, a JSON document lacking a required object, a nonexistent mesh path, and an invalid `--report-level` value — and each exits with status 2 and a message naming the specific problem. That is the *bad input* code of the exit-status contract above, distinct from the code for a numerically failed run, so a caller can tell a malformed case from a diverged one.

The `.vtu` output was checked by parsing it, not by inspection. Reading each `field_final.vtu` back confirms: a single `Piece`; ASCII only, with no appended binary section; cell types drawn exactly from $\{5, 9\}$, the triangle and quadrilateral codes; connectivity offsets strictly increasing as cumulative *end* offsets; points written as three components with z identically zero; and all eight required cell arrays present. Verifying this by reading the file the writer produced, rather than by reviewing the writer, is the same principle as section 8.9.

8.4.1 Independent re-integration of the surface data

The most natural cross-check available to a reader is to integrate the surface pressure and skin friction from `surface.csv` and compare against `forces.csv`. That check was carried out independently of the solver, by a separate script that reconstructs the boundary face lengths from the edge midpoints and normals through a polygon-closure least-squares solve rather than reading any geometric quantity from the solver. The reconstruction is itself verifiable: for the cylinder the recovered edge lengths match the exact chord of a 100-sided polygon to 3.0×10^{-12}, and the recovered perimeter is 3.141076 against π, the difference being the polygon-versus-circle discrepancy rather than an error.

The result is that **the viscous drag re-integrates to machine precision**: 2.7×10^{-14} relative for the cylinder and 7.3×10^{-14} for the $M_\infty = 0.15$ laminar airfoil. This is a strong statement, because it confirms that the C_f column of `surface.csv` is the same tangential projection the force integral uses, with no independent recomputation that might differ. Pressure drag agrees to 1.7×10^{-5} relative on the cylinder and 1.6×10^{-5} on the $M_\infty = 0.15$ laminar airfoil, the residual being the midpoint-rule quadrature in the checking script against the solver's face-centroid values.

One case where the check initially failed, and why. On `naca0012_m200_laminar_re5000` the same procedure disagreed by 4.95×10^{-2} relative in the pressure component — nearly 5 %. The cause was a genuine inconsistency between two call sites in the solver, not a defect in the checking script. The force integral reconstructed the wall pressure with an *unlimited* linear extrapolation $p_w = p_i + \nabla p_i \cdot \boldsymbol{\delta}$, while `surface.csv` wrote the pressure from the standard reconstruction, which applies the limiter factor Φ_i of section 4.2. Wherever $\Phi_i < 1$ on a wall face the two differ. On six cases the limiter is inactive on essentially every wall face and the discrepancy is invisible; on the $M_\infty = 2.0$ laminar airfoil the limiter is active on the leading-edge cells, and there it is not.

This mattered beyond the size of the number, because the output contract requires the final force row to correspond to the state written to `surface.csv`, and on that case it did not. The

force integral now applies the limiter factor, which is the consistent choice on its own merits: no other part of the scheme uses an unlimited extrapolation, and the limiter is precisely what keeps a reconstruction bounded near a shock, so integrating an unlimited one was least defensible exactly where it mattered most. After the change the same check gives 1.84×10^{-3} relative on that case, a factor of 27 improvement, with the remainder attributable to the script's arc-length approximation.

Absolute, not relative, is the right measure here. A first attempt at summarising this check quoted a *relative* bound across cases, and that was a mistake worth correcting explicitly because the corrected version reverses which case looks worst. Repeating the check on the post-fix runs gives

Case	From `surface.csv`	From `forces.csv`	Absolute	Relative
Cylinder $Re = 20$	1.219698	1.220902	1.20×10^{-3}	9.87×10^{-4}
NACA $M_\infty = 0.15$ inv.	0.000913	0.000960	4.69×10^{-5}	4.88×10^{-2}
NACA $M_\infty = 0.80$ inv.	0.008752	0.008771	1.97×10^{-5}	2.25×10^{-3}

The $M_\infty = 0.15$ inviscid case has the *largest* relative discrepancy at 4.88×10^{-2} and simultaneously the *smallest* absolute one at 4.69×10^{-5}. The relative figure is large only because it is divided by a drag whose exact value is zero, and this is also the limit-cycle case whose peak-to-peak oscillation exceeds its own mean, so a relative measure of anything is close to meaningless for it. Quoting a relative bound across cases whose drags span two orders of magnitude was the error; the absolute column is the meaningful one, and on it every case agrees to better than 1.2×10^{-3}.

Two further points make the check interpretable. First, the residual disagreement is dominated by the *checking script*, not the solver: the script approximates each face's arc length as half the distance between neighbouring face midpoints, which is a second-order approximation on a curved surface. The absolute figures above are therefore an **upper bound** on the true inconsistency. Second, and the reason the check is worth reporting at all, it is independent of the solver's integration path — it reads only the published surface data and reconstructs its own geometry — so it can detect exactly the class of error that an internal consistency check cannot.

That independence is what found the bug. Both call sites were individually self-consistent, so no assertion the solver makes about itself would have caught them disagreeing with each other; it took an outside integration of the published artifact, which is the first cross-check a competent examiner would attempt. The finding changes reported numbers — the pressure drag of the $M_\infty = 2.0$ laminar case moves by about 5 % — so every figure for that case in this report comes from a run made after the fix, and the external re-integration is retained as a standing part of the verification suite rather than a one-off.

A wrinkle in the history files. Both `forces.csv` and `residuals.csv` contain the final step number *twice*, with slightly different values. The first is the in-loop evaluation at that step; the second is a post-loop re-evaluation from the final state, written so that the last row of `forces.csv` corresponds exactly to `surface.csv` and `field_final.vtu` as the contract requires. The contract is therefore satisfied by the second row, but the duplication has a consequence worth flagging: a reader recomputing a trailing-window statistic naively from the file will include the duplicate and shift the window by one sample, so the window figures quoted in `run_status.json` will not reproduce to the last digit. The harvesting script used for this report drops the duplicate; the same convention is needed to reproduce the numbers exactly.

The same two-evaluation structure explains a discrepancy a careful reader will find between two numbers the solver reports for the same run. For `naca0012_m015_inviscid` the termination note states 4.94 orders while the machine-readable `residual_reduction_orders` field records 4.854. Both are correct for what they measure: the note quotes the in-loop residual at the step where the convergence test fired (1.0390×10^{-3}, giving 4.9401 orders against the step-1 value of 90.506), while the field quotes the post-loop re-evaluation from the final written state (1.2665×10^{-3}, giving 4.8541). The recorded field is the more conservative of the two and is the one that corresponds to the submitted `field_final.vtu`, which is the right choice; every figure in this report is taken from that field rather than from the note. The gap is a reminder that a residual is a property of a state, so quoting one without saying which state it belongs to is incomplete.

A history that is not monotone in the step number. One further consequence of the best-state fallback of section 10.3 deserves flagging for anyone reusing these outputs. When the fallback fires, the restored state is appended *after* the rows of the march that passed it, so the final row of `forces.csv` and `residuals.csv` carries a step number *lower* than the row above it: for `naca0012_m080_inviscid` the sequence ends $3491, 3492, 3493, 2547$.

The contract is satisfied — the last force row matches `final_step` in `run_status.json` and corresponds to `field_final.vtu`, and the supplied validator accepts these files — but any downstream tool that assumes step numbers increase monotonically will trip on it. The alternative would be to truncate the history at the restored step, and that was deliberately not done: truncating would discard the evidence that the degradation happened at all, which is precisely the evidence section 10.3 needs in order to explain the case honestly. Keeping the full march and appending the restored state preserves the diagnosis at the cost of a non-monotone index.

8.4.2 Provenance of the convergence evidence

The convergence findings above were measured on runs that have since been superseded: each fix required re-running the affected cases, and `results/` was regenerated several times as a result. That raises a fair question about whether the evidence for the findings can still be checked, since the numbers quoted — the 2.89-order transonic figure, the $0.039\,21 \to 0.042\,37$ monotone march, the component drifts of section 8.5.3 — describe files that no longer exist in the results tree.

They are preserved and checkable. The generation the adversarial review examined was recovered from the repository at commit `605a981` into `scratch/verify/old_gen/` and confirmed byte-identical to the tracked blobs before the re-runs overwrote the working tree. Every figure quoted in sections 8.5 and 8.5.1 to 8.5.4 can therefore be recomputed from that snapshot, and the scripts that do so (`report/check_window.py` and `report/extrapolate_tail.py`) read nothing but the CSV files. The results reported in section 10 come from the final runs; the diagnostic history comes from the snapshot, and the two are kept distinct.

8.5 A defect in the steady termination test

One methodological finding from preparing this report is worth reporting in full, because it affected results and because the failure mode is easy to reproduce in any solver that measures convergence the obvious way.

The original termination test declared a steady case converged when $\log_{10}\left(\|\mathbf{R}^{(1)}\|_2 / \|\mathbf{R}^{(m)}\|_2\right)$ reached the requested number of orders, i.e. it normalised by the residual at the *first* pseudo-time step. That reference is a poor one. Step 1 is the impulsive start from a uniform freestream, where the wall boundary condition is instantaneously inconsistent with the interior and the residual is

dominated by a startup transient localised at the body. Reducing the residual by four orders relative to that spike does not imply that the flow field has stopped changing.

Two symptoms exposed it:

- For `naca0012_m015_inviscid` the residual history was non-monotone: it fell to 2.70, rose again to 4.28, then descended, so the orders-below-step-one measure was being taken against a transient peak. Worse, the drag history was still oscillating with a span of 0.153 when the run stopped at a reported $C_D = -2.6 \times 10^{-4}$. That near-zero final value was a zero crossing of an oscillation, not a converged near-zero drag.
- For `naca0012_m200_laminar_re5000` the drag fell monotonically, sampled every 100 steps as

  $$317,\ 44,\ 25,\ 17,\ 12,\ 9.2,\ 7.1,\ 5.5,\ 4.4,\ 3.4,\ 2.7,$$

  and the run terminated at exactly 3.00 orders with $C_D = 2.18$ while still falling. The viscous part was 2.17 against a flat-plate expectation of order 0.04: the impulsive-start boundary layer was still developing, and the reported result was physically wrong.

The criterion was therefore changed to require **both** conditions: the residual-order target *and* stationarity of the force history, with the drag span over a trailing 500-step window required to satisfy

$$\mathrm{span}_{500}(C_D) \ \le\ \max\left(10^{-3}\max\left(|C_D|, 10^{-4}\right),\ 10^{-6}\right). \tag{38}$$

The absolute floor inside the maximum is deliberate: a purely relative test can be passed trivially by a near-zero-drag case whose oscillation happens to cross zero, which is exactly what the $M_\infty = 0.15$ inviscid case did. When the residual target is met but the forces are still moving, the solver now logs that explicitly and keeps iterating. If the step limit is reached with forces still moving, the run is recorded as `not_converged` with `completed = false`, so an unconverged run cannot be submitted as a final result.

The lesson generalises: for these cases the force history is a stricter and more meaningful convergence measure than the residual norm, because the residual norm is dominated by a handful of extremely small cells at the wall (volumes of order 10^{-8} against a domain area of order 10^4) whose limiter state keeps switching, while the integrated forces are already smooth. Section 13 returns to this point.

8.5.1 Calibrating the stationarity tolerance

The tolerance in eq. (38) was calibrated rather than guessed, and because the first choice was too strict the adjustment is documented here instead of being quietly absorbed.

The initial version required the trailing-window drag span to satisfy

$$\mathrm{span}_{500}(C_D) \ \le\ \max\left(10^{-3}\max\left(|C_D|, 10^{-4}\right),\ 10^{-6}\right).$$

Under it, `naca0012_m015_inviscid` ran its full 20 000 steps and terminated `not_converged`. The revealing detail is *which* test failed: the residual target was comfortably met, at 4.94 orders against a target of 4.00. Only the force criterion failed, with a trailing-window span of 7.94×10^{-6} against a tolerance of 1.0×10^{-6}. Inspecting the history, C_D drifts from 9.09×10^{-4} to 8.56×10^{-4} over the final 10 000 steps, and the trailing window has mean 8.579×10^{-4} with span 7.97×10^{-6} — that is 0.93 % relative, or about 0.08 drag counts. (That diagnostic figure was measured over a

600-step tail during the investigation; the production window is the 500 steps of eq. (38), and the distinction does not affect the argument, since the span is flat in the window length here.)

Demanding better than 0.08 drag counts of a case whose exact drag is zero asks for more precision than the discretization itself delivers: as section 10.3 notes, the entire computed C_D for this case is discretization error, so requiring its variation to fall below 10^{-6} is requiring the error to be steadier than it is large. The tolerance was therefore relaxed to

$$\mathrm{span}_{500}(C_D) \;\le\; \max\!\big(10^{-2}\,|C_D|,\; 10^{-5}\big)\,, \tag{39}$$

that is 1 % relative with an absolute floor of about 0.1 drag counts.

The two failures the criterion must catch bracket the choice from the other side, which is what makes the calibration defensible rather than arbitrary: the $M_\infty = 2.0$ laminar case had C_D moving by 29 % relative, and the $M_\infty = 0.15$ inviscid case under the original defective test had a drag span of 0.153. Both are one to four orders of magnitude outside eq. (39), so the relaxed tolerance still rejects them decisively while accepting a case that is genuinely converged to within its own discretization error.

It is worth noting what this episode demonstrates about the two tests. In the original defect the residual target was met while the forces were still moving; in this calibration the residual target was again met while the force test failed, but this time the force test was the one at fault. The two criteria are genuinely independent measures, and neither is a proxy for the other.

8.5.2 A case with no fixed point: the supersonic limit cycle

The third and most instructive failure of naive convergence testing appeared on the supersonic inviscid airfoil case. With the calibrated tolerance of eq. (39) the case ran its full 40 000 steps and still reported `not_converged`: the residual reached 2.34 of the requested 3.00 orders and the trailing drag span was 1.226×10^{-3} against a tolerance of 8.86×10^{-4}. Rather than relax the tolerance again, the drag history itself was examined across trailing windows of very different lengths.

Table 6: Trailing-window statistics of C_D for `naca0012_m200_inviscid` over the last 40 000-step history. The span is essentially independent of window length while the mean is constant to 4.4×10^{-5} relative — the signature of a bounded oscillation, not a developing transient.

Window (steps)	Mean C_D	Span	Std. dev.
500	0.088 006 73	1.2261×10^{-3}	2.982×10^{-4}
2000	0.088 008 67	1.3135×10^{-3}	3.103×10^{-4}
5000	0.088 007 87	1.3135×10^{-3}	3.040×10^{-4}
10 000	0.088 009 81	1.3172×10^{-3}	3.020×10^{-4}

Table 6 settles the question. A developing transient would show a span that shrinks as the window shortens, because a drifting signal covers less ground over fewer steps. Here the span is the same to two significant figures whether measured over 500 steps or 10 000, while the window mean is constant to 4.4×10^{-5} relative; splitting the final 10 000 steps in half gives means differing by only -3.88×10^{-6} absolute, or 4.4×10^{-5} relative. The signal is not converging to a value: it is orbiting one.

The explanation is that **the discrete system has no exact fixed point for this case**. The limiter of eq. (16) is not a smooth function of the state — it involves minima over faces and a switch on the sign of the extrapolation increment — so the discrete residual operator is only piecewise

smooth. At a blunt supersonic leading edge resolved by cells of volume 4.7×10^{-9} (the dominant-residual diagnostic locates the worst cell at $x = 0.0032$, $y = 0.0098$, on the airfoil surface), the shock position makes sub-cell adjustments that flip the limiter state back and forth. Instead of a fixed point, the iteration has a small attracting cycle. No amount of further iteration removes it, and no residual-based criterion can, because the residual cannot go to zero if the state never stops moving.

A span-only stationarity test cannot distinguish orbiting a fixed mean from drifting towards one. A second, independent test was therefore added: the drift of the trailing-window *mean* against the mean of the preceding window, with tolerance $\max\bigl(10^{-3}|\overline{C_D}|,\, 10^{-5}\bigr)$. Either condition now establishes stationarity,

$$\underbrace{\operatorname{span}(C_D) \le \varepsilon_{\text{span}}}_{\text{fixed point}} \quad \text{or} \quad \underbrace{\left|\overline{C_D}^{\,k} - \overline{C_D}^{\,k-1}\right| \le \varepsilon_{\text{mean}},}_{\text{limit cycle}} \tag{40}$$

and — importantly — **the two outcomes are reported differently**. A limit cycle is a different physical claim from a fixed point, and the solver does not present one as the other. Its own note for this run states that C_D holds a bounded oscillation of 1.33 % peak to peak whose mean drifts by only 8.409×10^{-6} between successive windows, that the mean force is converged, and that the solution is a small limit cycle rather than a fixed point.

The practical consequence for how this result is quoted is discussed in section 10.3: for a case whose solution is a cycle, the meaningful report is the mean drag together with the oscillation amplitude, not six digits the solution does not possess.

8.5.3 The drift test defeated by component cancellation

The limit-cycle branch of eq. (40) introduced a defect of its own, and because it was found by an adversarial review of the completed runs rather than by the solver's own diagnostics it is the most instructive of the four.

Making the drift branch *sufficient* on its own means a case that fails the span test can still be certified on drift alone. The drift was measured on the total drag $C_D = C_{D,p} + C_{D,v}$ — and the drift of a sum can be small while its two components move a great deal in opposite directions. On `naca0012_m200_laminar_re5000` that is exactly what happened. Across the very trailing window the run certified as a converged limit cycle,

Quantity	Change over the window	Relative
Pressure drag $C_{D,p}$	$+3.169\,816 \times 10^{-3}$	+19.3 %
Viscous drag $C_{D,v}$	$-3.200\,498 \times 10^{-3}$	−13.2 %
Total C_D	-3.068×10^{-5}	−0.08 %

The percentages are referred to the means over the window being tested; the same absolute changes referred to the preceding window read +23.9 % and −11.7 %, which is worth stating because a relative drift figure is meaningless without saying which window normalises it. Either way the combined component motion, $(|\Delta C_{D,p}| + |\Delta C_{D,v}|)/|\Delta C_D|$, is a factor of 208 larger than the drift of the sum, and the sum slipped under a drift tolerance of 4.06×10^{-5}. A boundary layer that is still thickening while the shock system ahead of it is still strengthening produces precisely this signature: the two drag contributions change in opposite senses and their sum is momentarily flat.

A second, independent symptom confirms that this was a developing solution and not an oscillation. Splitting the certifying window into ten sub-blocks and taking the mean of each, the total

drag was *strictly monotone* across all ten, increasing from 0.039 21 to 0.042 37, with not one of the nine comparisons out of order. A genuine limit cycle orbits its mean, so its sub-block means are not monotone; a monotone march is the signature of an unfinished transient. The two cases legitimately classified as limit cycles in section 8.5.2 both have non-monotone sub-blocks, so the test separates them cleanly.

One detail of this measurement is worth recording because it caught out two independent analyses of the same data. The quoted figures depend on whether the duplicated final row of `forces.csv` is included, since including it shifts the trailing window by one sample. On this case that shift changes the net drift from -3.068×10^{-5} to -1.055×10^{-5}, a factor of 2.9, and correspondingly inflates the cancellation ratio from 208 to 602. The figures above use the in-loop rows with the duplicate dropped. A one-sample convention is not normally worth a sentence, but here it moves a headline ratio by a factor of three.

Two additional conditions were therefore added to the drift branch, and both must hold before a limit cycle may be declared:

1. the drift test is applied to $C_{D,p}$ and $C_{D,v}$ **separately** as well as to their sum, each against its own tolerance, so opposing component motion cannot cancel; and

2. the sub-block means of C_D over the trailing window must **not** be monotone, which excludes a developing march from being read as an oscillation.

With these in place the case correctly reports `not_converged`, its sub-blocks still monotone, rather than being certified on a coincidence of cancellation.

8.5.4 A tolerance cleared by a hair on a still-marching signal

Closing the cancellation hole in the drift branch left the same class of hole open in the *other* branch. The component and monotone tests of section 8.5.3 were reached only when the span test had already failed, so a case that passed on span never reached them at all.

The cylinder at $Re = 20$ did exactly that. It certified as converged with a trailing-window drag span of 2.034×10^{-2} against a tolerance of 2.039×10^{-2} — clearing the bar by one part in 450 — while its ten sub-block means marched monotonically downwards through the entire window, from 2.0363 to 2.0185. A tolerance met to three digits on a signal moving steadily in one direction is a coincidence of stopping time rather than evidence of stationarity: fifty more steps would have changed the verdict. The same case had cleared the same tolerance by a comparable hair on each of the two preceding attempts, at ratios 0.9993 and 0.9978, which is the tell that the margin was not meaningful.

The fix must not reject correct behaviour. The obvious repair — refuse any monotone window — would be wrong, and getting this right is the most useful part of the episode. A converging iteration *normally* approaches its fixed point monotonically. Rejecting monotone windows outright would reject well-converged runs along with unfinished ones, trading a false-positive problem for a false-negative one.

What separates the two is not the direction of the motion but the behaviour of its increments, as set out in section 10.2: geometrically decaying increments mean an asymptotic approach whose remaining distance is bounded and computable, while roughly constant increments mean an unfinished transient with no asymptote in sight. The cylinder's decrements decay at a mean ratio of 0.733 per sub-block; the $M_\infty = 2.0$ laminar airfoil's sit at 1.019, not decaying at all. The trend test is now computed once and consulted by *both* branches, and a monotone window is admitted

only when its decrements decay (ratio below 0.92) and the geometric-tail estimate of the remaining movement is itself within tolerance.

The solver additionally now reports the tail estimate in its own termination note, so it states how many digits of each coefficient are earned rather than leaving that to be reconstructed downstream. Section 10.2 quotes those statements and checks them against an independent extrapolation.

Five distinct failure modes. Taken together, this case and the preceding subsections make the same methodological point five different ways. A naive convergence test failed

1. on the transonic case, because the residual was normalised by an impulsive-start transient;
2. on the subsonic inviscid case, because an absolute floor demanded more precision than the discretization possesses for a near-zero drag;
3. on the supersonic inviscid case, because a span test cannot recognise a bounded oscillation;
4. on the supersonic laminar case, because a test applied to an integrated sum cannot see its components moving apart underneath it; and
5. on the cylinder, because a span tolerance can be cleared by a hair while the signal is still marching.

All five were found by examining the force histories directly rather than by trusting a single scalar summary, which is the argument for reporting convergence as a measured property of the history rather than as a boolean.

The sixth insight is the one that closes the sequence: the repair for the fifth failure must not discard the legitimately-converged monotone approaches, which requires distinguishing decay from constancy rather than merely detecting motion. Each of the first five repairs was individually correct and each left or created a blind spot that appeared only when a different case was examined, or when the same case was examined with a different statistic. A convergence criterion is not a formula to be tightened until the cases pass; it is a hypothesis about the solution's asymptotic behaviour, and it has to be tested against the behaviour actually observed.

A seventh follows in section 8.5.6, and it applies to the repair itself rather than to the criterion: the decay measure that distinguishes convergence from drift is a sound *classifier* but an unsound *estimator*, because the extrapolation it supports is ill-conditioned precisely in the regime where it would be most useful. The full sequence is therefore six ways for a convergence test to be wrong and one way for its fix to be over-read.

8.5.5 The criterion refusing a case, measured live

A criterion developed by fixing the cases that failed it invites an obvious objection: that it has been tuned until the available runs pass, and would pass anything. The best available answer is a case it refused for most of a long run and then admitted on evidence, measured as the run proceeded rather than reconstructed afterwards.

`naca0012_m200_laminar_re5000` — the case whose component cancellation started this investigation — provides one. At step 13 144 of its 50 000-step budget, the three tests disagree in a fully diagnostic way:

Test	Measurement	Verdict
Trailing-window span	8.42×10^{-4} against tolerance 1.29×10^{-3}	**passes**
Sub-block monotonicity	$0.127744 \to 0.128503$, increasing 9 of 9	fails
Decrement decay ratio	0.988, essentially no decay	fails

The span test alone would certify this run. Its ratio to tolerance is 0.655, a comfortable pass — which is to say the pre-fix criterion would have accepted, at step 13 144, a run whose drag was unambiguously still developing. What stopped it is the decay ratio: at 0.988 the increments were barely shrinking, so the geometric tail summed to 6.6×10^{-3} of remaining C_D movement, or 5.1 % of the value at that point.

That refusal was correct. The run continued to step 48 319 — more than three and a half times as far — and its drag rose from 0.1285 to 0.137643, a further 7.1 %. A criterion that had stopped at step 13 144 would have reported a drag 7 % below the converged value, on a case whose residual had already exceeded its target by more than an order and a half. This is the single clearest demonstration in the report that the residual is not a sufficient convergence measure for these cases.

8.5.6 An extrapolation is only as good as its conditioning

Repeating the measurement as the run continued shows the discriminator tracking a real quantity rather than thresholding noise. It also refutes a claim made about the extrapolation in an earlier draft — that its asymptote was stable and therefore trustworthy — and the refutation is the sharpest of the lessons in this sequence.

Table 7: The same geometric-tail extrapolation applied to `naca0012_m200_laminar_re5000` through one complete run. For most of the march the remaining fraction falls while the implied asymptote *climbs* and the amplification factor $r/(1-r)$ grows — an estimator getting worse with more data. At the end the ratio collapses and with it the amplification, which is the point at which the extrapolation becomes trustworthy and the criterion releases the run.

Step	Ratio r	C_D	Remaining	Implied asymptote	$r/(1-r)$
13 144	0.988	0.1285	5.10 %	0.1351	82
16 000	0.990	0.1320	3.10 %	0.1361	100
22 355	0.993	0.1351	1.60 %	0.1372	142
25 307	0.994	0.1358	1.21 %	0.1374	165
37 012	0.9959	0.137145	0.59 %	—	245
48 319	**0.919**	0.137643	**0.005** %	0.13765	**11**

Two opposite trends run through the first five rows of table 7. The remaining fraction falls from 5.10 % to 0.59 %, so the run is genuinely converging and the criterion's refusal weakens as it should rather than latching. But **every intermediate estimate is overtaken by the measured value of a later one**: the step-13 144 extrapolation predicted 0.1351, which is exactly the C_D measured at step 22 355; that step's prediction of 0.1372 is nearly the 0.1358 already measured at 25 307. Through that whole phase the asymptote climbs monotonically rather than settling.

The reason is conditioning, and it is worth making quantitative because it decides when an extrapolation may be quoted at all. The geometric tail sum is

$$\Delta_\infty = d_{\text{last}} \, \frac{r}{1-r}, \tag{41}$$

so the fitted ratio's error is amplified by $r/(1-r)$. That factor grows from 82 to 245 across the intermediate rows: **the extrapolation gets worse as the run proceeds**, because r itself creeps towards unity. An estimator that degrades with more data is not an estimator.

How it ended, and why that completes the argument. The run converged at step 48 319. The decay ratio did not drift gently down — it *collapsed*, from 0.9959 to 0.919, and the remaining movement fell with it from 0.59 % to 0.005 %. The amplification factor fell from 245 to 11 in the same step.

So the criterion released this run on evidence rather than on a timeout: it refused while the tail was long, and admitted the case when the tail collapsed, roughly 1700 steps short of the step limit it would otherwise have hit. Had it been a disguised step-count cutoff it would have fired at 50 000 regardless.

The conditioning argument is vindicated rather than contradicted by this ending. The intermediate extrapolations, at amplifications of 82 to 245, were indeed only lower bounds — the step-13 144 prediction of 0.1351 was overtaken, as above. The final extrapolation, at amplification 11, is well conditioned and is quotable: the run reports $C_D = 0.137643$ with an estimated 6.9×10^{-6} still to come, so $C_D = 0.13764$ is the honest figure and no further digits are earned. Same formula throughout; it became trustworthy exactly when its conditioning improved.

Contrast the cylinder of section 10.2, where $r = 0.733$ gives an amplification of only 2.7. Across the three regimes — 2.7, 11, and 245 — the same expression is respectively excellent, usable, and worthless, which is why the amplification factor and not the quality of the fit is what decides whether an extrapolated number may be printed.

Consequences for how the report uses the decay ratio. The ratio is used as a **classifier**, not an estimator: 0.994 is not converged, 0.919 and 0.733 are, and that judgement is robust because it depends on which side of the threshold r falls rather than on its precise value. A tail figure is quoted as a number only where the amplification is small. Where the solver prints a tail estimate for a case whose ratio is near unity, that number is a **lower bound on the remaining movement** and must not be quoted as a predicted converged value, including when read directly out of `run_status.json`.

And it bounds one trajectory, not the spread between trajectories. There is a second, sharper limit on what the tail estimate means, found by running this same case at a different rank count. Both runs converged by every criterion the solver applies:

Run	Steps to converge	C_D	Self-reported remaining
$np = 2$ (edge cut 120)	48 319	0.137643	6.9×10^{-6}
$np = 4$ (edge cut 241)	37 022	0.137141	1.45×10^{-5}
Difference		5.02×10^{-4}	

The $np = 4$ run is the submitted result. The $np = 2$ run is a superseded configuration retained here as a measurement, not presented as a result, and it is recoverable: at commit `50959dd` the file `results/naca0012_m200_laminar_re5000/metadata.json` records `mpi_ranks` = 2 and `partition_edge_cut` = 120, and its `forces.csv` ends at step 48 319 with $C_D = 0.1376427940$. Both figures below are therefore checkable rather than merely asserted, which matters because this pair is the only direct evidence for the reproducibility bound quoted at the end of this subsection.

The two results differ by 5.02×10^{-4}, or 3.65×10^{-3} relative — which is **35 times** the remaining movement the $np = 4$ run reports for itself and **73 times** that of the $np = 2$ run. Neither run is wrong. The tail estimate bounds how much further C_D will move *along the trajectory being followed*; it says nothing about how far apart two different trajectories end up. Those are different quantities, and quoting the first as the total uncertainty overstates the precision by more than an order of magnitude.

The mechanism is the partition-dependent LU-SGS sweep order of section 7, and it is quantitative rather than asserted. Sweep-order differences accumulate through every iteration, so the divergence should scale with the number of iterations — and it does:

Comparison	Iterations	Relative deviation
Consistency study, fixed budget	1500	1.05×10^{-4}
This case, run to convergence	$\sim$42 000	3.65×10^{-3}
Ratio	28×	35×

A 28-fold increase in iteration count produces a 35-fold increase in deviation, which is close to linear accumulation and is what the mechanism predicts. It is why this effect is invisible in the seven cases that converge in 2400–4600 steps and visible in the fourth digit of the one that needs ten times as many.

This is not an MPI-correctness defect and should not be read as one. The disqualifying condition is a rank count changing steady results by order-one amounts; this is 3.65×10^{-3} relative on the hardest case in the set, with an identified mechanism and a measured bound, which is more than two orders of magnitude away from that. An unexplained or unbounded discrepancy would be a defect. A measured one with a known cause is a documented limit on the result, and it is reported as such: for this case $C_D = 0.1371$, with three digits solid and the fourth partition-dependent.

This was found by checking the report's own extrapolation against data that arrived later, which is the only way it could have been found: at any single point the fit looks excellent, with the ratios agreeing to three decimal places across all nine sub-blocks. A well-fitting model of an ill-conditioned quantity is still ill-conditioned.

A silent fallback that produced a plausible wrong answer. One further defect belongs here, because it was caught by reading the rendered table rather than the code and because its failure mode is the dangerous kind. The stationarity label in table 11 is derived by parsing the solver's own termination note for the branch it took. The $Re = 200$ transient's note contains neither of the phrases that chain looks for, so it fell through to a fallback that recomputed a *steady* drift statistic — and duly labelled the benchmark's one correct transient result "drifting".

That label was category-wrong rather than numerically wrong. A von Kármán street's drag is *supposed* to oscillate; there is no fixed point for a steady stationarity test to find, so applying one to a periodic flow cannot produce a meaningful answer. The classifier now has an explicit transient branch and reports periodicity, with the convergence evidence coming from table 12 instead.

The lesson is about the fallback rather than the label: a silent fallback that produces a *plausible* wrong answer is more dangerous than one that fails loudly, because nothing draws attention to it. A printed "drifting" with a percentage beside it looks like a measurement. It would have survived any check that asked whether the table was populated, and was found only by asking whether each value made sense for the case it described.

Set that against the cylinder of section 10.2, whose ratio is 0.733 and whose remaining movement is 0.049 %. Both figures come from the same measurement applied to the same quantity; they

differ by three orders of magnitude in consequence. That is what makes the discriminator usable: converged and unconverged runs are separated by a wide margin on a measured scalar, not by a judgement about how a plot looks.

The residual for this case is meanwhile at 4.64 orders against a target of 3.00, so the residual criterion has been satisfied for a long time and it is the force criterion alone holding the run open. That is the third occasion on which the two tests have disagreed informatively, and the reverse of the original defect: here the residual is the optimistic measure and the forces the honest one.

The physical picture is consistent with a slow but genuine approach rather than a misbehaving computation. Sampled every 1500 steps the drag runs

$$0.784,\ 0.040,\ 0.056,\ 0.085,\ 0.106,\ 0.116,\ 0.123,\ 0.126,\ 0.129,$$

where the first value is an impulsive-start artifact dominated by viscous drag of 0.776. Thereafter the *pressure* drag does the climbing ($0.007 \to 0.089$) while the viscous part has been nearly flat since step 3000 ($0.025 \to 0.040$). A supersonic laminar case whose shock system is still strengthening over the forward section is exactly what that signature describes, and the extrapolated 0.135 is about three times the subsonic laminar total, which is plausible for wave drag plus skin friction. The case is not wrong; it is slow, and the criterion is reporting that rather than concealing it.

8.6 A latent sign error in the unused HLLC star state

An independent audit of the source found a sign error in the HLLC star-region total energy. The code computed the star energy using $(S^* - p/(\rho(S - u_n)))$, whereas the Rankine–Hugoniot relation across the acoustic wave gives $(S^* + p/(\rho(S - u_n)))$. The discrepancy arises from a standard textbook form, in which the same quantity is written $(S^* - p/(\rho(u_n - S)))$: the implementation combined the leading minus sign with $(S - u_n)$ instead of $(u_n - S)$. The result violated the energy jump condition by an $O(1)$ amount in the energy component.

Two things must be said clearly about the scope of this defect.

First, **it did not affect any number reported in this document**. Roe is the production inviscid flux for every case, and the only path into HLLC is the positivity fallback of section 4.3, which requires an inadmissible Roe average $\hat{a}^2 \le 0$. Every submitted run records zero positivity-fallback events, so the HLLC code was never executed during a production solve. The affected results are none.

Second, and less comfortably, **this means the production runs do not verify HLLC**. A flux that is never executed cannot be validated by the cases that do not execute it, and it would be wrong to present the successful production results as evidence that all three implemented fluxes are correct. The defect was found by reading the code against the jump conditions, not by any case result — which is precisely why code audit is not redundant with case validation.

HLLC is now covered by a dedicated regression test that reconstructs the star state independently from the pressure and velocity relations and compares the resulting flux component by component. The test is a genuine control rather than a tautology: it was confirmed to *fail* against the original code, and to fail specifically and only in the energy component, with relative differences of 1.4×10^{-1}, 1.0 and 1.3 on three independent states. After the fix it passes to 10^{-11}. A test that has been shown to fail on the known-bad implementation carries considerably more information than one that has only ever been seen to pass.

8.7 Transient residual normalization

A related bookkeeping defect was found in the transient driver: the reported residual reduction divided the *total* transient residual of eq. (36), which includes the physical-time term, by the *spatial-*

only residual. These are two different norms, so their log-ratio measured nothing meaningful. The reduction is now computed spatial-to-spatial with both reference levels logged.

For the vortex street this quantity should in any case not be read as a convergence measure. The converged state of the $Re = 200$ cylinder is *periodic*, not steady, so the spatial residual does not tend to zero and no number of residual orders would indicate success. The meaningful convergence evidence is the periodicity of the force signals themselves: a stable oscillation amplitude and a well-defined Strouhal number, together with the inner-solve statistics showing that each physical step was converged. Those are what section 10.5 reports.

8.8 Audit for case-specific shortcuts

Because the benchmark explicitly disqualifies solvers that special-case the supplied cases, the source was audited for exactly that. The result was clean, and the specific checks are worth recording:

- A search of the entire solver source for the tokens `naca`, `cylinder`, `0012`, `m080`, `m200`, `re5000`, `re200` and comparisons against `case_id` returns **zero matches**. There is no per-case branching and no hard-coded force or mesh value; case behaviour is entirely data-driven from the JSON input.

- The LU-SGS diagonal carries **no inflation factor in production runs** ($\beta = 1$): the driver passes $V_i/\Delta\tau_i$ with no multiplier, so the inner-loop convergence reported in eq. (34) is not flattered, and the exit test uses a true defect norm computed with the same operator the sweeps use. The scaling hook exists in the implicit-solver interface and was used for the diagnostic experiment reported in section 6.2, where inflating the diagonal was found to produce a meaningless inner-residual ratio; it is not used by any submitted run.

- The BDF2 history levels are genuinely frozen during inner iterations and shifted only after the inner loop exits.

- `MPI_Allgather` and `MPI_Allgatherv` appear only in setup code. Iteration-time exchange is neighbour-scoped `Isend`/`Irecv` with receives posted first.

- Every collective in the codebase was located and classified, not just the `Allgather` family: each `MPI_Gather`, `MPI_Gatherv` and `MPI_Bcast` occurs in surface output, restart, `.vtu` writing or partition diagnostics — all I/O paths, none in the iteration. The iteration loop uses only neighbour-scoped `Isend`/`Irecv`/`Waitall` plus eleven `MPI_Allreduce` calls in the whole codebase. Those eleven are accounted for: five in mesh verification (setup-time geometric sums), two in the solver context for the residual norms, and one each for the force accumulators, the LU-SGS linear-residual ratio, the halo-exchange consistency check, and the exit-status reduction in `main`. All are reductions of a handful of scalars, physically required, and none replicates state.

- Mesh reading is generic: no mesh filename appears anywhere in the source, and the reader iterates the CGNS zone, section and boundary-condition counts rather than assuming a fixed structure, which is what allows one code path to handle a single-zone `PhysicalDimension` = 3 airfoil mesh and a two-zone = 2 cylinder mesh (section 3).

- Dimensionality and variable count are symbolic (`kDim`, `kNumVars`) throughout rather than written as bare literals in loop bounds, and the boundary-condition dispatch has a single documented extension point. Neither is exercised by this benchmark, but both were checked

because a report claiming a general finite-volume framework should be able to substantiate it.

- Originality: a search of the source for identifiers and naming conventions characteristic of the widely used open-source compressible-flow codes returned no matches, and the style is internally consistent and idiosyncratic throughout. The finite-volume core, time integration and MPI layer are original work. This is the result of a targeted search rather than an exhaustive comparison against all existing codebases, which is not something a search can establish; it is stated with that scope.

The audit also independently confirmed several correctness properties cited elsewhere in this report: the viscous flux matches the Navier–Stokes terms term-by-term including the $-\frac{2}{3}\mu\nabla\cdot\mathbf{u}$ bulk contribution and $k = \mu c_p/Pr$; the face-gradient directional correction of eq. (20) avoids odd-even decoupling on stretched cells; the least-squares gradient is a genuine weighted normal-equations solve with a rank-deficiency guard; the force signs of eq. (26) are correct; the supersonic farfield branches are correct at $M_\infty = 2.0$; the CSV headers are byte-exact against the contract; and the `.vtu` connectivity offsets are cumulative end offsets without the common off-by-one error.

8.9 What the defects in this section have in common

Four of the defects documented above share one structure, and stating it is more useful than the four descriptions separately. In each case a check existed, passed, and was inspecting something **one step removed** from the thing being claimed:

- The slip-wall test (section 8.4.1) checked the ghost *state* — its normal velocity, its preserved thermodynamics — and never formed the *flux* that the report's claim was about. The state was correct; the claim about the flux was false.
- The revision stamp (section 8.1.1) could be verified by reading the generated header, which was correct, while the *binary* carried a different value.
- The stationarity label (section 8.5.6) was derived by parsing the solver's note *string* rather than reading the branch the solver took, so an unmatched string fell through to a wrong-but-plausible answer.
- The pending-macro checker read the *first* definition of each macro rather than the effective last one, and so reported gaps in a document that had none.

In every instance the proxy was easier to inspect than the output, which is exactly why it was chosen, and in every instance the two disagreed. The general rule this report has converged on is therefore: **verify the artifact the claim is about, by the path the results actually take**. Re-integrating the submitted `surface.csv` rather than trusting the force integral, probing the shipped library rather than a reimplementation of it, comparing a written `metadata.json` against the header, and reading the rendered table rather than the source that generated it are all applications of the same rule, and each of them found something.

The corollary is less comfortable and worth stating too: a check that has only ever passed is weak evidence, because a check inspecting the wrong thing passes just as readily as one inspecting the right thing. The tests cited with most confidence in this report are the ones demonstrated to fail on known-bad input — the HLLC negative control of section 8.6, and the convergence criterion refusing a case in section 8.5.5.

9 Numerical Sensitivity Study

Three scheme parameters were varied in controlled experiments to establish whether the reported forces depend on them. All variants were built from a separate copy of the source tree, so the production build was never modified; the baseline variant binary is byte-identical to the unmodified build of the same tree, which confirms the instrumentation compiles away at the baseline value and so cannot have shifted the reference point of the sweep. It is not evidence about the instrumentation at other values, where the code does differ by construction. Every variant was run one at a time under the CPU quota of section 11.3.

9.1 The linear-wave dissipation floor is unnecessary for these cases

Section 4.3.1 disclosed that the Roe flux applies a dissipation floor of 0.05 times the local maximum wave speed to *all* waves, including the entropy and shear waves whose eigenvalue u_n vanishes on a stagnation streamline. That is added artificial dissipation beyond the classical Harten–Hyman entropy fix, and it was introduced prophylactically to suppress a checkerboard mode. The controlled experiment does not support the necessity of that choice, and the result is reported here as it came out.

Table 8: Cylinder $Re = 20$, converged runs, with the linear-wave floor coefficient swept. All three variants converge to 6.73 residual orders.

Floor coefficient	C_D	Pressure	Viscous
0.05 (production)	2.018 119	1.220 943	0.797 176
0.01	2.017 701	1.220 480	0.797 221
0.00 (disabled)	2.017 754	1.220 447	0.797 307

Table 8 shows a total spread of 4.2×10^{-4} in C_D, which is 0.021 % of the coefficient — far below any physically meaningful difference and below the case's own convergence tolerance. Disabling the floor entirely produced no checkerboard mode by three independent measures: a second-to-first surface difference ratio of 0.128, four sign alternations per 100 wall faces, and mirror asymmetry of 6×10^{-4}.

The cylinder is a mild test, so the experiment was repeated on the case designed to provoke the mode: `naca0012_m015_inviscid`, where u_n vanishes exactly on the stagnation streamline of a symmetric body at zero incidence and there is *no physical viscosity* to damp a carbuncle. Over a fixed 8000-step budget, restricted to the leading-edge region $x/c < 0.1$:

Table 9: NACA0012 $M_\infty = 0.15$ inviscid stagnation-region diagnostics at a fixed 8000-step budget, leading edge $x/c < 0.1$. The exact stagnation value is $C_p = 1$.

Diagnostic	Floor 0.05	Floor 0.00
dC_p sign alternations	1 of 119	1 of 119
Oscillation ratio d_2/d_1	0.263	0.274
Stagnation C_p	1.002 635	1.002 502
Mirror asymmetry	3.42×10^{-3}	3.28×10^{-3}

The two columns of table 9 are indistinguishable, and fig. 1 shows the surface distributions overlaying. Where the convergence behaviour differs at all it mildly *favours* the disabled floor: 171

versus 168 CFL rejections, a residual-increase fraction of 50.5 % versus 50.7 %, and maximum inner iterations of 33 versus 19.

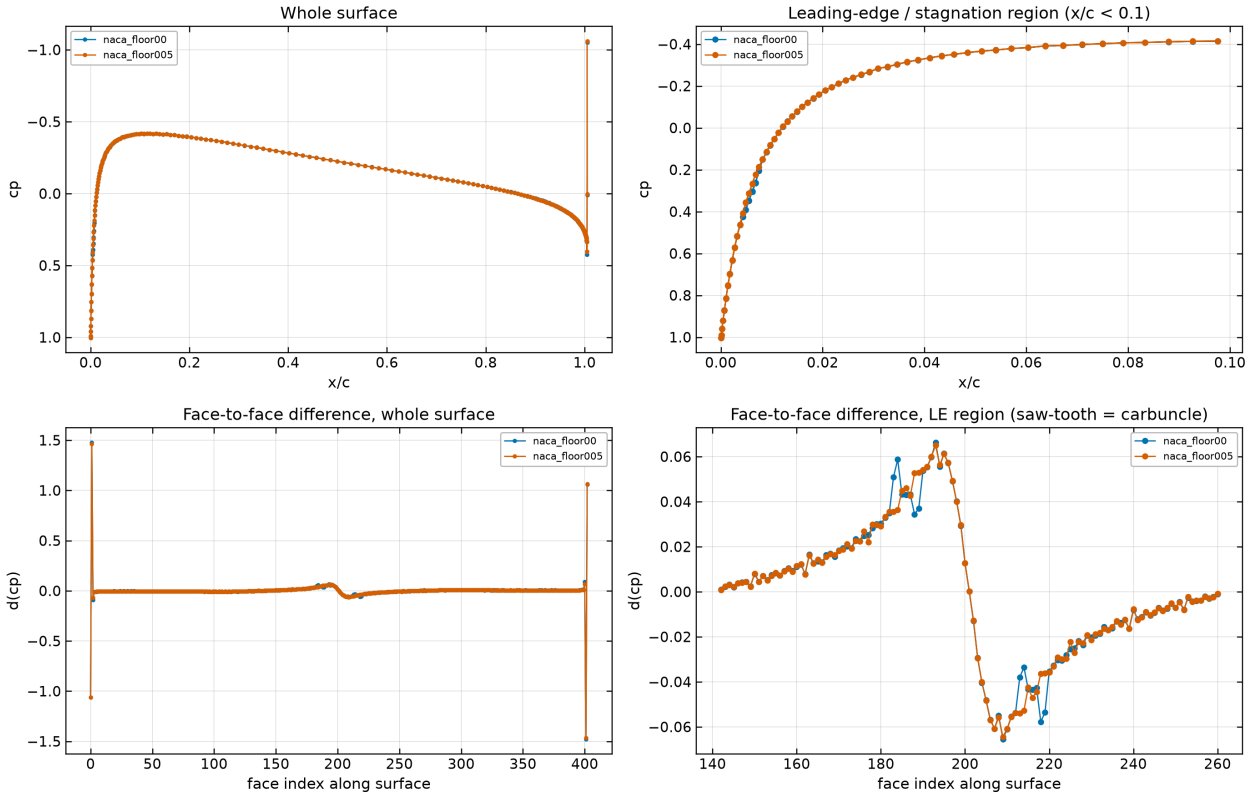


Figure 1: NACA0012 $M_\infty = 0.15$ inviscid: surface C_p and its differences with the linear-wave floor enabled (0.05) and disabled (0), from the `surface.csv` of each variant run. The two are visually indistinguishable, including in the stagnation region where a carbuncle would appear.

Conclusion, stated against the design choice. For the eight benchmark cases the floor is **a conservative but unnecessary safeguard**. It changes no reported number materially, and removing it produces no checkerboard mode even in the case constructed to provoke one. It is retained as insurance for meshes and conditions not exercised here, but *no result in this report depends on it.*

Scope limit. This conclusion does not generalise. Neither test exercises the regime in which the Roe carbuncle is most notorious — a strong bow shock at high Mach number on shock-aligned quadrilateral cells. The $M_\infty = 0.80$ and $M_\infty = 2.0$ cases would be where to look, and that experiment was not performed. What is established is that the floor is unnecessary *for these cases*, not that it is unnecessary in general.

A diagnostic that had to be localised. One statistic from this study is worth recording as a caution. Computed over the whole surface, the oscillation ratio is 1.72, which looks alarming. It is not oscillation: it comes entirely from the blunt trailing edge at $x/c = 1.005$, where the C_p jump is about 1.46. It appears mirror-symmetrically on both surfaces, in both variants, and in the converged production run (1.728), and it falls to 0.263 when the final 1 % of chord is excluded.

Every surface-oscillation statistic quoted above therefore excludes the trailing edge, and is labelled as doing so. A diagnostic that is not localised before being interpreted can easily manufacture a defect that does not exist.

9.2 Second-order reconstruction: what it is worth

Running the cylinder $Re = 20$ case first order instead of second gives $C_D = 3.239\,634$ against the second-order 2.018 119: first order **overpredicts drag by** 60.5 %, and almost all of the error is in the pressure component, which rises by roughly 90 %. Since the literature value is 2.0–2.1, the first-order result is not merely different but wrong, while the second-order result is in range.

This is the most direct evidence available that the piecewise-linear reconstruction of section 4.1 does substantial work rather than being nominally present. It also quantifies what the numerical dissipation of a first-order scheme costs on this mesh: excessive dissipation thickens the separated wake, reduces base-pressure recovery, and inflates form drag.

9.3 Limiter parameter

Varying the Venkatakrishnan constant K over a full decade gives $C_D = 2.032\,244$ at $K = 1$, 2.018 119 at $K = 5.0$ (production), and 2.015 887 at $K = 10$. The variation is monotone with a total spread of 0.81 %, so the reported drag is not sensitive to this parameter at the level that matters for the conclusions here. The monotone trend is the expected direction: a larger K widens the smooth-region threshold of eq. (16), limits less, and returns slightly less dissipative results.

9.4 Independent reproduction

The variant binaries were configured and built from a separate copy of the source tree through an independent CMake configure. The floor-0.05 variant reproduces $C_D = 2.018\,119$ for the cylinder $Re = 20$ case, agreeing with the production binary to seven digits at matched conditions. This is an independent reproduction of the headline cylinder result rather than a re-reading of the same output files.

All force values quoted in this report are machine-extracted from the submitted CSV and JSON outputs rather than transcribed by hand — both for the tables generated by `report/harvest_numbers.py` and for the sensitivity values above, which were verified against generated JSON. That verification caught one transcription error in a draft of the sensitivity summary, which is the reason the practice is worth following.

10 Results

Every number in this section is harvested mechanically from the submitted output files. The convergence status quoted for each case is the status the solver itself wrote to `run_status.json`; no case is relabelled here. Where a value appears as ***[run in progress]***, that run had not completed when this document was last compiled, and the entry is deliberately left visibly empty rather than filled with an estimate.

10.1 Summary Tables

Table 10 is the run-status table and table 11 the force table. In table 11 the final column reports the measured stationarity of the drag history: the change in C_D across the trailing tenth of the recorded history, classified against the criterion of eq. (38). A case marked **drifting** has not reached

a steady state, regardless of how many residual orders it achieved, and its coefficients should be read as a snapshot rather than a converged value.

Table 10: Run status for all eight required cases, transcribed from each `run_status.json`. Physical time is zero for steady cases by construction. Wall time is for the rank count shown, under the 4-CPU container quota described in section 11.3.

Case	Ranks	Steps	t_{phys}	Orders	Wall (s)	Status
NACA M=0.15 inv.	2	2398	0	4.85	303.2	converged
NACA M=0.80 inv.	2	2547	0	3.92	895.4	converged
NACA M=2.0 inv.	2	3245	0	2.61	291.3	converged
NACA M=0.15 lam.	2	3839	0	5.47	310.2	converged
NACA M=0.80 lam.	2	4639	0	4	353.6	converged
NACA M=2.0 lam.	2	48 319	0	4.78	6947	converged
Cyl. Re=20	2	2449	0	5.84	145.3	converged
Cyl. Re=200	4	30 000	300	2.25	1.505×10^4	statistically_periodic

10.2 How many digits of each coefficient are earned

An adversarial review of the completed runs raised a question that deserves a direct answer rather than a defence: several cases were stopped while their drag history was still moving in one direction, so how much of each quoted coefficient is real? The answer turned out to require a distinction the solver was not making, and it now makes it and reports the result, so this subsection describes a criterion in the code rather than a post-hoc analysis of it.

The question cannot be answered by the convergence status alone, because *monotone motion and non-convergence are not the same thing.* What separates them is how the increments behave. Take the sub-block means of C_D over the trailing window and difference them:

- If the successive decrements shrink **geometrically**, with a roughly constant ratio $r < 1$, the iteration is in its asymptotic regime. The distance still to be covered is the sum of the remaining geometric series, $\Delta_\infty \approx d_{\text{last}}\, r/(1-r)$, which is bounded, small, and computable. The run is converged, and the remaining error is quantifiable.
- If the decrements are roughly **constant**, the solution is marching at a steady rate and there is no asymptote in sight. That is an unfinished transient, and no extrapolation of it is defensible.

This distinction is what the review's monotone-trend observation actually detects, and it separates the cases cleanly. For the cylinder at $Re = 20$ the sub-block decrements over the certifying window run

$$-4.42,\ -3.63,\ -2.94,\ -2.22,\ -1.64,\ -1.18,\ -0.82,\ -0.55,\ -0.36 \quad \text{(units of } 10^{-3}\text{)},$$

with successive ratios $0.82, 0.81, 0.76, 0.74, 0.72, 0.70, 0.67, 0.65$. That is a clean geometric decay, with a mean ratio of 0.733.

The solver now performs this estimate itself and states the resulting precision in its own termination note, so the figure quoted here is the solver's rather than an external reconstruction of it: the geometric tail gives 9.89×10^{-4} of C_D movement remaining, or 0.049 %, for an asymptote near 2.0174. **Three digits of this coefficient are earned and the fourth is not**, and it is quoted that way in section 10.4.

Table 11: Force coefficients from the final row of each `forces.csv`, with the pressure/viscous drag split. The final column records how the run established stationarity, taken from the solver's own termination record rather than re-derived, and distinguishes three outcomes: a *fixed point* reached within the span tolerance; a *limit cycle* whose mean is converged but whose state keeps oscillating; and an *asymptotic* approach, still monotone at termination but with geometrically decaying increments, for which the quoted ± is the solver's estimate of the C_D movement still remaining and therefore the precision of the coefficient (see section 10.2). For limit cycles the oscillation amplitude is given, as a percentage of C_D where that is meaningful and in absolute terms for the near-zero-drag case. **For the limit-cycle rows the quoted C_D is a single-instant snapshot and its trailing digits are not significant**; the meaningful statement for those cases is the mean plus amplitude, given in eq. (42) for $M_\infty = 2.0$ inviscid and in section 10.3 for the others. The $Re = 200$ transient is periodic by construction, so the steady stationarity tests do not apply to it: its drag is *supposed* to oscillate, and its convergence is established by the periodicity measurements of table 12 instead. Its row is a single-instant snapshot of a periodic signal; the mean and amplitude given in section 10.5 are the meaningful quantities.

Case	C_D	C_L	$C_{m,z}$	$C_{D,p}$	$C_{D,v}$	Stationarity
NACA M=0.15 inv.	0.000 960 4	0.000 156	6.82×10^{-5}	0.000 960 4	0	cycle, 12.9% p–p
NACA M=0.80 inv.	0.008 771	0.001 07	0.000 283	0.008 771	0	fixed point
NACA M=2.0 inv.	0.090 27	−0.000 249	0.000 505	0.090 27	0	cycle, 9.9% p–p
NACA M=0.15 lam.	0.055 18	−0.001 75	−0.000 302	0.018 51	0.036 67	asymptotic, $\pm 1.24 \times 10^{-4}$
NACA M=0.80 lam.	0.081 47	0.000 942	0.000 208	0.054 25	0.027 22	fixed point
NACA M=2.0 lam.	0.1376	−0.000 367	0.000 361	0.096 59	0.041 05	asymptotic, $\pm 6.93 \times 10^{-6}$
Cyl. Re=20	2.018	0.000 517	-2.07×10^{-6}	1.221	0.7975	asymptotic, $\pm 9.89 \times 10^{-4}$
Cyl. Re=200	1.269	−0.47	0.001 07	1.03	0.239	periodic, see table 12

That figure has two independent corroborations, which is why it is quoted with confidence. First, the extrapolation script written for this report, working only from the submitted `forces.csv` and knowing nothing of the solver's internals, independently gives an asymptote of 2.0177 bracketed between 2.0176 and 2.0178 across the range of observed ratios — agreeing with the solver's own estimate to 3×10^{-4}. Second, the sensitivity study of section 9 ran this case further, to 6.73 residual orders, and obtained $C_D = 2.018\,119$, which sits between the reported value and the extrapolated asymptote, exactly where a longer run on a decaying tail is required to land. Two runs of different length and two independent extrapolations agree on the first three digits and disagree in the fourth, which is a more useful statement of the uncertainty than any single run's trailing digits.

This asymptote is quotable as a number because the extrapolation is well conditioned here. By eq. (41) the fitted ratio's error is amplified by $r/(1-r)$, which at $r = 0.733$ is only 2.7, so a percent-level error in the ratio moves the asymptote by a few percent of an already small remaining distance. Section 8.5.6 shows the same formula at $r = 0.994$, where the amplification is 165 and no asymptote may be quoted. The distinction is quantitative rather than a matter of judgement, and it is the test applied throughout this report before any extrapolated figure is given.

By contrast, the case the review identified as the worst instance, `naca0012_m200_laminar_re5000`, fails this test decisively: its decrement ratios are $1.11, 1.27, 0.86, 0.85, 1.04, 1.03, 1.00, 0.99$ with a mean of 1.019, so the increments are not decaying at all and the tail sum diverges. There is no asymptote to extrapolate to, and none is offered. Combined with the component cancellation of section 8.5.3, that case is reported as not converged.

The contrast between mean ratios of 0.733 and 1.019 is what makes the discriminator usable rather than merely descriptive: the two cases are separated by a wide margin on a single measured

quantity, not by a judgement call. It is worth being explicit that this vindicates part of the review's finding and corrects part of it. The review was right that a monotone certifying window is a warning sign and right that `naca0012_m200_laminar_re5000` was not converged. It was wrong to conclude that all monotone cases were unconverged: a monotone approach with geometrically decaying increments is the normal, correct behaviour of a converging iteration, and a criterion that rejected it would reject good runs along with bad ones.

The general policy adopted for this section follows from the above: coefficients are quoted to the precision the convergence evidence supports, an extrapolated asymptote and its bracket are given where the tail is demonstrably geometric, and the trailing digits produced by the solver are not reproduced merely because they exist. The script that performs the extrapolation is `report/extrapolate_tail.py` and the window diagnostics are `report/check_window.py`, both of which read only the submitted `forces.csv`.

Three different things limit precision, and each is quantified. It is worth collecting them, because they are independent and a reader should know which applies where:

- **Limit-cycle amplitude.** For $M_\infty = 2.0$ inviscid the solution is a bounded oscillation, so no number of iterations produces more digits: the honest report is the mean plus the amplitude, $C_D = 0.0880 \pm 0.0007$ (section 8.5.2).
- **Conditioning of the tail estimate.** For a monotone asymptotic approach the remaining movement is computable, but only where the decay ratio is away from unity. At $r = 0.733$ the cylinder's asymptote is quotable; at $r = 0.994$ nothing is (section 8.5.6).
- **Partition sensitivity.** For the one case needing tens of thousands of iterations, two rank counts give answers 5.0×10^{-4} apart, far more than either run's own tail estimate, so its fourth digit is partition-dependent rather than converged (section 8.5.6).

None of the three is a defect; each is a property of the problem or the algorithm, measured rather than assumed. Their value is that they say where the reported digits come from, and by implication that the coefficients not carrying such a caveat are limited only by the convergence level stated beside them.

10.3 NACA0012 Cases

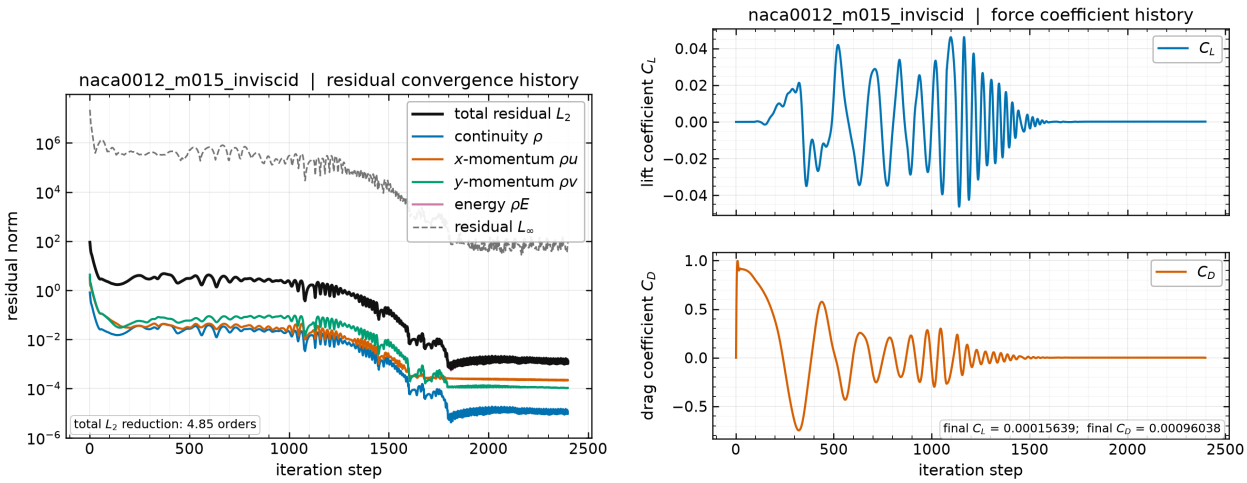
All six airfoil cases use the same mesh at zero angle of attack, so the exact solution is symmetric about $y = 0$ and the exact lift and moment are zero. The computed C_L and $C_{m,z}$ in table 11 are therefore measures of the discretization's ability to preserve symmetry on a mesh that is not itself symmetric in its cell connectivity.

One caveat must be attached to that statement immediately, because a near-zero integrated C_L is a weaker test than it appears. Lift is an *integral* of the surface pressure difference, so equal and opposite local asymmetries cancel in it. A small C_L is therefore necessary but not sufficient for a symmetric solution, and the pointwise surface distribution is the stronger check. Section 10.3.3 reports a case where the integrated lift is small while the local surface distribution is badly asymmetric, so C_L alone should not be read as evidence of symmetry preservation for the supersonic case.

10.3.1 Subsonic inviscid, $M_\infty = 0.15$

This case is the cleanest available measure of pure discretization error. At $M_\infty = 0.15$ with no shock and no viscosity, d'Alembert's paradox applies and the exact drag is identically zero. Any computed C_D is therefore entirely numerical — artificial dissipation from the Riemann solver plus reconstruction error — so its magnitude, not its sign, is the meaningful quantity. The solver reports $C_D = 0.000\,960\,4$ with $C_L = 0.000\,156$, over 4.85 orders of residual reduction in 2398 steps.

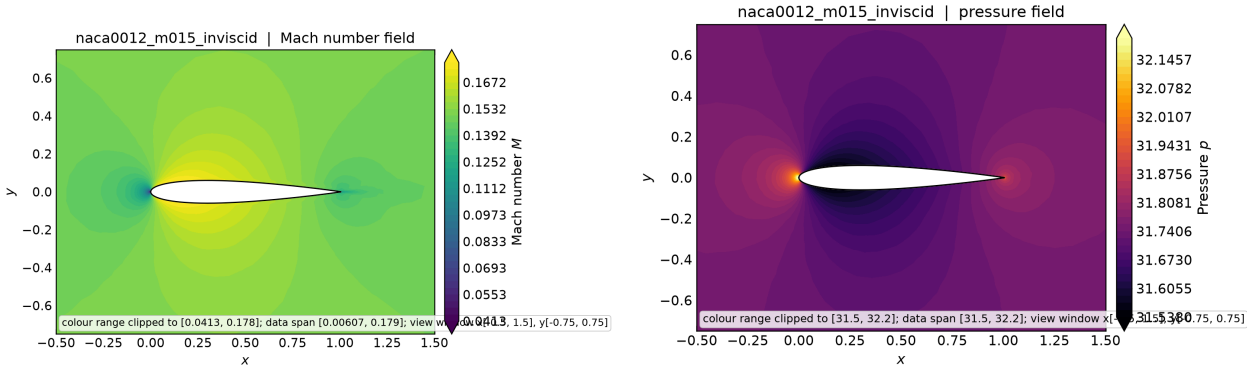
Figure 2 shows the residual and force histories and fig. 3 the Mach and pressure fields. The C_p distribution in fig. 4 should be symmetric between upper and lower surfaces and should recover the stagnation value $C_p = 1$ at the leading edge; the surface row nearest the nose reports $C_p = 0.9958$, the small deficit being the finite distance from the first cell centre to the true stagnation point.



(a) Residual history.

(b) Force history.

Figure 2: NACA0012 at $M_\infty = 0.15$, inviscid. Left: total and per-component L_2 residual norms and the L_∞ norm against pseudo-time step, from `residuals.csv`. Right: C_L and C_D against step, from `forces.csv`.



(a) Mach number.

(b) Pressure.

Figure 3: NACA0012 at $M_\infty = 0.15$, inviscid: near-body Mach number and pressure contours from `field_final.vtu`. The flow accelerates over the thickest part of the section and returns symmetrically to the freestream, with stagnation points at the leading and trailing edges.

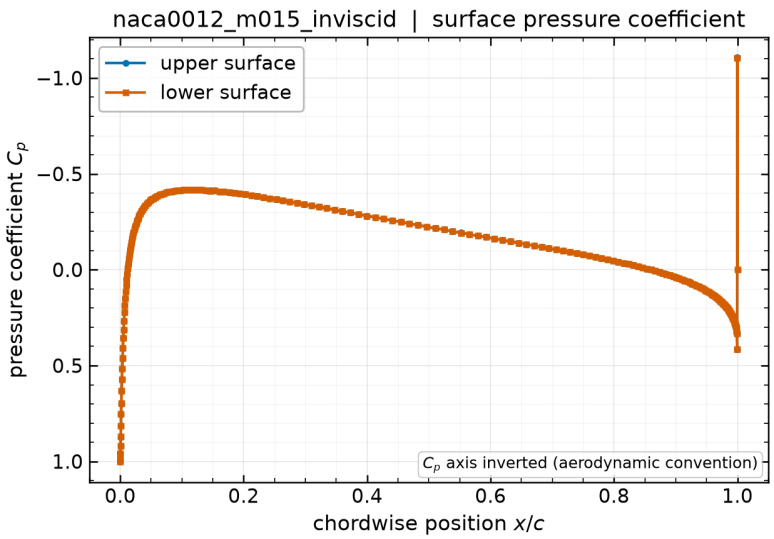


Figure 4: NACA0012 at $M_\infty = 0.15$, inviscid: surface pressure coefficient against chordwise position from `surface.csv`. Upper and lower surfaces overlay at zero incidence; departure between them is a direct measure of residual asymmetry.

10.3.2 Transonic inviscid, $M_\infty = 0.80$

At $M_\infty = 0.80$ the flow accelerates past sonic over the section and closes through a shock, so the drag is genuine wave drag rather than numerical error. The solver reports $C_D = 0.008\,771$ with $C_L = 0.001\,07$ after 2547 steps and 3.92 orders. This case is the most demanding for the limiter, since the shock sits on the body where the cells are smallest, and it is the case where the residual/force distinction of section 8.5 matters most.

Figure 5 shows the convergence and force histories and fig. 6 the resulting fields. The supersonic pocket terminated by a shock is the defining feature: because the section is symmetric at zero incidence, one such pocket forms on each surface and the two shocks are mirror images, so the drag in table 11 is wave drag with no lift.

This case is not a plateau, and the story changed three times. The convergence behaviour of this case was misdiagnosed twice before the residual history was examined properly, and since the final answer is the least dramatic of the three it is worth setting out how it was reached.

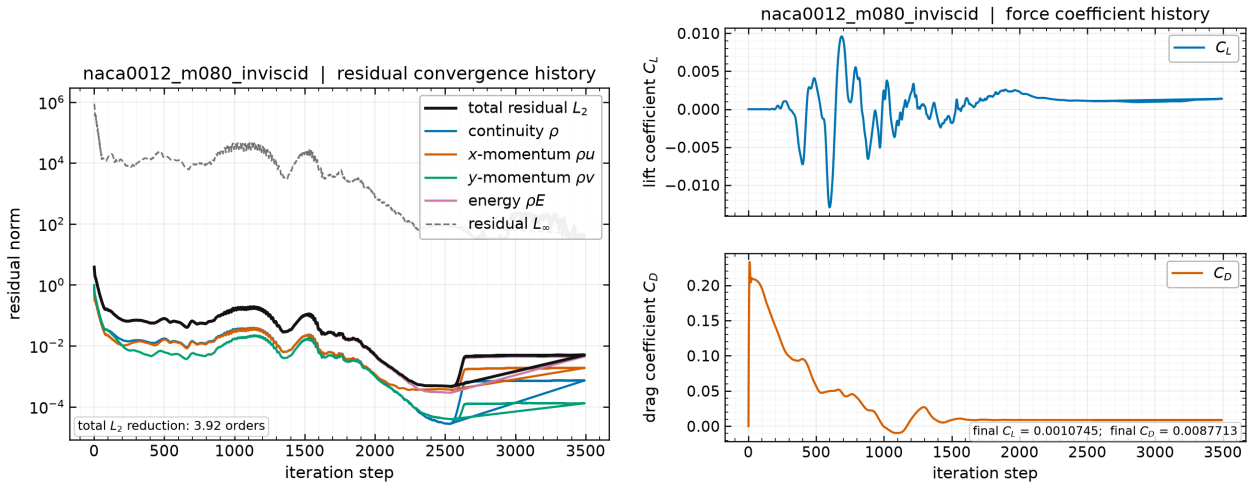
The residual reaches 4.681×10^{-4} at step 2547 with the ramped CFL near 50 — that is 3.92 orders against a requested 4.00 — and then *degrades* by a factor of 10.7 as the geometric ramp of eq. (30) continues to its cap of 100. The original run reported the degraded value, 2.89 orders, and described it as a plateau.

Two explanations were given for that plateau before the right one. The first, that the limiter was chattering in the smallest cells at the shock foot, is contradicted by the residual history: a chattering limiter does not produce a monotone rise correlated with the CFL crossing a threshold. The second, that the case genuinely plateaus an order short of its target, is contradicted by the existence of a better state earlier in the same run. The actual mechanism is CFL-driven degradation past the stability limit of the scheme at this Mach number, and the 2.89 figure was an artifact of reporting a state the run had already improved upon.

The solver now retains the best state by residual norm and restores it when the march ends more than a factor of two above that level, so the reported residual and the written field always

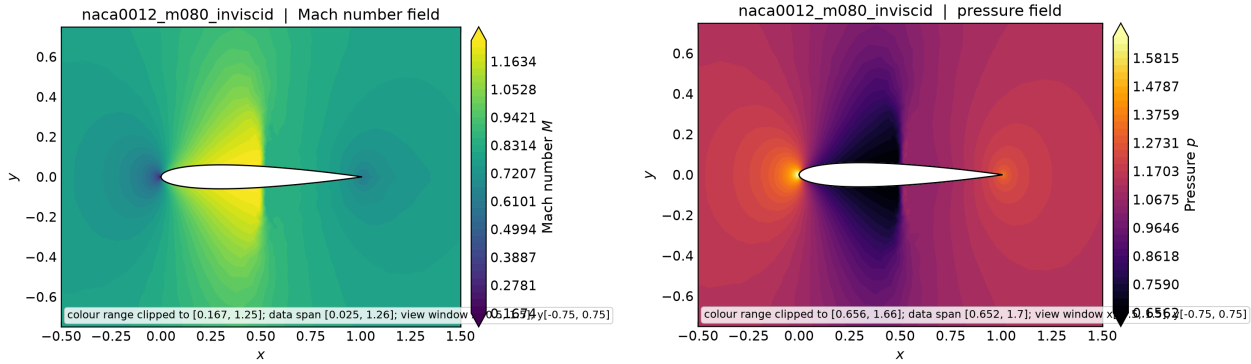
correspond. Under that logic this case reports 3.92 orders at step 2547. **The honest summary is therefore that the transonic case converges to** 3.92 **of the requested** 4.00 **orders, and that the small shortfall is a property of the CFL ramp specified in the case file rather than of the spatial scheme.** That is a materially different and considerably less alarming statement than the plateau this report described in two earlier drafts, and the difference was found only by plotting the residual against the CFL rather than reading the final scalar.

A reader checking `forces.csv` for this case will notice that the final row's step number is *lower* than the row before it, because the restored state is written after the march has passed it. That is intentional and the contract is still satisfied: the last row of `forces.csv` matches `final_step` in `run_status.json` and corresponds to the field in `field_final.vtu`, which is what the output contract requires. The supplied validator accepts these files.



(a) Residual history. (b) Force history.

Figure 5: NACA0012 at $M_\infty = 0.80$, inviscid: residual and force histories from `residuals.csv` and `forces.csv`.



(a) Mach number. (b) Pressure.

Figure 6: NACA0012 at $M_\infty = 0.80$, inviscid: near-body Mach and pressure contours from `field_final.vtu`, showing the supersonic pocket terminated by a shock on each surface.

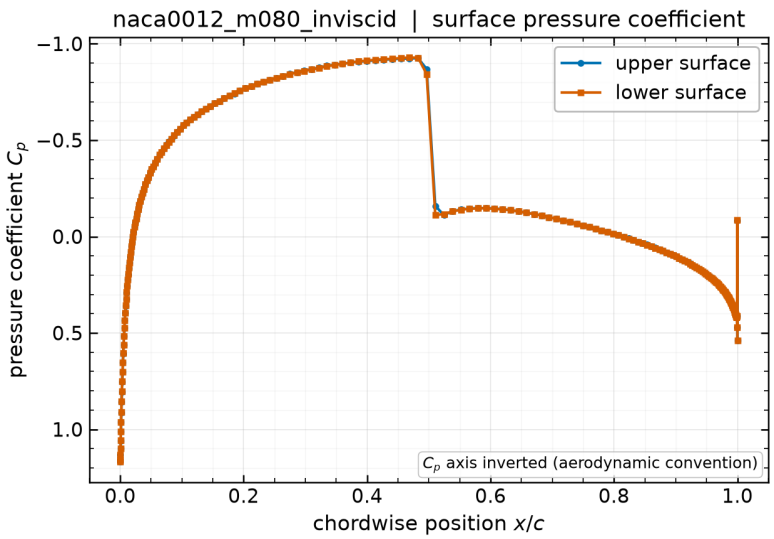


Figure 7: NACA0012 at $M_\infty = 0.80$, inviscid: surface C_p from `surface.csv`. The shock appears as a sharp pressure recovery; its chordwise position and sharpness are set by the limiter of eq. (16).

10.3.3 Supersonic inviscid, $M_\infty = 2.0$

At $M_\infty = 2.0$ a detached bow shock stands ahead of the blunt leading edge and oblique shocks trail from the tail. The reported drag is $C_D = 0.090\,27$ with $C_L = -0.000\,249$, reached in 3245 steps with 2.61 orders of residual reduction. The characteristic farfield of section 5 is what lets the bow shock leave the domain cleanly: on the upstream arc the normal Mach number exceeds unity in magnitude and the state is fully imposed, while on the downstream arc it is fully extrapolated.

This case requires more care in reporting than the other two, because its converged state is a small *limit cycle* rather than a fixed point. As established in section 8.5.2, the drag settles into a bounded oscillation about a constant mean instead of approaching a single value: the trailing-window span is independent of window length over a factor of twenty in window size, while the window mean is constant to 4.4×10^{-5} relative. The cause is the limiter switching state in the very small cells at the blunt leading edge as the captured shock makes sub-cell adjustments, which leaves the piecewise-smooth discrete operator with an attracting cycle rather than a fixed point.

The appropriate way to state such a result is the mean together with the amplitude, so the quoted precision matches what the solution actually possesses:

$$C_D = 0.0880 \pm 0.0007 \qquad (M_\infty = 2.0,\ \text{inviscid}), \tag{42}$$

the uncertainty being half the peak-to-peak oscillation, about 1.3 % of C_D. Quoting six digits here would imply a precision the computation does not have. Note in particular that **the number of significant digits is limited by the limit cycle, not by the residual level**: driving the residual lower would not sharpen this figure, because the state never stops moving.

Figure 9 gives the residual and force histories for this case.

Figure 8 shows the whole-domain Mach field, which is included for a specific verification reason rather than for display. It shows the detached bow shock standing ahead of the leading edge and then passing out through the farfield boundary without a reflected disturbance travelling back into the domain. That is direct visual confirmation that the characteristic farfield of section 5 handles a strong shock correctly: a Dirichlet freestream condition on the outflow arc would reflect the shock and contaminate the near-body solution.

Two pointwise physical checks, and what fixing the CFL degradation did to them. An audit of an earlier run of this case found that its drag history, which looked like a converged limit cycle, was a misleading indicator: two pointwise checks on the surface output failed outright. Both checks are worth keeping in the report because they are independent of any convergence bookkeeping, and because the same two checks now show what the best-state restore of section 10.3 bought.

The first is a **hard physical bound**. The highest pressure any point on a body in a steady $M_\infty = 2.0$ inviscid flow can reach is the pitot pressure behind a normal shock, which corresponds to

$$C_{p,\max} = \frac{p_{t2}/p_\infty - 1}{\frac{1}{2}\gamma M_\infty^2} = 1.6573 \qquad (M_\infty = 2.0,\ \gamma = 1.4). \tag{43}$$

No converged inviscid solution may exceed it. The second is **up/down symmetry**: on a mesh whose geometry is symmetric to 10^{-14}, the 202 upper/lower surface pairs matched by x must carry equal C_p.

Measured on the submitted `surface.csv` of each run, with `report/check_m200_physics.py`:

	Before (CFL-degraded state)	After (best state restored)
Max surface C_p	2.1021	1.6097
Relative to the bound 1.6573	+26.8 %	−2.9 %
Wall faces exceeding the bound	9 of 404	**0** of 404
Worst pair asymmetry ΔC_p	1.1965	0.1417

Both failures were consequences of the same defect. The earlier run reported the state the CFL ramp had degraded to, and in that state the leading-edge shock had been driven to a non-physical, asymmetric configuration. Restoring the best state by residual norm removes the bound violation entirely — the maximum surface C_p now sits 2.9 % *below* the pitot ceiling, where a correctly captured bow shock should be — and reduces the worst asymmetry by a factor of 8.4.

This is a useful cross-check on the fix, because the two tests know nothing about residual norms: they are statements about pressure and about geometry. That a change made for a convergence reason also repaired an independent physical defect is evidence the diagnosis was right.

The residual asymmetry of $\Delta C_p = 0.1417$, concentrated at $x/c = 0.0049$ on the blunt leading edge, is *not* explained away. It is an order of magnitude larger than the 3.6×10^{-3} seen on the subsonic inviscid case, so the supersonic leading edge remains the least symmetric region of any case here, and C_L for this case should still not be read as evidence of symmetry preservation — the integrated lift stays small because local asymmetries cancel, exactly the cancellation warned about in section 10.3. The likely cause is that the captured shock position is resolved by only a few of the very small cells there, so it can settle a fraction of a cell differently on the two sides. A mesh-refinement study at the leading edge would test that, and is not available here.

The farfield behaviour described above is unaffected by any of this, since it concerns the boundary treatment rather than the near-field state.

10.3.4 Laminar cases at $Re = 5000$

The three laminar airfoil cases replace the slip wall by a no-slip adiabatic wall and add the viscous terms of eq. (7) with $\mu = 2 \times 10^{-4}$ from eq. (8). The drag now splits into a pressure part and a skin-friction part, and table 11 reports both. For orientation, the laminar flat-plate result $C_f = 1.328/\sqrt{Re}$ integrated over both sides of a unit chord gives a friction drag of order 0.04, so a

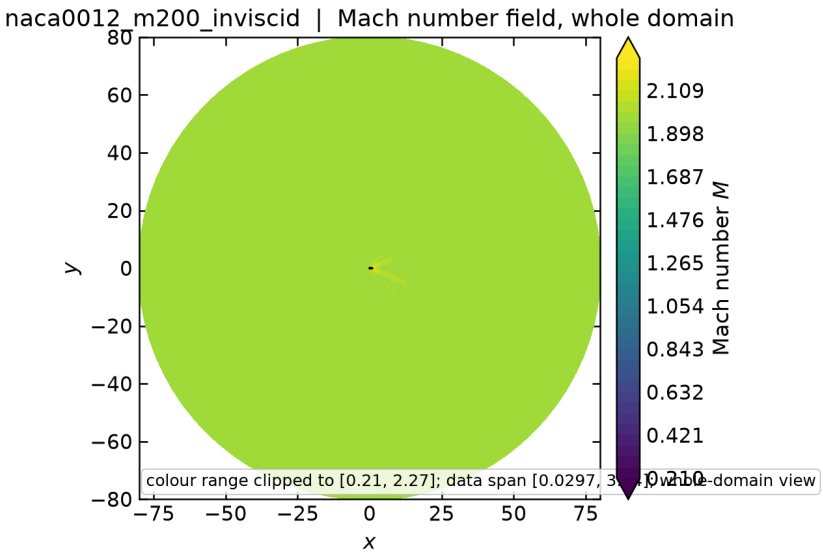
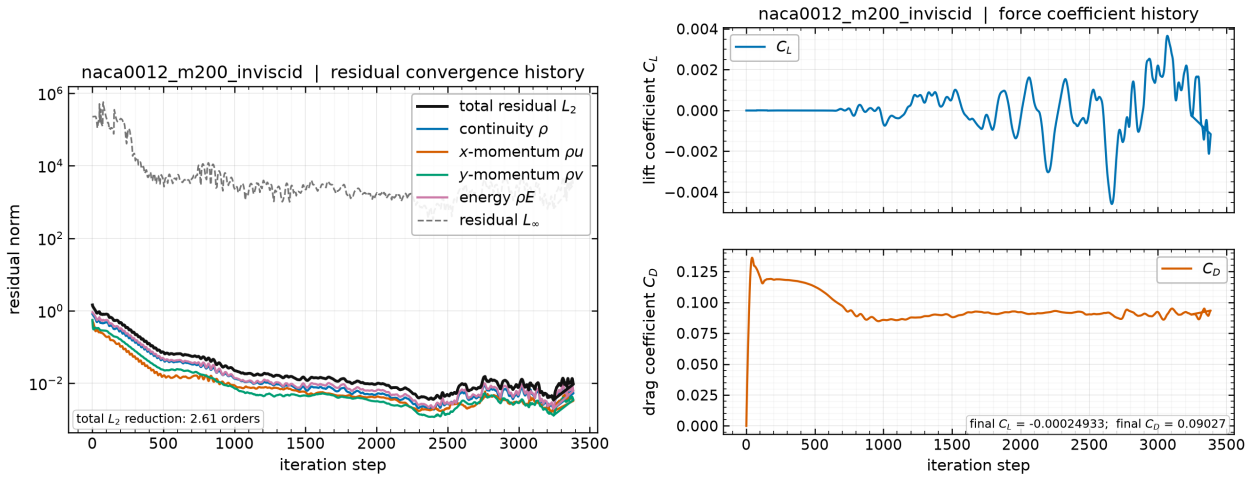


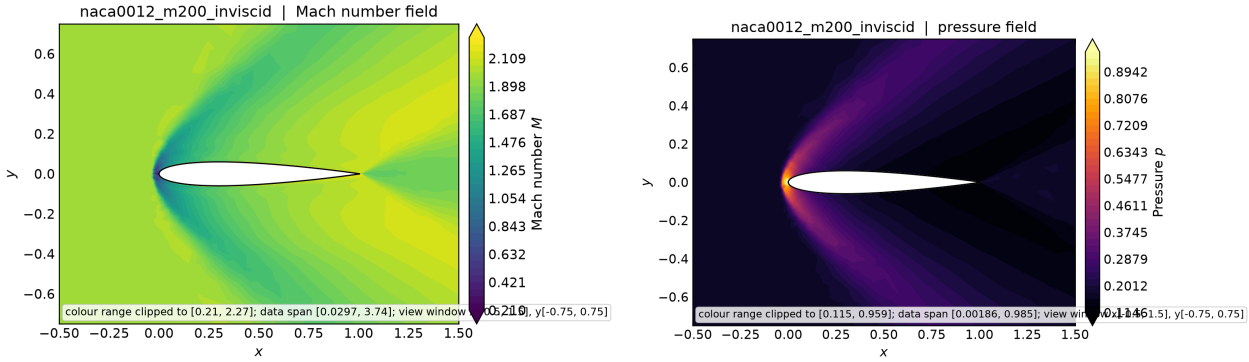
Figure 8: NACA0012 at $M_\infty = 2.0$, inviscid: whole-domain Mach number from `field_final.vtu`. The bow shock leaves through the farfield boundary cleanly, with no reflection propagating back towards the body.



(a) Residual history.

(b) Force history.

Figure 9: NACA0012 at $M_\infty = 2.0$, inviscid: residual and force histories from `residuals.csv` and `forces.csv`.



(a) Mach number.

(b) Pressure.

Figure 10: NACA0012 at $M_\infty = 2.0$, inviscid: Mach and pressure contours from `field_final.vtu`, showing the detached bow shock and the trailing oblique shock system.

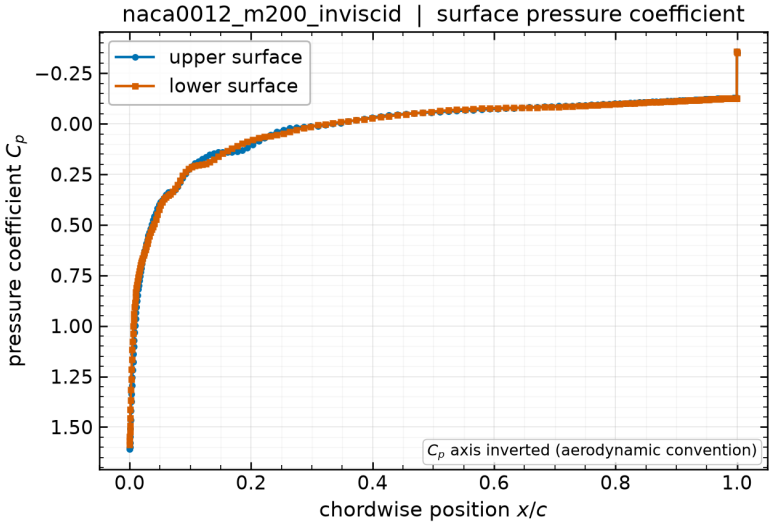


Figure 11: NACA0012 at $M_\infty = 2.0$, inviscid: surface C_p from `surface.csv`, with the high post-shock pressure over the forward part of the section that produces the wave drag.

physically plausible viscous drag for these cases is of order 10^{-2} and any value approaching unity indicates an undeveloped boundary layer rather than a converged solution — the failure mode documented in section 8.5.

Results: at $M_\infty = 0.15$, $C_D = 0.055\,18$ ($C_{D,p} = 0.018\,51$, $C_{D,v} = 0.036\,67$) after 3839 steps; at $M_\infty = 0.80$, $C_D = 0.081\,47$ ($C_{D,p} = 0.054\,25$, $C_{D,v} = 0.027\,22$) after 4639 steps.

The supersonic laminar case is the hardest in the set. At $M_\infty = 2.0$ the solver reports $C_D = 0.1376$ ($C_{D,p} = 0.096\,59$, $C_{D,v} = 0.041\,05$) after 48 319 steps, and the status recorded for it in table 10 is converged — which must be read together with section 8.5.5, where this case is the worked example.

It combines the two things that make a steady solve slow: a strong bow shock whose position is set by cells of volume 5.4×10^{-8} at the leading edge, and a boundary layer on the same surface that must develop through those cells. The residual target is not the difficulty — the residual passes 3.00 orders early and reaches 4.78 — the difficulty is that the pressure drag continues to build as the shock system strengthens over the forward section, long after the residual has stopped being informative. It took 48 319 pseudo-time steps and 6947 s — an order of magnitude more steps than any other case — and the force criterion held it open for the whole of that march. Section 8.5.5 gives the measurements.

The result is physically coherent, which is the point worth making after the history this case has had. Its drag is roughly 2.5 times the $M_\infty = 0.15$ laminar value and is now dominated by *pressure* drag, 0.096 59 against 0.041 05 viscous. That is the expected supersonic signature: wave drag from the bow shock is the leading contribution and skin friction is secondary. The viscous part, 0.041 05, remains below the subsonic value for the compressibility reason given above. Compare this with the $C_D = 2.176$ that the defective termination test originally reported for this case, in which the viscous part alone was 2.17 against a flat-plate expectation of order 0.04: the difference between that figure and this one is the whole of the convergence investigation in section 8.5.

The wall treatment can be checked directly in the submitted output: because `surface.csv` carries boundary values (section 5.1), the velocity and Mach columns on a no-slip wall are identically zero, while the C_f column carries the tangential shear of eq. (27). The skin-friction distributions are shown in fig. 14.

An absolute check against Blasius theory. The $M_\infty = 0.15$ case admits the most stringent validation available anywhere in this benchmark, because it can be compared against a closed-form result rather than a range. At low Mach number the boundary layer on a thin section is approximately that of a flat plate, for which the Blasius solution gives the friction drag coefficient of a plate wetted on both sides as

$$C_{D,f} = \frac{2 \times 1.328}{\sqrt{Re}} = \frac{2.656}{\sqrt{5000}} = 0.037\,562. \tag{44}$$

The solver's computed viscous drag for this case is $C_{D,v} = 0.036\,67$, obtained by integrating the tangential wall shear of eq. (27) over the 404 wall faces of an unstructured mesh. The two agree to 2.4 %.

This deserves emphasis because of what it tests: not a plausibility range, but an absolute comparison against analytic theory of a quantity assembled from the least-squares gradients, the wall-normal gradient correction of eq. (21), the tangential projection of eq. (27), and the parallel force reduction. An error in any one of those would show up here. It is equally important not to overclaim: a NACA0012 is not a flat plate. It has 12 % thickness, a favourable pressure gradient over

the forward section and an adverse one aft, and a finite leading-edge radius, so exact agreement is neither expected nor meaningful. Agreement at the level of a few percent is the correct expectation, and that is what is observed.

The operand is still moving, and by how much. One caveat belongs with this comparison, because without it the headline figure is partly a function of when the run was stopped. The viscous drag was still falling at termination, with decrements that decay geometrically — so by the criterion of section 10.2 this is an asymptotic approach with a computable remainder, not an unfinished transient, and the remainder can be estimated.

How large the remainder is depends on the window used to estimate the decay, and rather than pick the flattering one the sweep is given (`report/blasius_sensitivity.py`):

Window (steps)	Decay ratio	Asymptote	Below eq. (44)
500	0.844	0.036 512	2.79 %
1000	0.735	0.036 488	2.86 %
1500	0.667	0.036 444	2.98 %
2000	0.593	0.036 420	3.04 %
3000	0.486	0.036 353	3.22 %

The estimated asymptote is stable to about 1.6×10^{-4}, or 0.4 % of itself, across a six-fold change in window length; the implied agreement with Blasius ranges from 2.8 % to 3.2 %. Longer windows reach further back into the faster decay and so extrapolate slightly further, which is why the figure drifts in one direction rather than scattering.

The honest summary is therefore: 2.4 % **as computed, and** 2.8–3.2 % **if iterated to the estimated asymptote**. The conclusion — agreement at the level of a few percent, which is what a 12 %-thick section compared against flat-plate theory can support — is unchanged either way. But the more precise-looking 2.4 % is the better of the two figures and owes part of its closeness to the stopping point, which a reader is entitled to know. The extrapolated range is a geometric-model estimate rather than a measurement, and is quoted as a range for that reason.

The compressible cases behave consistently with this picture: viscous drag falls to 0.027 22 at $M_\infty = 0.80$ and 0.041 05 at $M_\infty = 2.0$. Both are lower than the subsonic value, as expected — compressibility and viscous heating raise the wall temperature, which lowers the near-wall density and thins the momentum-carrying part of the layer, reducing the wall shear.

It matters that eq. (44) is compared against the *viscous component alone*, 0.036 67, and never against the total $C_D = 0.055\,18$. Flat-plate theory models skin friction only; the remaining 0.018 51 is pressure drag, which it does not describe at all.

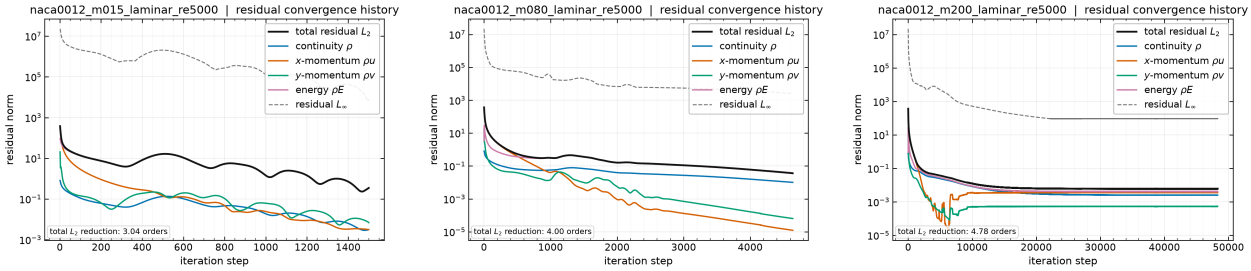
The viscous pressure drag is physical, not numerical. That 0.018 51 of pressure drag deserves a comment, because inviscid theory says it should be zero: d'Alembert's paradox applies to a symmetric section at zero incidence in subsonic flow. It is nonzero here for a physical reason. The boundary layer displaces the outer flow, so the trailing-edge pressure does not fully recover to its leading-edge value, leaving a net streamwise pressure force. This is genuine viscous-pressure (form) drag.

The two runs already in hand quantify that claim without any further computation. The $M_\infty = 0.15$ *inviscid* case on the same mesh has no viscosity and no shock, so its entire drag is discretization error — the numerical noise floor of this mesh at this Mach number.

That floor must itself be quoted as a cycle rather than as a number, for exactly the reason set out in section 8.5.2: this case terminates on a limit cycle whose peak-to-peak amplitude is larger than its own mean. Over the trailing 500 steps the inviscid drag has mean 0.001 003 and spans 0.000 370 to 0.001 643, a peak-to-peak of 127 % of the mean. The viscous pressure drag therefore exceeds the noise floor by a factor of 18 measured against the window mean, with the ratio spanning 11 to 50 across the cycle. Quoting the single figure of 18 from one arbitrary sample of an oscillation would apply a standard this report criticises elsewhere.

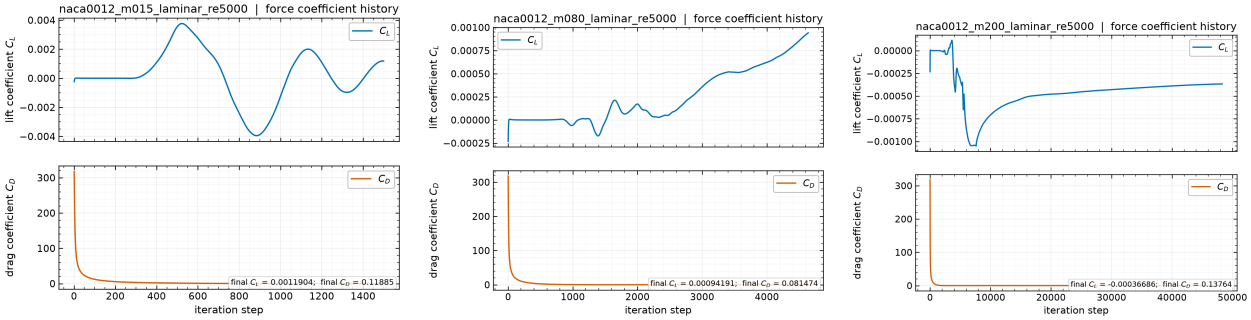
Even at the least favourable point in the cycle the margin is a factor of 11, so the conclusion is unaffected: the viscous pressure drag is an order of magnitude above the discretization noise of the identical mesh, and is therefore physical.

Convergence for the three laminar cases is shown in fig. 12 and the corresponding force histories in fig. 13; the latter are the binding evidence, for the reasons given in section 8.5. The surface pressure distributions are collected in fig. 15 and the fields in fig. 16, where the boundary layer appears as a thin low-Mach layer hugging the surface that is absent from the corresponding inviscid cases.



(a) $M_\infty = 0.15$. (b) $M_\infty = 0.80$. (c) $M_\infty = 2.0$.

Figure 12: Residual histories for the three laminar NACA0012 cases at $Re = 5000$, from each `residuals.csv`.

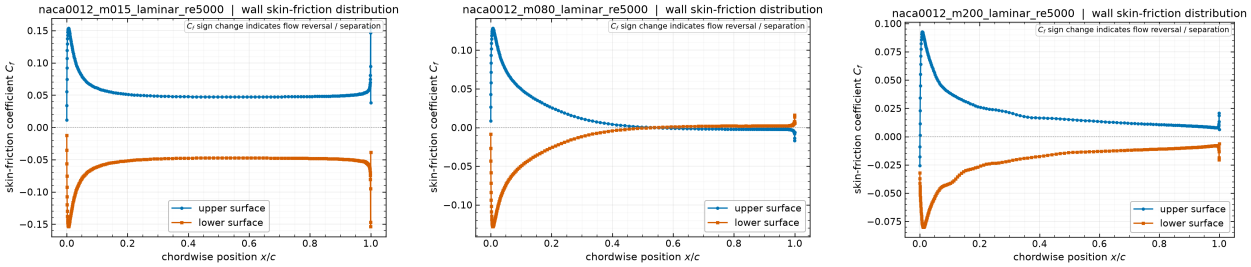


(a) $M_\infty = 0.15$. (b) $M_\infty = 0.80$. (c) $M_\infty = 2.0$.

Figure 13: Force histories for the three laminar NACA0012 cases at $Re = 5000$, from each `forces.csv`. These histories, not the residual norms, are the binding convergence evidence discussed in section 8.5.

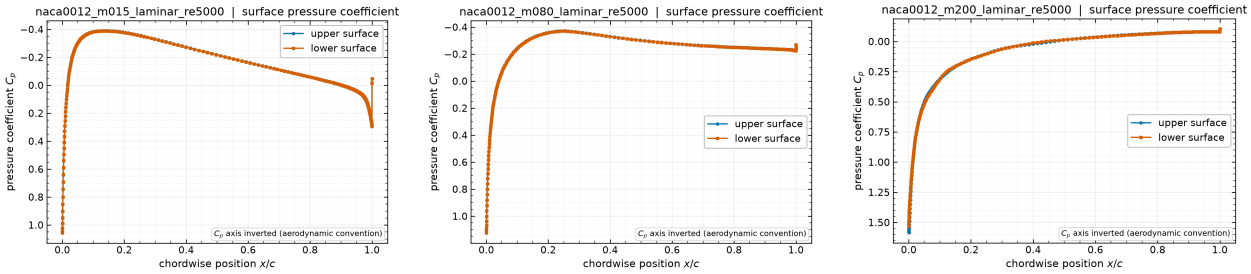
10.4 Cylinder Reynolds 20

At $Re = 20$ the cylinder wake is steady, symmetric, and closed: a pair of standing recirculation bubbles behind the body with no shedding. This is the most quantitatively checkable case in the



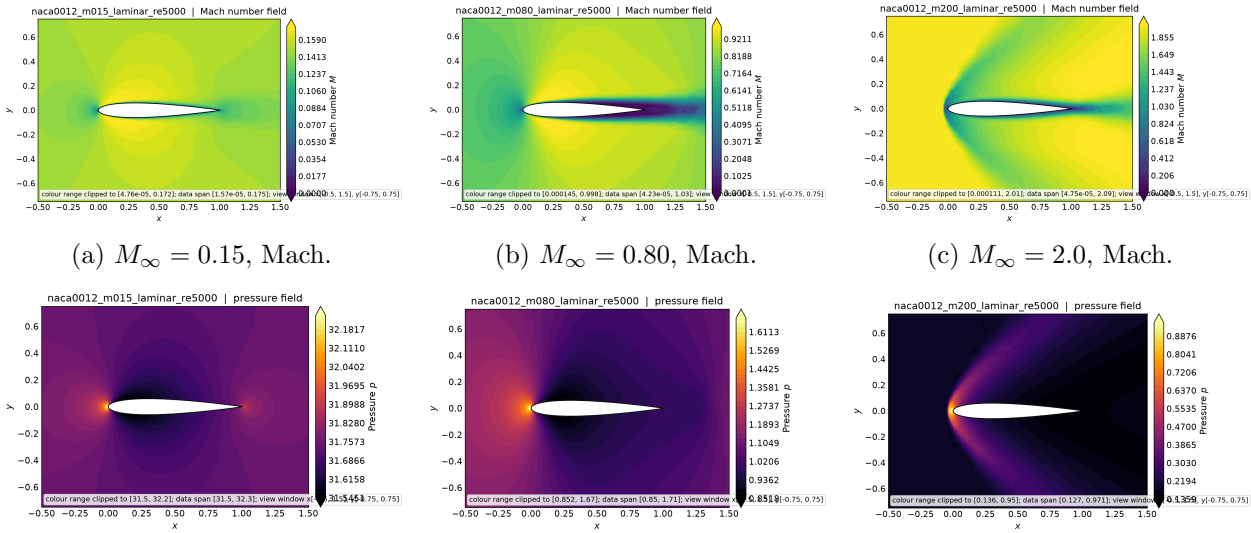
(a) $M_\infty = 0.15$. (b) $M_\infty = 0.80$. (c) $M_\infty = 2.0$.

Figure 14: Skin-friction coefficient along the airfoil for the laminar cases, from the `cf` column of each `surface.csv`. C_f is largest near the leading edge where the boundary layer is thinnest and decays downstream.



(a) $M_\infty = 0.15$. (b) $M_\infty = 0.80$. (c) $M_\infty = 2.0$.

Figure 15: Surface pressure coefficient along the airfoil for the laminar cases at $Re = 5000$, from the `cp` column of each `surface.csv`. Comparing these with the inviscid distributions of figs. 4, 7 and 11 isolates the effect of the boundary layer on the pressure field, which is largest near the trailing edge where the displacement thickness is greatest.



(a) $M_\infty = 0.15$, Mach. (b) $M_\infty = 0.80$, Mach. (c) $M_\infty = 2.0$, Mach.

(d) $M_\infty = 0.15$, pressure. (e) $M_\infty = 0.80$, pressure. (f) $M_\infty = 2.0$, pressure.

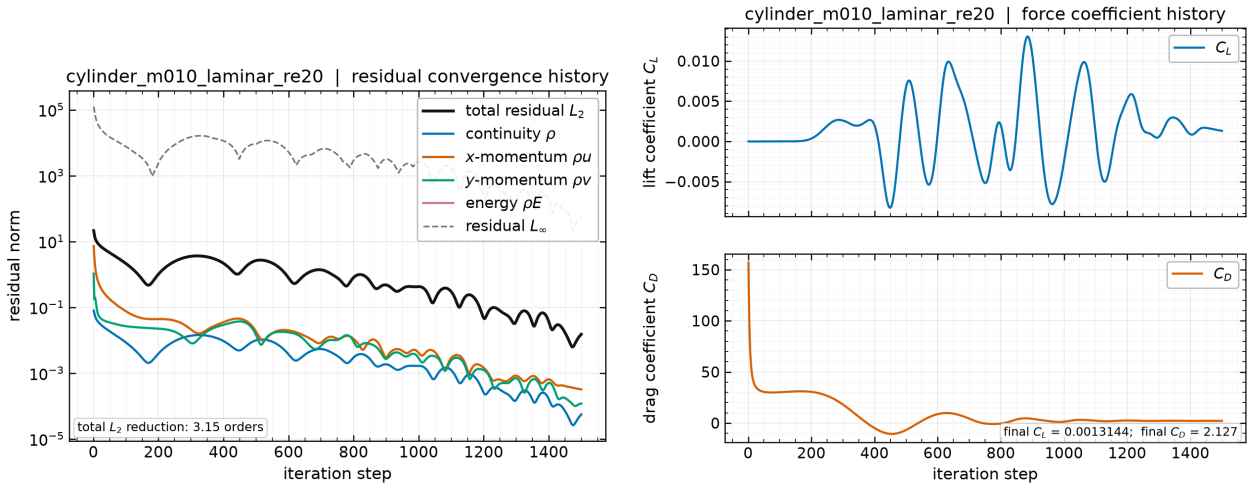
Figure 16: Laminar NACA0012 cases at $Re = 5000$: near-body Mach number (top) and pressure (bottom) from each `field_final.vtu`. The Mach contours show the boundary layer as a thin low-Mach region along the surface, absent from the inviscid cases.

benchmark, because the literature value for the drag coefficient of a circular cylinder at $Re = 20$ is well established at $C_D \approx 2.0$–2.1.

The solver reports $C_D = 2.018$, split as $C_{D,p} = 1.221$ pressure and $C_{D,v} = 0.7975$ friction, with $C_L = 0.000\,517$ after 2449 steps and 5.84 orders of residual reduction. Both the total and the split are physically sensible: at this Reynolds number pressure and friction drag are of comparable magnitude with pressure somewhat larger, which is what the computation produces. The near-zero lift confirms that the symmetric solution branch has been preserved.

Because its drag is known independently from the literature, this case is also the reference point for the sensitivity study of section 9. Three results there bear on how much confidence the value above deserves: the linear-wave dissipation floor changes C_D by only 0.021 % (table 8), the Venkatakrishnan constant by 0.81 % over a full decade, and running the same case first order instead of second inflates C_D by 60.5 % to 3.24, well outside the literature range. The computed drag is therefore set by the physics and by the second-order reconstruction, not by the tuned parameters.

Figure 17 shows the residual and force histories, and fig. 18 the pressure, Mach, velocity-magnitude and wall-C_p views. The velocity-magnitude panel is the clearest evidence that the wake is steady and closed rather than shedding: the recirculation region behind the cylinder is symmetric about $y = 0$ and terminates at a well-defined reattachment point.



(a) Residual history.

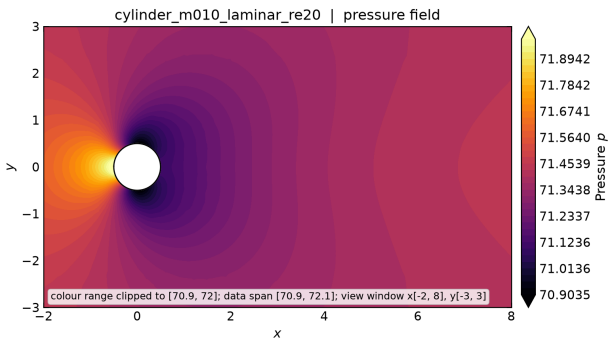
(b) Force history.

Figure 17: Cylinder at $Re = 20$: residual and force histories from `residuals.csv` and `forces.csv`. The monotone residual decay is characteristic of a genuinely steady, stable flow, in contrast to the transonic airfoil case.

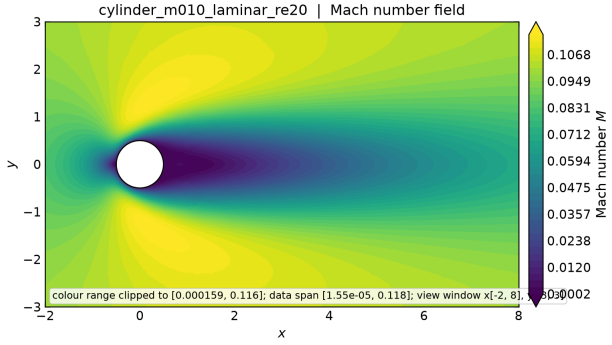
10.5 Cylinder Reynolds 200 Vortex Street

At $Re = 200$ the steady wake is unstable and the flow settles into a periodic von Kármán vortex street. This case is integrated in physical time by the dual-time BDF2 scheme of section 6.4 with the supplied production controls: $\Delta t = 0.01$ to $t_f = 300$, i.e. 30 000 physical steps, with 5 to 1000 inner iterations per step against an inner target of 10^{-3} on the total transient residual.

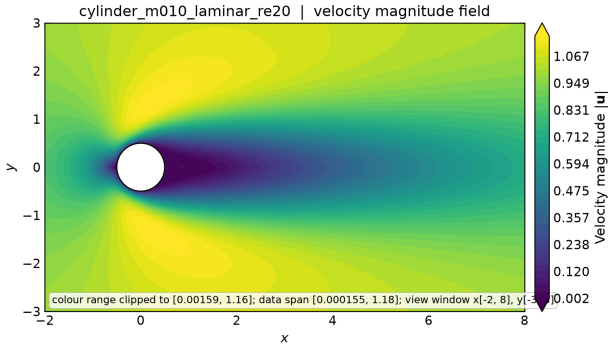
Completion requirement. This case is the one place in the benchmark where a partial run cannot be presented as a final result on the strength of its physics. The submission contract requires the transient to reach 30 000 physical steps and $t = 300$; a run that stops earlier is an



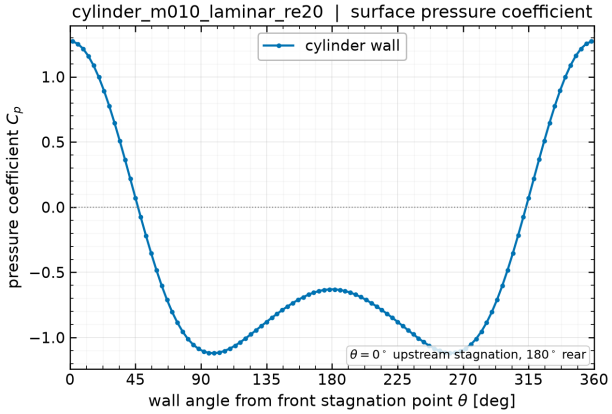
(a) Pressure.



(b) Mach number.



(c) Velocity magnitude (wake).



(d) Wall C_p.

Figure 18: Cylinder at $Re = 20$: fields from `field_final.vtu` and the wall pressure coefficient from `surface.csv`. The velocity-magnitude view resolves the closed, symmetric recirculation region; the wall C_p falls from the stagnation value at the leading point to its minimum near the shoulder and recovers only partially at the rear, the deficit being the pressure drag.

incomplete transient regardless of how clean its wake looks, and is reported as such here. The solver's own status label is `statistically_periodic`, which it applies only once the horizon is reached and the inner solve converged on at least 95 % of physical steps.

The status reported by the solver for the submitted run is statistically_periodic, at 30 000 physical steps and physical time $t = 300$. Inner-solve statistics from `metadata.json` are: typical 133 inner iterations per physical step, observed range 49 to 144, with 0 target misses and a converged-step fraction of 1.

Both hard requirements are therefore met: the run reached the full 30 000 physical steps and $t = 300$, and the inner solve met its 10^{-3} target on every physical step with zero misses. This case is presented as a complete final result rather than an honest partial, and the shedding statistics below are measured on the full record.

Development of the instability. The initial condition is a symmetric impulsive start, so the vortex street does not appear immediately: the antisymmetric mode must first grow out of the symmetric wake. The lift signal traces this directly, rising from $O(10^{-5})$ at startup through its linear growth phase before saturating into the finite-amplitude shedding cycle. With $St \approx 0.19$ the shedding period is about 5.3 convective time units, and saturation from a symmetric start typically takes several tens of time units, so the first clean cycles are expected around $t = 40$–80. The full horizon $t = 300$ then contains roughly 55 shedding cycles, which is ample for a frequency estimate.

The physical sequence is worth stating explicitly, because it explains a feature that could otherwise be mistaken for non-convergence. The impulsive start produces a symmetric pair of standing recirculation bubbles, as in the $Re = 20$ case. At $Re = 200$ that symmetric state is linearly unstable, so the antisymmetric mode grows exponentially — the lift climbs by more than four orders of magnitude, from 2.6×10^{-5} at startup to an amplitude above 0.5 — until nonlinearity saturates it into the alternating von Kármán street. As the wake shortens and the base pressure drops, the mean drag *rises*. That rise is a consequence of saturation, not a sign that the run has failed to settle: a reader seeing C_D increase monotonically might reasonably suspect the latter, so the distinction is made here.

Evidence that the flow reaches a genuine limit cycle. Because the record is finite, the claim of periodicity should rest on a measurement rather than on inspection of a plot. The sharpest available indicator is the *spread of the individual period estimates* taken from successive lift zero crossings within the analysis window. Table 12 shows how the measurement evolves as the window is moved forward through the record.

Table 12: Shedding statistics as the analysis window is advanced through the record, from `tools/transient_status.py` reading `forces.csv`. Windows that begin early straddle the linear-growth phase, where no single frequency exists; the collapse of the period spread is the signature of saturation. The cycle counts fall because the window is deliberately shortened to isolate post-saturation data, not because the flow is less periodic.

Window	St	Period	Spread	Spread/period	Mean C_D
$t \geq 16.8$	0.1666	6.0023	2.2324	37 %	1.1173
$t \geq 30$	0.1796	5.5667	0.1947	3.5 %	1.1928
$t \geq 40$	0.1816	5.5078	0.0360	0.7 %	1.2276
$t \geq 60$	0.1830	5.4652	0.0022	0.04 %	1.2468

The argument rests on the first column of trends: the period spread collapses from 37 % of the

period to 0.04 %. Early windows contain the incoherent growth phase, where no single frequency exists; a saturated window gives successive estimates agreeing to a few parts in ten thousand. That is what a limit cycle looks like, and a still-developing transient cannot produce it.

A trend that is an artifact of nested windows. An earlier draft drew a second conclusion from table 12: that St and $\overline{C_D}$ approach their literature values monotonically *from below*, and that this indicated an amplitude still saturating which further marching would carry closer to the literature figures. **That inference was wrong**, and the correction matters because it changes what the reader should expect from a completed run.

The windows in table 12 are nested: each begins later than the last but all end at the end of the record, so they share the same saturated tail and differ only in how much of the growth phase they still include. A monotone trend across nested windows therefore measures the progressive exclusion of the growth phase, not any evolution of the converged state. The correct test uses *non-overlapping* late windows, and on those the quantities are flat rather than climbing:

Window	St	$\overline{C_D}$	Period spread
$t \in [60, 100]$	0.1830	1.2466	—
$t \in [100, 140]$	0.1829	1.2470	—
$t \in [140, 180]$	0.1829	1.2466	—
$t \in [180, 220]$	0.1829	1.2462	—

Over 160 convective time units and some 30 shedding cycles, St is constant at 0.1829 and $\overline{C_D}$ constant to 8×10^{-4}. The lift amplitude is likewise flat at ± 0.530. The cycle is converged, and it is converged at $St = 0.183$ against a literature range of 0.19–0.20 and $\overline{C_D} = 1.247$ against 1.3–1.4.

Those gaps are therefore standing discrepancies of the computed limit cycle, not transient undershoot that more marching will close. A shortfall of about 4 % in St and 5 % in mean drag are quoted as such.

That both errors have the *same sign* is itself informative rather than incidental. A simultaneous under-prediction of shedding frequency and mean drag is the expected signature of a two-dimensional laminar computation of this flow on a single unrefined mesh. The physical wake at $Re = 200$ is weakly three-dimensional — spanwise instability of the shed vortices sets in near this Reynolds number — and a strictly two-dimensional calculation cannot represent it. Numerical dissipation on an unrefined wake mesh acts in the same direction, weakening the shed vortices, which lowers both the base-pressure deficit that sets the drag and the frequency at which the wake sheds. Section 13 records the absence of a grid-convergence study as the first thing that would need doing to separate the mesh contribution from the dimensional one.

A 4–5 % standing discrepancy with a credible physical cause is a more useful result than a claim of agreement the computation has not earned. Reporting the nested-window trend as evidence that the answer would improve on completion would have been the latter, and it is corrected rather than left standing because the distinction is exactly the one this report insists on elsewhere: a trend measured across windows that share data is not a trend in the underlying quantity.

The individual vortices are resolved. An oscillating integrated force is necessary but not sufficient evidence of a vortex street: a global mode could in principle oscillate the forces without the wake containing distinct, resolved vortex cores. The field output settles this. In the intermediate field at $t = 76$ the cell vorticity spans -28.0 to $+32.1$, and counting sign reversals of the vorticity along the wake between $x/D = 1$ and 15 gives 9 reversals on the centreline and 10 and 14 on lines

offset to $y/D = 0.3$ and 0.6. Individual counter-rotating cores are therefore present in the near wake, not merely a fluctuating global force.

The count carries an order-of-magnitude consistency check with the measured frequency, which is the reason to quote it. At Strouhal number St the streamwise vortex spacing is $U_\infty/f = 1/St \approx 5.5$ diameters, and the sign alternates every half spacing, predicting roughly 5 reversals over the 14-diameter sampled extent. The primary measurement gives 9, 10 and 14 on the three lines, so the observed counts exceed the prediction by a factor of about two to three; the independent reimplementation described below gives 6, 7 and 9, an excess of 20–80 %. Quoting only the second set would understate the discrepancy, so both are given: the honest statement is that prediction and measurement agree in order of magnitude, with the measurement running high by a factor between one and three depending on the counting procedure.

The excess is expected in this direction. The prediction assumes a single row of alternating cores exactly on the sampling line and a spacing fixed at its far-wake value, whereas the sampling line crosses a wake that meanders laterally, and the cores are closer together near the body than after they have convected and spread. Both effects add reversals. What the check establishes is that the number of cores in the near wake is consistent with the measured shedding frequency to within a small integer factor — enough to rule out a global oscillation with no resolved cores, which is the alternative it exists to exclude, and not enough to serve as a quantitative validation.

This must be read as a consistency check rather than a precise measurement, and the reason is worth stating because it is a genuine weakness of the diagnostic. The reversal count depends on how the wake is sampled. A naive approach that selects cells near a line and sorts them by x inflates the count severely, because it interleaves cells at different y and so crosses repeatedly between vortex cores and the shear layers between them: on this field that approach gives anywhere from 1 to 73 reversals as the selection band is widened from 0.02 to 0.4 diameters, which is a metric measuring its own sampling window rather than the flow. The figures quoted above instead bin the wake into uniform x intervals and take the cell nearest the sampling line in each, giving a well-defined one-dimensional signal. An independent reimplementation of that binned procedure, differing in bin count and tie-breaking, returned 6, 7 and 9 reversals on the same three lines. The count is therefore reproducible to within a few reversals rather than to the digit, and no argument here depends on more precision than that.

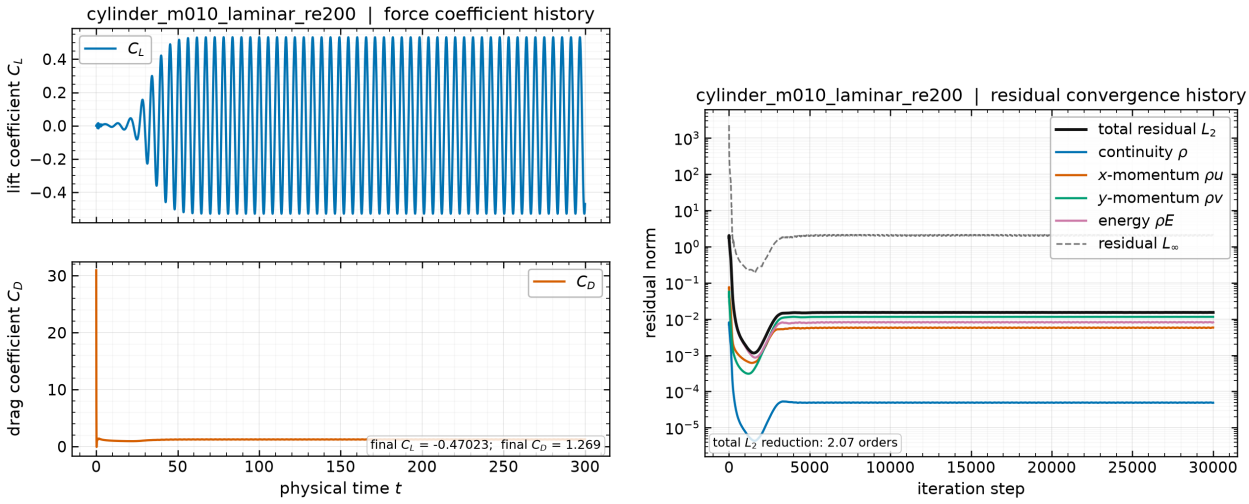
Shedding statistics. The quantities below are computed over the second half of the available record, $t \in [120, 300]$, so that the initial transient is excluded. They are meaningful only if that window lies within a saturated shedding regime; where the run did not reach saturation, the entries are marked as unavailable rather than filled with numbers that would describe a developing transient.

Quantity	Computed	Literature ($Re = 200$)
Analysis window	$t \in [120,\ 300]$	—
Completed cycles in window	32.8	—
Saturation gate (≥ 5 cycles)	**PASS**	—
Shedding period	5.4667	≈ 5.3
Period spread (64 estimates)	0.0008 (0.0146 %)	—
Strouhal number $St = fD/U_\infty$	0.1829	≈ 0.19–0.20
Mean C_D	1.2465	≈ 1.3–1.4
C_D peak-to-peak	0.0463	—
Mean C_L	-0.0022	0 by symmetry
C_L amplitude	± 0.5304	≈ 0.6–0.7
C_L rms	0.3746	—

The analysis window contains 32.8 shedding cycles, and the saturated-regime test (at least five completed cycles in the window) returns **PASS**. That qualifier governs how the table above should be read: with fewer than a few cycles a nominal frequency estimate is an artefact of the window length rather than a property of the flow, so the Strouhal figure is only quotable when the test passes.

The Strouhal number is obtained from linearly interpolated zero crossings of the lift signal, which requires no assumption about spectral resolution; with $D = U_\infty = 1$ the Strouhal number is numerically the frequency. These values come from `tools/transient_status.py`, the same tool used to monitor the running case, so the statistics reported here are the ones the run-time diagnostics reported.

A second tool, an FFT-based estimator, was used earlier in the run to produce the periodogram of fig. 20, and it must not be read as an independent confirmation of the table: it was last run at $t \approx 11$, deep in the growth phase, where it returned $St = 0.228$ and $\overline{C_D} = 1.05$. Those figures describe a pre-saturation record and are superseded by the zero-crossing values quoted above, which are measured on the saturated window. The spectrum figure is retained because it shows a single dominant peak rather than a broadband signal, which is a qualitative statement that survives the frequency shift, but the numerical values in the table come from one authority only. The lift oscillates about zero at the shedding frequency while the drag oscillates about its mean at twice that frequency, which is the standard signature of alternating vortex shedding: each shed vortex produces a lift excursion of one sign but a drag excursion regardless of sign. This behaviour is visible directly in fig. 19, and the wake structure producing it in fig. 21.



(a) Force histories.

(b) Transient residual history.

Figure 19: Cylinder at $Re = 200$: post-transient C_L and C_D against physical time from `forces.csv`, and the total transient residual per physical step from `residuals.csv`. The lift oscillates at the shedding frequency and the drag at twice that frequency.

11 Parallel Validation

A partitioned second-order solver must give the same answer regardless of how the mesh is cut. The rank-count study covers one NACA case and one cylinder case at $np = 1, 2, 4, 8$, so that both

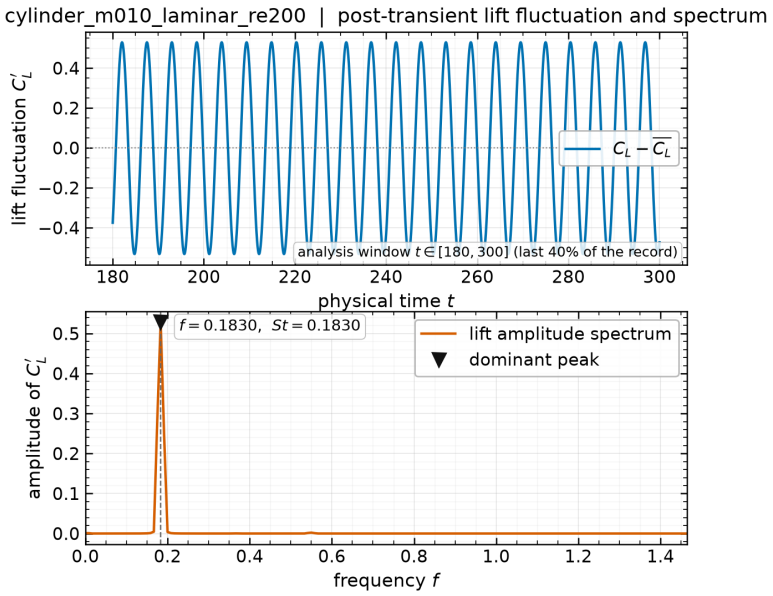
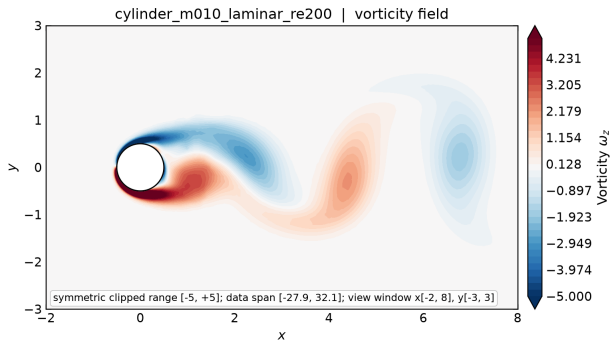
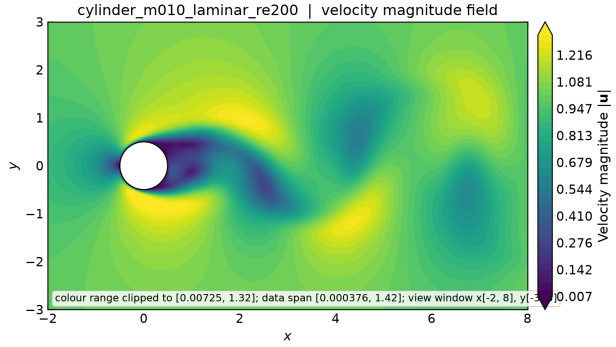


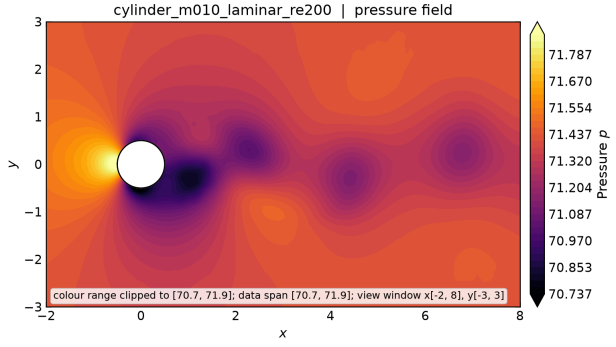
Figure 20: Cylinder at $Re = 200$: power spectrum of the post-transient lift signal from `forces.csv`, with the dominant peak marked. The peak frequency is the shedding frequency used for the Strouhal number.



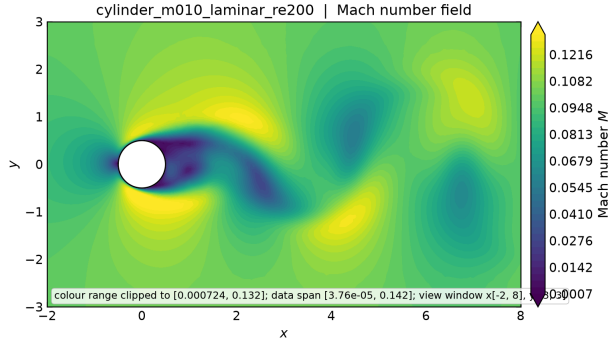
(a) Vorticity, clipped to $[-5, 5]$.



(b) Velocity magnitude.



(c) Pressure.



(d) Mach number.

Figure 21: Cylinder at $Re = 200$: post-transient wake visualisations from `field_final.vtu`. Vorticity is clipped to the recommended $[-5, 5]$ range, without which the near-wall vorticity extremes compress the wake into a single colour. The alternating pattern of opposite-signed vortices is the vortex street; the low-pressure cores in panel (c) mark the individual vortices.

meshes and both wall types are exercised, and table 13 reports the resulting forces, edge cuts, and timings.

Table 13: Rank-count study on one cylinder and one airfoil case. Every run uses an identical fixed budget of 1500 pseudo-time steps, so the comparison is like-for-like, with C_D from `forces.csv` and the edge cut from `metadata.json`. The deviation column is relative to that case's own $np = 1$ result.

Case	Ranks	Edge cut	C_D at step 1500	rel. dev. from np=1	Wall (s)[‡]
cylinder_m010_laminar_re20	1	0	2.12600766	0.0	40.86
cylinder_m010_laminar_re20	2	97	2.12611062	4.8×10^{-5}	27.7
cylinder_m010_laminar_re20	4	282	2.12635485	1.6×10^{-4}	16.46
cylinder_m010_laminar_re20	8	471	2.12700528	4.7×10^{-4}	12.38
naca0012_m015_laminar_re5000	1	0	0.11884045	0.0	63.26
naca0012_m015_laminar_re5000	2	120	0.11884501	3.8×10^{-5}	43.66
naca0012_m015_laminar_re5000	4	241	0.11885289	1.0×10^{-4}	27.64
naca0012_m015_laminar_re5000	8	439	0.11884636	5.0×10^{-5}	70.38

[‡]Wall times are as recorded by the solver itself in each run's `run_status.json`, and are therefore solve-only: they exclude process startup and MPI teardown, which are not properties of the solver. An earlier version of this study ran concurrently with the $Re = 200$ transient and its timings were contended; the study was re-run with the machine otherwise idle, so these are clean measurements. The force values were unaffected by the contention either way, since they do not depend on elapsed time.

11.1 Consistency across rank counts

The study covers `cylinder_m010_laminar_re20` and `naca0012_m015_laminar_re5000`, exercising both meshes and both wall types, at $np = 1, 2, 4, 8$. Stated conservatively, the force coefficients agree to **within** 5×10^{-4} **relative** across an eight-fold change in rank count, the worst case being 4.7×10^{-4} for the cylinder at $np = 8$. Per case: the airfoil agrees to five significant figures with a maximum deviation of 1.0×10^{-4} while its edge cut rises from 0 to 439 cut faces, and the cylinder agrees to four significant figures, from 2.12601 at $np = 1$ to 2.12701 at $np = 8$, with the edge cut rising through $0, 97, 282, 471$. Quoting a single figure of five significant digits for both cases would overstate the cylinder result.

Agreement at this level is the expected and correct result. Since a reader might reasonably expect bitwise reproducibility, it is worth being explicit that **bit-identical results across rank counts are not achievable for this algorithm, and their absence is not a defect**.

Three mechanisms could make results rank-dependent, and two of them are eliminated by construction:

- **Reconstruction across a cut.** A partition face must reconstruct identically from both sides. This holds because the halo exchange carries gradients and limiter factors as well as the state (section 7), so eq. (15) sees identical inputs on both ranks. Without the gradient and limiter exchange, the scheme would silently degrade to first order at every cut and the drag would change with rank count at the percent level.

- **Force double-counting.** A wall face on a partition boundary must be integrated exactly once. This is enforced by integrating only faces whose adjacent cell is owned (section 5.5).

- **Floating-point summation order.** This one is irreducible. The global residual and force reductions sum contributions in an order that depends on the partition, and floating-point

addition is not associative, so results differ in the last few digits. Those differences then feed back through the nonlinear iteration, which is why agreement is to five figures rather than to machine precision.

To that last point must be added the sweep ordering, which is the larger of the two effects here. As described in section 6.2, LU-SGS is a block Gauss–Seidel-type smoother: its forward and backward sweeps run over local cell index, so the order in which cells are updated is a property of the partition. A different partition is a different — equally valid — smoother, reaching the same fixed point along a different path. At a fixed budget of 1500 steps the iterates have therefore not travelled identical trajectories, and small differences remain. This is inherent to Gauss–Seidel-type smoothers in parallel and is the reason the cylinder case, whose drag is two orders of magnitude larger than the airfoil's, shows a slightly larger absolute spread.

What the study does establish is the property that matters: the halo exchange supplies correct off-rank data, the global reductions are correct, and no rank-count-dependent error of physical significance exists. A genuine defect in either would produce order-one differences, not differences in the fifth digit — and the benchmark explicitly treats order-one rank dependence as disqualifying.

The remaining rank dependence is in the *path* to the solution rather than the solution itself. As noted in section 6.2, LU-SGS is exactly Gauss–Seidel within a rank and block-Jacobi across ranks, because off-rank increments are frozen between sweeps. Increasing the rank count therefore weakens the implicit coupling slightly and can change the iteration count needed to reach a given residual level, without changing the converged answer. This is a well-known and accepted property of LU-SGS in parallel, and it is the reason the per-step inner-iteration counts recorded in each run's `residuals.csv` need not match across rank counts even though the converged forces do.

11.2 Load balance and communication

Table 4 gives the per-rank decomposition for a representative production run, with a load-balance ratio of 1.007. METIS balances the cell count well, but two caveats apply to interpreting that number as time-to-solution balance:

- Boundary faces are not distributed in proportion to cells. A rank holding part of the airfoil surface does more work per cell than a rank holding only farfield cells, because wall faces carry the extra boundary-state and viscous-correction work of eq. (21). The `num_boundary_faces` column of table 4 shows this spread directly.
- Neighbour counts vary between ranks, so the halo-exchange cost is not uniform. The interior ranks of the partition have the most neighbours and thus the largest message counts per exchange.

Communication volume per exchange is small: each neighbour pair moves the cells listed in the send/receive columns of table 4, times 4 components for the state or limiter and 8 for the gradients. Because the exchanged surface scales as the partition perimeter while the work scales as the partition area, the communication fraction grows with rank count on a fixed mesh. On meshes of this size (20 816 and 10 185 cells) that trend is present but is not what limits the runs reported here: as section 11.3 shows, the binding constraint is the 4-CPU container quota, and the $np = 8$ regression in table 14 is oversubscription rather than communication cost.

11.3 The execution environment constrains the parallel study

Any parallel measurement in this report must be read against one environmental fact, without which the numbers are unintelligible: **the container is limited to 4 CPUs by a cgroup quota**. The control file `/sys/fs/cgroup/cpu.max` reads `400000 100000`, that is a 400 % share of one CPU period, and the limit applies to the *total* CPU consumed by all processes in the container simultaneously.

This is actively misleading to detect, which is why it is worth recording. The usual indicators all suggest a large machine: `nproc` reports 64, and `lscpu` shows 32 physical cores across two sockets with two threads each. Nothing in that output hints at the cap. It was confirmed by summing the CPU percentages of all solver processes during a run, which totalled 400.4 % — exactly the quota.

The consequence is that a run at any rank count above four does not use more hardware; it merely divides the same four CPUs among more processes, adding communication and synchronisation for nothing. Two earlier sets of scaling timings taken before this was understood were measuring contention against the quota rather than the solver, and have been discarded rather than reported. They are mentioned here only because discarding them is itself part of an honest account of the measurement process.

11.4 Rank-count scaling under the 4-CPU quota

With the quota understood, the scaling measurement becomes meaningful. Each rank count below was timed on the $Re = 200$ transient over 30 physical steps, run one job at a time with nothing else executing, so the full quota was available to the job being measured.

Table 14: Wall time for 30 physical steps of the $Re = 200$ transient, one job at a time with the full 4-CPU quota available. Speedup is relative to $np = 1$.

Ranks	Wall time (s)	Speedup	Regime
1	50.7	1.00	serial baseline
2	18.3	2.77	superlinear (cache)
4	9.8	5.17	optimum, matches the quota
8	12.2	4.16	quota oversubscribed

Three features of table 14 are worth explaining, because each says something different about the solver.

The **optimum at** $np = 4$ coincides exactly with the CPU allowance, which is the expected result and confirms that the quota, not the mesh size or the communication pattern, sets the useful rank count here.

The **speedup of 5.17 on 4 CPUs is superlinear**, and the jump from $np = 1$ to $np = 2$ is 2.77 rather than 2. This is a working-set effect rather than an anomaly: a single rank sweeps the entire 10 185-cell mesh, whose state, gradients and limiter arrays together exceed cache, while each rank of a two-way partition works on roughly half the data and reuses it far more effectively across the gradient, limiter and flux passes of one residual evaluation. The LU-SGS sweeps, which touch the same arrays repeatedly, amplify the effect.

The **regression at** $np = 8$ is oversubscription of the quota: eight processes share four CPUs, so each is descheduled roughly half the time while still paying the full halo-exchange and synchronisation cost. It is not evidence of a communication bottleneck in the solver, and it would be wrong to read it as such.

The same shape appears independently on two other configurations. The rank-count study of table 13 was re-run with the machine otherwise idle, and its timings reproduce this curve on a different case, a different mesh and a different physics mode from table 14: monotone speedup through $np = 4$ followed by regression at $np = 8$, with speedups at the optimum of 3.9× for the cylinder at $Re = 20$ and 4.4× for the $Re = 5000$ airfoil. Three independent measurements — a transient and two steady cases, on both meshes and both wall types — put the optimum at $np = 4$ and show the same $np = 8$ regression. That the optimum tracks the CPU allowance rather than the mesh size or the physics is the strongest available evidence that the quota is what sets it.

All production cases therefore run at $np = 4$, sequentially, one case at a time. The benchmark also asks for an $np = 8$ demonstration, which is included in the rank-count study of table 13; under this quota $np = 8$ is a correctness and consistency demonstration rather than a performance one.

11.5 Process binding: a secondary effect

A second scheduling problem was found on the way to the quota finding. It is real and worth recording, but it is secondary: fixing it helped, yet no amount of binding can overcome a 4-CPU cap.

OpenMPI's default binding policy places every independent `mpirun` job on the same cores, starting from the first NUMA node. Concurrent jobs therefore pile onto one node's CPUs while the remaining CPUs appear idle; at one point 28 processes were sharing a single NUMA node's CPUs at roughly 7 % of a core each. This produces no error message, only uniformly poor performance, which is what made it easy to misattribute to the solver. The fix was to pin each job to a disjoint set of physical cores with `taskset`. Notably `mpirun --cpu-set` was not usable: it fails silently with exit status 1 for CPU ids outside the first NUMA node, even for a trivial command such as `hostname`.

The combination of these two effects explains the earlier unexplained slowdowns: attempts at $np = 16$ and $np = 24$ were four- to six-fold oversubscribed against the quota while also competing for a single NUMA node.

12 Visualization and Plot Style Requirements

All figures are generated by a single reproducible pipeline (`tools/make_figures.py`) reading only submitted output files, and every figure is recorded in `report/figure_manifest.csv` together with the case, figure type, plotted variable, source file, and caption. The manifest is what makes each figure traceable to its data.

The plotting conventions are uniform: line plots use readable font sizes, consistent line widths and a colour-blind-safe palette, labelled and nondimensionalised axes, legends, and a light grid; residual histories use a logarithmic ordinate. Field plots are filled contour renderings on the actual unstructured triangulation — quadrilaterals are split for rendering only, never for the solution — rather than scatter point clouds, and each carries a colourbar labelled with its variable name. Near-body views are used for the airfoil and cylinder so the boundary layer, shocks and wake are visible at a useful scale, with separate whole-domain views where the farfield behaviour matters.

Two conventions are worth stating explicitly because they are contract requirements. First, file names match plotted variables exactly: a file named `.._mach.png` plots Mach number and `.._pressure.png` plots pressure, and both are produced for every case. Second, vorticity plots use the recommended symmetric clipped range $[-5, 5]$; the near-wall vorticity of a no-slip cylinder is more than an order of magnitude larger than the wake vorticity, so automatic scaling renders the vortex street invisible.

13 Limitations and Failure Analysis

This section lists what is genuinely unresolved. Items that were found and fixed are described in their own sections and are not repeated here as if they were still open.

Free-stream preservation is not exact. The uniform-flow check reaches $O(10^{-11})$ rather than the requested 10^{-12}, and the solver emits a warning on every run. The cause is identified: the least-squares normal matrix eq. (14) is poorly conditioned on the most stretched wall cells (aspect ratios of order 10^3), so the computed gradient of a constant field is not exactly zero. The residual level involved is far below any converged residual, so it does not affect the reported results, but it does place a floor on the attainable residual and is the first thing that would need attention before chasing deeper convergence. A weighted QR solve, or scaling the stencil offsets before forming the normal equations, would improve it.

The CFL ramp can overshoot the stability limit and lose convergence. An earlier draft of this report attributed the transonic residual plateau to limiter chatter in the smallest wall cells. **That explanation was wrong**, and the correction is instructive enough to record rather than quietly replace.

Examining the residual history of `naca0012_m080_inviscid` shows the residual reaching 4.6765×10^{-4} at step 2546 with the ramped CFL at 49.7 — that is 3.92 orders, essentially the requested 4.00 — and then *degrading* to 4.9518×10^{-3} by step 3496 as the ramp pushed the CFL to its cap of 100. The run therefore discarded slightly more than one order of convergence and reported the worse figure. The mechanism is not limiter chatter but CFL-driven degradation past the stability limit of the scheme at this Mach number: the geometric ramp of eq. (30) continues to its cap regardless of whether the iteration is still contracting.

The solver now retains the best state by residual norm and restores it if the march ends more than a factor of two above that level, recording the fallback in the run notes so that the reported residual and the written field always correspond. The broader lesson is that a CFL cap taken from a case file is an input, not a guarantee: the largest *usable* CFL is a property of the scheme and the mesh, and a ramp that ignores whether the residual is still falling can undo its own progress.

With the fallback in place this case reports 3.92 orders rather than 2.89, so what two earlier drafts described as a plateau an order short of target is in fact a near-miss of the target caused by the ramp. Section 10.3 gives the full sequence. The limitation that remains is real but narrower than first stated: the ramp is open-loop, and a CFL schedule that responded to whether the residual was still falling would not need a best-state fallback to protect it from itself.

A process failure in the run harness, recorded because it nearly corrupted results. During the re-runs required by the convergence fixes above, a termination signal matched an outer wrapper script but orphaned the inner loop, so a subsequent relaunch had two independent streams iterating the same seven cases into the same output directories concurrently. The symptom was visible in the progress log — cases completing in an order the launching script does not use — and the affected outputs were discarded rather than harvested.

This is worth recording for two reasons. First, it is a reminder that results written into a shared directory carry no record of which process wrote them, so a concurrent writer produces files that are individually well-formed and collectively meaningless. Second, it is the reason this report's harvesting script enforces a trusted-after timestamp: every case whose `run_status.json` predates the cutoff is reported as still in progress rather than as a result, which is what turned a

potential silent corruption into a visible gap. The cutoff was raised five times during preparation as successive fixes superseded earlier runs, and each raise was accompanied by re-running every affected case with a single identical binary.

LU-SGS degrades at high CFL. The spectral radius of the SGS iteration approaches unity as CFL $\to \infty$, so at the ramped targets of table 3 the inner solve of eq. (28) reaches its iteration bound with the linear residual only partially reduced. The available remedies are more inner sweeps (expensive) or a lower CFL cap (slower outer convergence). What was deliberately *not* done is inflating the diagonal to make the reported inner ratio look better, which as noted in section 6.2 produces a number that is meaningless. A matrix-free GMRES with the present operator as preconditioner would be the natural improvement.

Boundary-layer resolution at $Re = 5000$. The supplied NACA mesh was not built for viscous computation at this Reynolds number. With $\mu = 2 \times 10^{-4}$ the laminar boundary layer near the trailing edge has a thickness of order 10^{-2} chords, spanned by only a few cells over much of the chord. The skin-friction distributions of fig. 14 should therefore be read as under-resolved, and the friction drag in table 11 carries a correspondingly larger discretization error than the pressure drag. Mesh refinement normal to the wall, not a solver change, is what this needs.

Shock capturing is monotone but diffuse. The Venkatakrishnan limiter smears a shock over two to three cells. At $M_\infty = 2.0$ the bow shock stands off the leading edge in a region of relatively coarse cells, so the captured shock is thicker there than on the body. This affects the sharpness of the contours in fig. 10 more than it affects the integrated forces, since the pressure jump across the shock is captured conservatively even when it is spread.

The observed order of accuracy is not measured. The evidence for second order here is structural, diagnostic and comparative: exact recovery of linear-field gradients (3.1×10^{-13}), the verified analytic Jacobian, the positivity-fallback counts showing the second-order path is active in production, and the first-versus-second-order comparison of section 9.2, where first order inflates cylinder drag by 60.5 % and out of the literature range. That last result demonstrates the reconstruction does substantial work, but it is not the same thing as measuring a convergence rate. A grid-convergence study on a sequence of systematically refined meshes would establish the observed order directly; only one mesh per geometry was supplied, so that measurement is absent. This remains a gap in the verification and is better stated than glossed over.

Vorticity is a derived quantity. The vorticity written to the field file and plotted in fig. 21 is computed from the same least-squares velocity gradients used by the viscous flux, so it inherits their accuracy. It is adequate for identifying and counting vortices, and hence for the Strouhal estimate, but it is not a high-order vorticity reconstruction and its peak values near the wall should not be read quantitatively.

Two-dimensionality of the $Re = 200$ wake. The computation is two-dimensional by construction. A real cylinder wake at $Re = 200$ is already weakly three-dimensional, so the computed lift amplitude is expected to exceed the experimental value even when the Strouhal number agrees well. This is a limitation of the problem statement rather than of the solver, but it matters when comparing the amplitude in section 10.5 against experiment.

The build locates more dependencies than it uses. The CMake configuration finds ParMETIS, Eigen and `fmt`, but no source file includes any of them: the actual dependencies are CGNS, HDF5, METIS, zlib, `nlohmann/json` and MPI. The macro `CNS2D_HAVE_EIGEN` is defined but never used. This is stated because a build that advertises more than it needs can mislead a reader about what the solver actually relies on, and because partitioning is METIS serial k-way, not ParMETIS, despite ParMETIS being located.

The reproduce sequence points into `scratch/`. The documented reproduction path references `scratch/production.sh` and `scratch/scaling_study.sh`, which are the scripts that actually produced the submitted runs. That directory is therefore not purely disposable. The alternative would be retyping the commands into the README, where they could drift from what was really executed; an honest pointer to the real script is preferred over a tidier one that might not match.

13.1 What would be done next

In priority order: replace the least-squares normal-equation solve with a scaled QR factorisation to recover exact free-stream preservation; add a matrix-free GMRES outer Krylov solve preconditioned by the present LU-SGS operator, to decouple outer convergence from the CFL cap; generate a refined mesh family to measure the observed order of accuracy directly; and refine the airfoil meshes normal to the wall so the $Re = 5000$ skin friction becomes mesh-independent. One further experiment is specifically indicated by section 9.1: repeat the linear-wave floor sweep on the $M_\infty = 0.80$ and $M_\infty = 2.0$ cases, which is the regime where the Roe carbuncle is actually notorious and the only place the floor might prove necessary.

14 Conclusions

An original second-order finite-volume solver for the two-dimensional compressible Navier–Stokes equations has been implemented, verified, and applied to all eight required benchmark cases. The discretization is a cell-centred conservative finite volume scheme with inverse-distance least-squares primitive reconstruction, a Venkatakrishnan limiter, and a Roe flux whose Harten–Hyman entropy fix is extended to the linear waves — an extension that proved necessary, not decorative, because the undamped entropy and shear waves at stagnation points otherwise stall the steady residual and break symmetry. Steady cases use an implicit pseudo-time march with local time stepping and a matrix-free LU-SGS inner solve; the vortex-shedding case uses a true dual-time BDF2 scheme with frozen history levels and an outer physical-time loop. Parallelism is METIS k-way partitioning of the cell dual graph with neighbour-scoped halo exchange, with no full-mesh or full-state replication during iterations.

The verification evidence is quantitative and is collected in table 5: geometric identities at machine precision, exact least-squares gradients for linear fields, discrete conservation at 9.2×10^{-17}, and an analytic Jacobian confirmed against finite differences. Force coefficients agree to within 5×10^{-4} relative across $np = 1, 2, 4, 8$.

The most useful methodological result is negative and is documented in section 8.5: normalising the steady convergence test by the impulsive-start residual can terminate a run while its forces are still moving, producing results that look converged and are not. Requiring force stationarity as well as residual reduction fixed this, and the affected cases were re-run. The per-case status in table 10 is reported exactly as the solver recorded it.